

UNIVERSITÀ DEGLI STUDI DELLA BASILICATA
DIPARTIMENTO DI SCIENZE DI BASE E APPLICATE

Appunti delle lezioni ed esercizi rielaborati

Calcolo Scientifico I

Studente:
Donato Martinelli
Matr. 69060

Docente:
Prof.ssa **Maria Carmela De Bonis**

Anno Accademico 2025/2026

Indice

1	Introduzione	5
2	Rappresentazione dei Numeri in un Calcolatore	7
2.1	Errori	13
2.2	Operazioni Macchina e Cancellazione Numerica	19
2.3	Propagazione degli Errori Introdotti nei Dati Iniziali	21
3	Risoluzione di un Sistema Lineare Quadrato	25
3.1	Il Problema della Risoluzione di un Sistema Lineare	25
3.2	Sistemi Diagonali e Triangolari	35
3.3	Metodo di Eliminazione di Gauss	44
3.4	Strategia Pivoting e Fattorizzazione LU	47
3.5	Metodo di Cholesky	57
3.6	Stabilità dei Metodi di Gauss	61
3.7	Esercizi	68
4	Metodi Numerici per la Risoluzione di un Sistema Lineare Rettangolare nel Senso dei Minimi Quadrati	83
4.1	Esercizi	89
5	Metodi Numerici per la Risoluzione di una Equazione non Lineare	93
5.1	Risoluzione di Equazioni non Lineari	93
5.2	Risoluzione di Equazioni Algebriche	100
5.3	Esercizi	110

1 Introduzione

Il Ruolo della Matematica e la Modellizzazione La matematica è uno strumento indispensabile per l'interpretazione e la predizione dei fenomeni reali, ponendosi alla base di tutte le scienze. Gli scienziati cercano di ricavare un modello matematico (variabili, parametri, relazioni) che sia rigoroso e coerente con il fenomeno. La complessità del fenomeno determina la quantità di dati necessari e il numero di variabili del modello. Le proprietà incognite si deducono risolvendo tale modello.

Necessità dell'Approccio Numerico Spesso la soluzione non è disponibile in forma esplicita o non è calcolabile analiticamente, rendendo necessario un metodo numerico di approssimazione. Anche quando la risoluzione analitica è possibile, essa può risultare troppo onerosa computazionalmente all'aumentare delle dimensioni del problema.

Metodo Numerico

Definizione 1.1. Un metodo numerico è un processo implementabile su un calcolatore che fornisce una soluzione numerica in un numero finito di passi.

Esistono metodi matematicamente validi che però non sono implementabili in un calcolatore per via dei tempi di esecuzione.

Caso Studio: Sistemi Lineari (Cramer vs Gauss) Un esempio cruciale riguarda la risoluzione di sistemi lineari, problema centrale nella modellistica.

- **Il Metodo di Cramer (Non implementabile):** Se volessimo risolvere un sistema 20×20 ($n = 20$) con Cramer, dovremmo calcolare 21 determinanti di ordine 20. Con la regola di Laplace, le operazioni sono circa $21 \cdot 20! \approx 5.2 \cdot 10^{19}$. Su un processore da 3GHz ($3 \cdot 10^9$ op/sec), il tempo richiesto sarebbe superiore a 5 secoli. È una procedura con tempo di esecuzione proibitivo.
- **Il Metodo di Eliminazione di Gauss (Efficiente):** Questo metodo richiede circa $\frac{n^3}{3}$ operazioni. Per $n = 20$, servono solo 2667 operazioni ($0.88 \cdot 10^{-6}$ secondi). Per $n = 1000$, servono 334 milioni di operazioni, eseguibili in un decimo di secondo.

Criteri di Scelta ed Errori La scelta del metodo si basa su due criteri fondamentali:

1. **Efficienza:** minor numero di operazioni richieste.
2. **Efficacia:** maggiore precisione di calcolo.

Nonostante la potenza dei calcolatori moderni, i matematici affrontano ancora difficoltà. Poiché i numeri sono rappresentati in binario su una memoria finita, ogni memorizzazione introduce un errore.

Propagazione dell'Errore Un tema centrale del Calcolo Scientifico è la consapevolezza dell'errore e della sua propagazione. Alcuni metodi propagano gli errori (introdotti nei dati iniziali) al punto da fornire soluzioni completamente sbagliate. Si studia quindi la stabilità degli algoritmi rispetto a questi errori.

2 Rappresentazione dei Numeri in un Calcolatore

Sistemi di Numerazione Il numero è un concetto che esiste indipendentemente dall'insieme dei simboli o segni che si utilizzano per rappresentarlo. Un sistema di numerazione non è altro che uno schema che permette di rappresentare i numeri "ideali" attraverso un insieme di simboli per i quali vengono definite delle tecniche di manipolazione (operazioni aritmetiche). E' indubbio che tutti i popoli impararono a contare, spinti dalle necessità della vita quotidiana, per quantificare un insieme di elementi. Gli uomini primitivi limitarono la propria conoscenza ai primi numeri naturali, per indicare i quali ricorsero a nomi di oggetti concreti, io per indicare 1, ali per indicare 2, mano per indicare 5. In seguito incominciarono a rappresentare i numeri con delle barrette verticali. Ogni intero era rappresentato con un numero di barrette pari alle unità in esso contenute. Questo sistema benché sia il più semplice possibile, presenta lo svantaggio di assegnare a ciascun numero una rappresentazione la cui lunghezza è proporzionale al numero stesso. Così, ben presto, si ebbe la necessità di rappresentare l'infinità dei numeri con un numero limitato di segni particolari, detti cifre e di fissare alcuni numeri fondamentali che facevano da riferimento. Nacque così l'idea di quella che viene chiamata base di numerazione. Quasi tutti i popoli scelsero come base di numerazione il 10. Scelta dovuta sicuramente al numero delle dita delle mani. I sistemi di numerazioni delle civiltà conosciute, a partire dalle popolazioni primitive, si classificano in:

- sistema di numerazione additivo
- sistema di numerazione posizionale

Un sistema di numerazione additivo è un sistema di numerazione basato su una legge additiva applicata a determinati simboli numerici fondamentali. Ogni numero è rappresentato attraverso una successione di tali simboli ed il suo valore è dato dalla somma dei valori attribuiti a ciascuno di essi. Nei sistemi additivi non serve un simbolo per lo zero. Tra i più famosi sistemi additivi, oltre a quello delle barrette, figurano quello romano e quello egizio. Solo verso il X secolo il sistema di numerazione additivo incominciò ad essere soppiantato dal sistema di numerazione posizionale introdotto dai matematici indiani ed arabi. Un sistema di numerazione si dice posizionale se i simboli usati per scrivere i numeri assumono valori diversi a seconda della posizione che occupano nella notazione.

Sistema decimale Il sistema di numerazione da noi comunemente usato per rappresentare i numeri è quello decimale. Esso è un sistema di numerazione posizionale basato sull'impiego di 10 cifre 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ed è stato inventato dagli Indiani. Agli Indiani si deve l'introduzione dello zero che essi chiamavano sunya (nulla) anche se alcuni studiosi affermano che già i Cinesi conoscevano il metodo posizionale fin da tempi antichissimi. Questo sistema di numerazione venne introdotto in occidente dagli Arabi probabilmente nel XII secolo quando venne tradotta *Algoritmi de numero Indorum* del grande matematico arabo al-Khuwarizmi (IX secolo). La base del sistema decimale è 10 e

ogni numero viene rappresentato come

$$a = \pm a_m a_{m-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots a_{-M}$$

con

$$0 \leq a_i \leq 9$$

Il sistema è posizionale perché il valore di ogni cifra varia in funzione della sua posizione nella rappresentazione decimale del numero

$$z = \pm a_m 10^m + a_{m-1} 10^{m-1} + \dots + a_1 10^1 + a_0 10^0 + a_{-1} 10^{-1} + a_{-2} 10^{-2} + \dots$$

La rappresentazione decimale di ogni numero reale è unica, eccetto quando la parte frazionaria contiene una sequenza di 9 consecutivi.

Sistema in base N Qualunque intero $N > 1$ può essere scelto come base ed ogni numero reale a può essere scritto nella forma

$$z = \pm a_m N^m + a_{m-1} N^{m-1} + \dots + a_1 N^1 + a_0 N^0 + a_{-1} N^{-1} + a_{-2} N^{-2} + \dots$$

ovvero

$$a = \pm a_m a_{m-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots a_{-M}$$

con $0 \leq a_i \leq N - 1$. La rappresentazione di ogni numero reale in base N è unica, eccetto quando la parte frazionaria contiene una sequenza di cifre $a_k = N - 1$ consecutive. Più piccola è la base scelta, più è lunga la stringa di caratteri necessari per rappresentare lo stesso numero.

Sistema binario La base del sistema binario è 2. Le cifre utilizzate da questo sistema sono 0 e 1 e vengono dette bit da *binary digit*. Ogni numero reale a è rappresentato come una sequenza di 0 e 1, ovvero

$$a = \pm a_m 2^m + a_{m-1} 2^{m-1} + \dots + a_1 2^1 + a_0 2^0 + a_{-1} 2^{-1} + a_{-2} 2^{-2} + \dots$$

con $0 \leq a_i \leq 1$. Questo sistema è particolarmente interessante perché può essere realizzato con qualsiasi oggetto capace di assumere due stati diversi, uno per la cifra 0 e l'altro per la cifra 1.

Per queste sue caratteristiche è stato adottato per la rappresentazione dei dati e, in particolare, dei numeri in un calcolatore.

Rappresentazione dei numeri in un calcolatore I numeri vengono rappresentati nel calcolatore secondo il sistema binario e, quindi, come una sequenza di bit. Per ovvie ragioni, ad ogni numero reale viene riservato uno spazio finito di memoria, capace di contenere un numero finito di bit detto **parola**. Di conseguenza, non tutti i numeri reali sono rappresentabili in modo esatto. Detta l la lunghezza della parola, si possono rappresentare esattamente solo quei numeri la cui rappresentazione binaria consta di un numero di bit inferiore o uguale ad l . Si parla di **numeri macchina**. Tutte le operazioni fra i numeri macchina vengono effettuate utilizzando l'aritmetica binaria.

Numeri interi Con parole di lunghezza l è possibile rappresentare tutti i numeri interi appartenenti all'intervallo

$$\left[-\frac{2^l}{2}, \frac{2^l}{2} - 1 \right]$$

Esempio: Se $l = 16$ sono rappresentabili tutti i numeri interi appartenenti all'intervallo

$$[-32768, 32767]$$

Rappresentazione in virgola mobile (floating point)

Definizione 2.1. Ogni numero reale a può essere scritto nella forma

$$a = pN^q$$

dove p è un numero reale detto **mantissa** di a , N è la base del sistema di numerazione e q è un numero intero positivo o negativo detto **esponente** di a . La rappresentazione di a si dice **normalizzata** quando

$$N^{-1} \leq |p| < 1$$

Le cifre di p si dicono **cifre significative**. Fissata la base N , ogni numero reale a è univocamente definito dalla coppia

$$a = (p, q)$$

Questa rappresentazione, detta in virgola mobile (floating-point), non è unica, infatti

$$321.25 = 32.125 \times 10^1 = 0.32125 \times 10^3$$

I numeri reali in virgola mobile vengono rappresentati in un calcolatore in forma normalizzata secondo lo Standard IEEE 754 (Institute of Electrical and Electronical Engineering).

Standard IEEE 754: Singola Precisione

Definizione 2.2. La struttura è definita dai seguenti parametri:

$$s = 1 \text{ bit}$$

$$p = 23 \text{ bit}$$

$$q = 8 \text{ bit}$$

È possibile rappresentare tutti i numeri reali della seguente forma:

$$\pm 0.d_1d_2d_3d_4d_5d_6 \times 10^q$$

dove:

$$0 < d_1 \leq 9, \quad 0 \leq d_i \leq 9, \quad i \in \{2, \dots, 6\}$$

e

$$-38 \leq q \leq 38$$

Si hanno **6 cifre significative**.**Standard IEEE 754: Doppia Precisione****Definizione 2.3.** La struttura è definita dai seguenti parametri:

$$s = 1 \text{ bit}$$

$$p = 52 \text{ bit}$$

$$q = 11 \text{ bit}$$

È possibile rappresentare tutti i numeri reali della seguente forma:

$$\pm 0.d_1 d_2 \dots d_{16} \times 10^q$$

dove:

$$0 < d_1 \leq 9, \quad 0 \leq d_i \leq 9, \quad i \in \{2, \dots, 16\}$$

e

$$-308 \leq q \leq 308$$

Si hanno **16 cifre significative**.

Supponendo di utilizzare un calcolatore che lavora in doppia precisione, dato

$$a = \pm 0.d_1 d_2 \dots d_{16} \times 10^q, \quad d_1 \neq 0$$

un numero reale non nullo, si possono presentare i seguenti casi:

1. L'esponente q è tale che $-308 \leq q \leq 308$ e le cifre dopo la 16-esima sono tutte nulle ovvero $d_i = 0$ per ogni $i > 16$, cioè

$$a = \pm 0.d_1 d_2 \dots d_{16} 10^q$$

Allora a è **esattamente rappresentabile**.

2. L'esponente q non appartiene all'intervallo $[-308, 308]$. Se $q < -308$, si associa 0 ad a e il calcolatore segnala **underflow**. Se $q > 308$, a non viene rappresentato e il calcolatore segnala **overflow**.
3. L'esponente q appartiene all'intervallo $[-308, 308]$ ma le cifre d_i , $i > 16$, non sono tutte nulle. In questo caso è possibile associare al numero a un numero di macchina $fl(a)$ seguendo due criteri diversi:
 - troncamento
 - arrotondamento

Regola di troncamento Per troncare un numero

$$a = 0.d_1 d_2 \dots d_{t-1} d_t d_{t+1} d_{t+2} \dots N^q$$

fino a t cifre significative, si eliminano tutte le cifre $d_{t+1} d_{t+2} \dots$ a destra della t -esima. Dunque

$$fl(a) = trn(a) = 0.d_1 d_2 \dots d_{t-1} d_t N^q.$$

Regola di arrotondamento Per arrotondare un numero

$$a = 0.d_1d_2 \dots d_{t-1}d_t d_{t+1}d_{t+2} \dots N^q$$

fino a t cifre significative, si eliminano tutte le cifre $d_{t+1}d_{t+2} \dots$ a destra della t -esima e si rimpiazza d_t con la cifra c , dove

$$\begin{cases} c = d_t & \text{se } 0 \leq d_{t+1} \leq 4 \\ c = d_t + 1 & \text{se } 5 \leq d_{t+1} \leq 9 \end{cases}$$

Dunque

$$fl(a) = arr(a) = 0.d_1d_2 \dots d_{t-1}cN^q.$$

Vantaggi della rappresentazione normalizzata Consideriamo il numero

$$a = \frac{1}{7000}$$

Poniamo

$$\bar{a} = 0.000142857142857 \dots \quad (\text{rappresentazione non normalizzata})$$

$$\bar{\bar{a}} = 0.142857142857142 \dots \cdot 10^{-3} \quad (\text{rappresentazione normalizzata})$$

Applichiamo le regole di troncamento e arrotondamento ad $\bar{a}$:

- $t = 6$: $trn(\bar{a}) = 0.000142$, $arr(\bar{a}) = 0.000143$
- $t = 3$: $trn(\bar{a}) = 0$, $arr(\bar{a}) = 0$

Applichiamo le regole di troncamento e arrotondamento ad $\bar{\bar{a}}$:

- $t = 6$: $trn(\bar{\bar{a}}) = 0.142857 \cdot 10^{-3}$, $arr(\bar{\bar{a}}) = 0.142857 \cdot 10^{-3}$
- $t = 3$: $trn(\bar{\bar{a}}) = 0.142 \cdot 10^{-3}$, $arr(\bar{\bar{a}}) = 0.143 \cdot 10^{-3}$

Dunque, la normalizzazione permette di ridurre l'errore di rappresentazione dovuto al troncamento o all'arrotondamento ad un numero finito di cifre. Inoltre, l'informazione contenuta negli zeri dopo il punto può essere memorizzata, con minor spreco di memoria, in termini di esponente.

Conseguenze della rappresentazione dei numeri come parole di lunghezza fissa

Prima conseguenza: Limitatezza dell'insieme dei numeri rappresentabili L'insieme $\mathbb{R}$ dei numeri reali è infinito, ma, soltanto un sottoinsieme di $\mathbb{R}$, limitato inferiormente e superiormente, è rappresentabile in un calcolatore. In seguito denoteremo con $\mathbb{F}$ il sottoinsieme dei numeri reali rappresentabili in un calcolatore.

$$\mathbb{F} = \{0\} \cup \{\pm 0.d_1d_2 \dots d_{16} \times 10^q \in \mathbb{R} : -308 \leq q \leq 308\}$$

$$0 < d_1 \leq 9, \quad 0 \leq d_i \leq 9, \quad i \in \{2, \dots, 16\}$$

Si ha l'intervallo contenente valori rappresentabili tra $-\text{realmax}$ e $-\text{realmin}$, lo zero, e tra realmin e realmax . Al di fuori di questo intervallo si ha overflow o underflow.

Seconda conseguenza: Insieme dei numeri rappresentabili "bucato" L'insieme $\mathbb{R}$ dei numeri reali è denso, cioè tra due numeri reali r_1 e r_2 esiste sempre un numero reale r_3 tale che

$$r_1 < r_3 < r_2$$

Invece, il sottoinsieme $\mathbb{F}$, oltre ad essere limitato inferiormente e superiormente, è anche bucato.

Esercizi in MatLab

- Digitare il numero 0.95842567894152678954126354 e verificare che il programma lo arrotonda a 16 cifre significative fornendo

$$9.584256789415268e - 001 = 0.9584256789415268$$

```
>> 0.95842567894152678954126354
ans =
    9.584256789415268e-001
```

- Digitare il numero 0.00000052135658365124858985 e verificare che il programma lo arrotonda a 16 cifre significative fornendo

$$5.21356583651249e - 007 = 0.521356583651249e - 006$$

```
>> 0.00000052135658365124858985
ans =
    5.213565836512486e-007
```

- Digitare il numero 12356.00056256984154646 e verificare che il programma lo arrotonda a 16 cifre significative fornendo

$$1.23560005625698e + 004 = 0.123560005625698e + 005$$

```
>> 12356.00056256984154646
ans =
    1.235600056256984e+004
```

- Calcolare $\text{realmin} = 2^{-1022} \sim 2.225073858507201e - 308$

```
>> 2^(-1022)
ans =
    2.225073858507201e-308
```

- Calcolare i valori 2^{-k} con $k = 1040, 1050, 1060, 1070$ e osservare che il programma riesce a rappresentare anche numeri con esponente compreso tra -308 e -324 (numeri denormalizzati).

```
>> k = 1040;
>> 2^(-k)
ans =
    8.487983163861089e-314
>> k = 1050;
>> 2^(-k)
ans =
```

```

      8.289046058458095e-317
>> k = 1060;
>> 2^(-k)
ans =
      8.094771541462983e-320
>> k = 1070;
>> 2^(-k)
ans =
      7.90505033459945e-323

```

- Verificare che $2^{-1074} \sim 4.94065645841247e-324$ e che calcolando 2^{-1075} il programma restituisce come valore 0, dunque non riesce a rappresentare numeri con esponente più piccolo di -324.

```

>> k = 1074;
>> 2^(-k)
ans =
      4.940656458412465e-324
>> k = 1075;
>> 2^(-k)
ans =
      0

```

- Verificare che $2^{1023} \sim 8.988465674311580e+307$ e che calcolando 2^{1024} il programma restituisce Inf, dove Inf denota l'overflow.

```

>> k = 1023;
>> 2^k
ans =
      8.988465674311580e+307
>> k = 1024;
>> 2^k
ans =
      Inf

```

2.1 Errori

Cause principali di errori nella risoluzione di problemi matematici con un calcolatore

Nella risoluzione di un problema matematico con un calcolatore le sorgenti di errore possono essere, in generale, suddivise in due gruppi.

1. Errori di rappresentazione dei dati o di arrotondamento

Abbiamo visto che non tutti i numeri reali possono essere rappresentati in un calcolatore. Se il calcolatore utilizza una rappresentazione con t cifre per la mantissa, tutti i numeri reali compresi tra l'estremo inferiore e l'estremo superiore di $\mathbb{F}$ la cui mantissa ha un numero di cifre superiore a t dovranno essere arrotondati o troncati a t cifre.

2. Errori nei calcoli

Effettuando calcoli tra numeri macchina approssimati, gli errori di rappresentazione dei dati iniziali si propagano in qualche misura sui risultati di tali calcoli. L'entità della propagazione dipende anche dall'algoritmo utilizzato.

Errori di rappresentazione dei dati Nel rappresentare un numero reale $a \neq 0$, con un corrispondente numero macchina $fl(a)$ occorre valutare l'errore commesso che sarà ovviamente nullo se a è esso stesso un numero di macchina, cioè $a = fl(a)$. A tale scopo, dato un valore reale esatto a e un sua approssimazione A diamo alcune definizioni.

Errore Assoluto

Definizione 2.4. La quantità

$$\Delta a = |a - A|$$

si chiama errore assoluto.

```

1 function e = errAbs(valExact, valApprox)
2 % ERRABS Calcola l'errore assoluto (norma Euclidea della differenza).
3 % E = ERRABS(VALEXACT, VALAPPROX) restituisce \|valExact - valApprox\|_2.
4 % Supporta scalari, vettori e matrici multidimensionali.
5
6 %% 1. Validazione degli Input
7 arguments
8     valExact double {mustBeNumeric}
9     valApprox double {mustBeNumeric}
10 end
11
12 %% 2. Controllo Dimensioni
13 % isequal è il metodo standard per confrontare le dimensioni di array/
14 % matrici
15 if ~isequal(size(valExact), size(valApprox))
16     error("errAbs:DimensionMismatch", "Dimensioni incompatibili (%d e %d)
17     .", size(valExact), size(valApprox));
18 end
19
20 %% 3. Calcolo Errore Assoluto
21 e = norm(valExact - valApprox);
22 end

```

Function errAbs

Errore Relativo L'errore assoluto non riesce a caratterizzare ciò che intuitivamente si può ritenere una misura della precisione del valore approssimato. Per misurare la bontà di un'approssimazione A di a occorre confrontare l'errore assoluto con l'ordine di grandezza a del dato da approssimare. Cioè fare il rapporto tra l'errore assoluto e il valore assoluto del dato esatto.

Definizione 2.5. La quantità

$$\delta a = \frac{|a - A|}{|a|}$$

si chiama errore relativo.

```

1 function e = errRel(valExact, valApprox, tol)
2 % ERRREL Calcola l'errore relativo in norma 2.
3 % E = ERRREL(VALEXACT, VALAPPROX, TOL) calcola \|valExact - valApprox\| / \|
4 % valExact\|.

```

```

5 %  NOTA: Se ||valExact|| < tol, la funzione restituisce l'errore assoluto
6 %  per evitare divisioni per zero (NaN/Inf).
7
8  %% 1. Validazione degli Input
9  arguments
10     valExact double {mustBeNumeric}
11     valApprox double {mustBeNumeric}
12     tol (1,1) double {mustBeNumeric} = 1e-10
13 end
14
15 %% 2. Calcolo Numeratore (Errore Assoluto)
16 num = errAbs(valExact, valApprox);
17
18 %% 3. Calcolo Denominatore e Gestione Singularità
19 den = norm(valExact);
20
21 if den < tol
22     % Caso critico: La soluzione esatta è zero.
23     % L'errore relativo non è definito. Restituiamo l'assoluto.
24     warning("errRel:ZeroNorm", "La norma della soluzione esatta è minore
della tolleranza. Restituisco Errore Assoluto.");
25     e = num;
26 else
27     e = num / den;
28 end
29 end

```

Function errRel

Cifre corrette Siano

$$a = pN^q \quad \text{e} \quad A = \bar{p}N^q$$

con

$$N^{-1} \leq |p|, |\bar{p}| < 1$$

Cifre decimali corrette

Definizione 2.6. Se

$$\Delta a \leq \frac{1}{2} N^{-t}, \quad t \geq 1,$$

si dice che A ha almeno t cifre decimali corrette nella base N .

Partendo dalla disuguaglianza che lega l'errore assoluto Δa al numero di cifre decimali t :

$$\Delta a \leq \frac{1}{2} \cdot 10^{-t}$$

$$2 \cdot \Delta a \leq 10^{-t}$$

$$\log_{10}(2 \cdot \Delta a) \leq -t$$

$$t \leq -\log_{10}(2 \cdot \Delta a)$$

Poiché t deve essere un intero, si assume il massimo valore che soddisfa la condizione (parte intera inferiore):

$$t = \lfloor -\log_{10}(2 \cdot \Delta a) \rfloor.$$

```

1 function n = cifDecCorr(valExact, valApprox)
2 % CIFDECCORR Calcola il numero di cifre decimali corrette.
3 % n = CIFDECCORR(valExact, valApprox) calcola il numero di cifre
4 % decimali corrette basandosi sull'errore assoluto.
5 %
6 % Input:
7 %   valExact - Valore esatto (numerico: scalare, vettore o matrice)
8 %   valApprox - Valore approssimato (numerico: stesse dimensioni di valExact)
9 %
10 % Output:
11 %   n - Numero intero di cifre decimali corrette.
12 %   Restituisce Inf se valExact e valApprox sono identici (errAbs 0).
13
14 %% 1. Validazione degli Input
15 arguments
16     valExact {mustBeNumeric}
17     valApprox {mustBeNumeric}
18 end
19
20 %% 2. Controllo Dimensioni
21 if ~isequal(size(valExact), size(valApprox))
22     error('cifDecCorr:SizeMismatch', 'Gli input devono avere la stessa
23     dimensione.');
```

```

24 end
25
26 %% 3. Calcolo cifre decimali corrette
27 err = errAbs(valExact, valApprox); % Calcolo l'errore assoluto
28 n = floor(-log10(2 * err));
end

```

Function cifDecCorr

Cifre significative corrette

Definizione 2.7. Se

$$\delta a \leq \frac{1}{2} N^{-t+1}, \quad t \geq 1,$$

si dice che A ha almeno t cifre significative corrette nella base N .

Partendo dalla disuguaglianza che lega l'errore relativo δa al numero di cifre significative t :

$$\delta a \leq \frac{1}{2} \cdot 10^{1-t}$$

$$2 \cdot \delta a \leq 10^{1-t}$$

$$\log_{10}(2 \cdot \delta a) \leq 1 - t$$

$$t \leq 1 - \log_{10}(2 \cdot \delta a)$$

Poiché t deve essere un intero, si assume il massimo valore che soddisfa la condizione (parte intera inferiore):

$$t = \lfloor 1 - \log_{10}(2 \cdot \delta a) \rfloor.$$


```

1 function n = cifSigCorr(valExact, valApprox)
2 % CIFSIGCORR Calcola il numero di cifre significative corrette.
3 % n = CIFSIGCORR(valExact, valApprox) calcola il numero di cifre
4 % significative corrette basandosi sull'errore relativo.
5 %
6 % Input:
7 %   valExact - Valore esatto (numerico: scalare, vettore o matrice)
8 %   valApprox - Valore approssimato (numerico: stesse dimensioni di valExact)
9 %
10 % Output:
11 %   n - Numero intero di cifre significative corrette.
12
13 %% 1. Validazione degli Input
14 arguments
15     valExact {mustBeNumeric}
16     valApprox {mustBeNumeric}
17 end
18
19 %% 2. Controllo Dimensioni
20 if ~isequal(size(valExact), size(valApprox))
21     error('cifSigCorr:SizeMismatch', 'Gli input devono avere la stessa
22     dimensione.');
```

Function cifSigCorr

Unità di roundoff

Teorema 2.1. Denotando con $A = fl(a)$ il numero macchina che si ottiene arrotondando il numero reale a a t cifre significative nella base N , si ha

$$\delta a \leq u$$

dove

$$u = \frac{1}{2}N^{1-t}$$

La quantità u è detta **unità di roundoff**.

Il teorema precedente ci dice che ogni numero può essere rappresentato su un calcolatore con un errore che non eccede u . Infatti, ponendo $\delta = \frac{fl(a)-a}{a}$, si ha

$$fl(a) = a(1 + \delta), \quad |\delta| \leq u$$

La quantità u è una costante propria di ciascuna aritmetica floating-point e rappresenta la massima precisione di calcolo raggiungibile con un calcolatore su cui tale aritmetica è implementata.

Epsilon-macchina Si chiama invece **precisione di macchina** o **epsilon di macchina** la distanza tra due numeri macchina consecutivi. È possibile dimostrare che l'epsilon di

macchina è esattamente il doppio della unità di roundoff, cioè $\varepsilon = 2u$. Dunque parleremo indifferentemente di unità di roundoff o di precisione di macchina. Nel caso dell'insieme $\mathbb{F}$ (doppia precisione standard IEEE 754, con 53 bit di mantissa inclusa quella nascosta), l'unità di roundoff è uguale a:

$$u = \frac{1}{2}2^{1-53} = 2^{-53} \approx 1.110223024625157e - 016$$

e quindi

$$\varepsilon = 2^{-52} \approx 2.220446049250313e - 016$$

La precisione di macchina ε rappresenta l'ampiezza del buco esistente tra i numeri macchina consecutivi:

1.0000000000000000

e

1.0000000000000001

Esercizi in MatLab Dato il numero reale a e la sua approssimazione A calcolare l'errore assoluto Δa e l'errore relativo δa e stabilire quante sono le cifre decimali corrette e le cifre significative corrette di A .

Variabile eps in MatLab In MatLab la variabile `eps` contiene il valore

$$\varepsilon = 2^{-52} = 2.22044604925031e - 016$$

```
>> eps
ans =
    2.22044604925031e-016
```

Verifichiamo che ε è il buco tra i numeri macchina 1.0000000000000000 e 1.0000000000000001 utilizzando la sua caratterizzazione e ricordando che il programma fornisce come risposta 1 per vero e 0 per falso.

```
>> 1+eps>1
ans =
    1
>> 1+eps/2>1
ans =
    0
```

Osserviamo, però, che, contrariamente a quanto ci aspettiamo:

```
>> 1+eps
ans =
    1.0000000000000000e+000
```

Ciò è dovuto ad un errore di visualizzazione del numero $1 + \text{eps}$. Invece, rispettando le nostre aspettative:

```
>> 1+eps/2
ans =
    1
```

In MatLab esiste una routine che permette di calcolare il massimo errore relativo che si commette approssimando un qualunque numero macchina A .

```
>> x=10.23;
>> eps(x)
ans =
    1.77635683940025e-015
>> x+eps(x)>x
ans =
    1
>> x+eps(x)/2>x
ans =
    0
```

Ovviamente:

```
>> eps(1)
ans =
    2.22044604925031e-016
```

cioè $\text{eps}(1) = \text{eps}$.

2.2 Operazioni Macchina e Cancellazione Numerica

Terza conseguenza della rappresentazione dei numeri come parole di lunghezza fissa - Errori nei calcoli Non sempre una operazione tra due o più numeri macchina produce un risultato che è un numero macchina. Si può verificare una situazione di overflow o underflow. Inoltre la forma normalizzata del risultato potrebbe avere un numero di cifre decimali superiore alla precisione con cui si sta lavorando. Pertanto in un calcolatore è impossibile implementare esattamente le operazioni aritmetiche. Dovremo accontentarci delle cosiddette operazioni macchina: a due numeri macchina viene associato un terzo numero macchina ottenuto arrotondando l'esatto risultato dell'operazione aritmetica.

Siano a e b due numeri reali e siano $A = fl(a)$ e $B = fl(b)$ i corrispondenti numeri macchina. Denotando con $\oplus, \ominus, \otimes, \oslash$ le operazioni macchina corrispondenti rispettivamente alle operazioni aritmetiche $+, -, \times, /$, si ha:

$$A \oplus B = fl(A + B) = (A + B)(1 + \delta_1)$$

$$A \ominus B = fl(A - B) = (A - B)(1 + \delta_2)$$

$$A \otimes B = fl(A \times B) = (A \times B)(1 + \delta_3)$$

$$A \oslash B = fl(A/B) = (A/B)(1 + \delta_4)$$

con

$$|\delta_i| \leq u \leq \varepsilon, \quad i \in \{1, 2, 3, 4\}$$

Dunque, quando si effettua una operazione macchina si commette un errore dell'ordine della precisione di macchina ε .

Utilizzando i precedenti risultati è possibile stimare, in via teorica, gli errori in tutte le espressioni. Per esempio, se X, Y e Z sono numeri macchina si ha:

$$X \otimes (Y \oplus Z) = X \otimes fl(Y + Z) = fl(X \times fl(Y + Z))$$

$$\begin{aligned}
&= (X \times fl(Y + Z)) \times (1 + \delta_1) \\
&= (X \times (Y + Z) \times (1 + \delta_2)) \times (1 + \delta_1) \\
&\approx X \times (Y + Z) \times (1 + \delta_1 + \delta_2)
\end{aligned}$$

avendo trascurato il prodotto $\delta_1 \delta_2$ perché piccolo. Dunque

$$X \otimes (Y \oplus Z) \approx X \times (Y + Z)(1 + \delta_3), \quad \delta_3 = \delta_1 + \delta_2, \quad |\delta_3| \leq \varepsilon$$

Per le operazioni macchina non sempre valgono le note proprietà (commutativa, associativa, distributiva, etc) delle operazioni aritmetiche. La proprietà commutativa permane:

$$A \oplus B = B \oplus A \quad \text{e} \quad A \otimes B = B \otimes A$$

mentre, in generale,

$$(A \oplus B) \oplus C \neq A \oplus (B \oplus C)$$

$$(A \otimes B) \otimes C \neq A \otimes (B \otimes C)$$

$$A \otimes (B \oplus C) \neq (A \otimes B) \oplus (A \otimes C)$$

Un'ulteriore relazione anomala è:

$$A \oplus B = A, \quad \text{se } |B| \ll |A|$$

dunque, l'elemento neutro della somma non è unico.

Ciò accade perché, nell'eseguire la somma tra numeri macchina aventi esponenti diversi, il calcolatore esegue i seguenti passi:

1. individua il numero avente l'esponente $\bar{q}$ più grande;
2. memorizza tutti gli altri numeri utilizzando la loro rappresentazione con esponente $\bar{q}$;
3. esegue la somma.

Infatti, nel precedente esempio, si ha

$$\begin{aligned}
B &= fl(b) = 0.2061153622438558 \cdot 10^{-8} \\
&= 0.00000000000000002061153622438558 \cdot 10^8
\end{aligned}$$

e, arrotondando a 16 cifre,

$$B = fl(b) = 0$$

Somma di più numeri Sempre a causa dell'errore di incolonnamento, se occorre sommare più numeri, tutti positivi e aventi ordini di grandezza diversi, per ottenere un risultato finale accurato conviene procedere in ordine ascendente (dal più piccolo al più grande), cosicché anche i valori più piccoli diano contributo alla somma.

Cancellazione Numerica È un fenomeno che si verifica durante l'operazione di sottrazione tra due numeri quasi uguali. Siano x_1 e x_2 due numeri reali. Se $x = x_1 - x_2$ è molto piccolo, l'errore relativo

$$\delta x = \left| \frac{fl(x_1 - x_2) - x}{x} \right|$$

può essere molto grande e ciò produce una perdita di cifre significative nel calcolo di $fl(x_1 - x_2)$.

È sempre preferibile evitare la sottrazione tra numeri macchina quasi uguali utilizzando, laddove possibile, delle espressioni alternative.

Per esempio:

1.

$$\sqrt{x + \delta} - \sqrt{x} = \frac{\delta}{\sqrt{x + \delta} + \sqrt{x}}$$

2.

$$\cos(x + \delta) - \cos(x) = -2 \sin\left(\frac{\delta}{2}\right) \sin\left(x + \frac{\delta}{2}\right)$$

In generale, se si presenta il caso di dover computare

$$f(x + \delta) - f(x),$$

con $|\delta| \ll |x|$, può convenire considerare l'espansione di Taylor

$$f(x + \delta) - f(x) = f'(x)\delta + f(x)\frac{\delta}{2} + \dots$$

Un problema di cancellazione si può presentare anche nella formula risolutiva dell'equazione di secondo grado $ax^2 + bx + c = 0$:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

per cui conviene usare la formula alternativa:

$$x_1 = \frac{-b - \text{sign}(b)\sqrt{b^2 - 4ac}}{2a}$$

$$x_2 = \frac{c}{ax_1}$$

Esempi in MatLab Da quest'ultimo esempio si evince anche che il prodotto tra due numeri macchina può essere nullo anche se entrambi i numeri sono non nulli, cioè non vale la legge di annullamento del prodotto.

2.3 Propagazione degli Errori Introdotti nei Dati Iniziali

Condizionamento di un problema L'esempio della cancellazione numerica introduce un concetto molto rilevante del Calcolo Scientifico: il condizionamento di un problema. Nella risoluzione di un problema matematico è in molti casi utile avere una misura di quanto i risultati siano sensibili a piccoli cambiamenti dei dati iniziali. In generale se a piccoli cambiamenti nei dati iniziali corrispondono grandi cambiamenti nei dati

finali diremo che il problema è **mal condizionato**. In caso contrario diremo che è **ben condizionato**. La quantità responsabile dell'amplificazione degli errori nei dati iniziali si chiama **indice di condizionamento**. Il condizionamento è una caratteristica propria del problema matematico e non ha alcun legame né con gli errori di arrotondamento delle operazioni macchina, né con gli algoritmi eventualmente scelti per risolverlo. Determinare l'indice di condizionamento di un problema non è sempre facile. Consideriamo il caso semplice del problema numerico della somma. Siano x, y e z tre numeri reali con

$$z = x + y$$

Supponiamo di perturbare x con Δx e y con Δy e sia

$$\bar{z} = (x + \Delta x) + (y + \Delta y)$$

Calcoliamo l'errore relativo che commettiamo approssimando z con $\bar{z}$:

$$\begin{aligned} \delta z &= \frac{|z - \bar{z}|}{|z|} = \frac{|(x + y) - (x + \Delta x + y + \Delta y)|}{|x + y|} = \frac{|-\Delta x - \Delta y|}{|x + y|} = \frac{|\Delta x + \Delta y|}{|x + y|} \\ &\leq \frac{|\Delta x| + |\Delta y|}{|x + y|} = \frac{1}{|x + y|} \left(|\Delta x| \cdot \frac{|x|}{|x|} + |\Delta y| \cdot \frac{|y|}{|y|} \right) \\ &= \frac{1}{|x + y|} \left(|x| \underbrace{\frac{|\Delta x|}{|x|}}_{\delta x} + |y| \underbrace{\frac{|\Delta y|}{|y|}}_{\delta y} \right) = \frac{1}{|x + y|} (|x|\delta x + |y|\delta y) \\ &\leq \frac{1}{|x + y|} (\max\{|x|, |y|\} \delta x + \max\{|x|, |y|\} \delta y) = \frac{\max\{|x|, |y|\}}{|x + y|} (\delta x + \delta y) \end{aligned}$$

La quantità

$$\frac{\max\{|x|, |y|\}}{|x + y|}$$

rappresenta l'**indice di condizionamento**. Ciò significa che se facciamo la somma tra due numeri molto vicini e di segno opposto, anche se gli errori relativi δx e δy sono piccoli, il valore $\bar{z}$ approssimerà z con un errore relativo grande, cioè l'operazione somma diventa mal condizionata e si verifica il fenomeno della Cancellazione Numerica. In generale, denotato con $f(x)$ un generico problema numerico avente x come dato iniziale, si cerca di determinare una relazione del tipo

$$\frac{|f(x) - f(\bar{x})|}{|f(x)|} \leq K \frac{|\Delta x|}{|x|}$$

dove K rappresenta l'indice di condizionamento del problema f . Quanto più è piccolo K , tanto più il problema è ben condizionato.

Osservazioni

- Provando a risolvere i precedenti sistemi con il metodo di Cramer o il metodo di sostituzione si ottengono soluzioni diverse da quelle indicate. Per verificare che le soluzioni indicate sono corrette, basta sostituirle in ciascuna riga del sistema.
- Poiché le soluzioni sono state ricavate lavorando in aritmetica infinita, l'unico motivo per il quale, in seguito alla piccola perturbazione, si sono ottenuti risultati tanto diversi è che la risoluzione del sistema di partenza è un problema mal condizionato.

Stabilità di un algoritmo

Definizione 2.8. Un algoritmo, per un problema numerico, è una sequenza finita di operazioni non ambigue che trasforma i dati di input e fornisce uno o più valori di output. Diremo che un algoritmo è **instabile** se amplifica gli errori di arrotondamento durante la computazione della soluzione di un problema ben condizionato. Dunque il concetto di stabilità di un algoritmo è strettamente connesso con la precisione di calcolo finita e con le operazioni in cui si risolve l'algoritmo stesso.

Questa procedura che, sotto opportune condizioni, prevede lo scambio delle righe di un sistema lineare prende il nome di **pivoting**. La causa di questo bizzarro comportamento è da ricercarsi nella sequenza di operazioni che ci conducono dai dati iniziali ai risultati, ovvero nell'algoritmo. Il metodo di eliminazione di Gauss in generale non è stabile, ma se viene combinato con il pivoting è sempre stabile.

Abbiamo detto che il problema della somma tra due numeri molto vicini è mal condizionato e genera il fenomeno della Cancellazione Numerica. Per questo tipo di problema è ancora più importante scegliere un algoritmo stabile.

Se consideriamo il problema della differenza tra due radicali, abbiamo visto che

$$\sqrt{x + \delta} - \sqrt{x} = \frac{\delta}{\sqrt{x + \delta} + \sqrt{x}}$$

Come abbiamo già verificato con dei tests numerici, l'algoritmo che implementa la formula al primo membro è instabile mentre quello che implementa la formula al secondo membro è sempre stabile.

Analogo discorso si può fare per:

- il calcolo della differenza tra due coseni con le formule equivalenti:

$$\cos(x + \delta) - \cos(x)$$

e

$$-2 \sin\left(\frac{\delta}{2}\right) \sin\left(x + \frac{\delta}{2}\right)$$

- il calcolo delle radici dell'equazione $ax^2 + bx + c = 0$ con le formule equivalenti:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

e

$$x_1 = \frac{-b - \text{sign}(b)\sqrt{b^2 - 4ac}}{2a}, \quad x_2 = \frac{c}{ax_1}$$

Abbiamo già verificato con dei tests numerici, che per entrambi l'algoritmo che implementa la prima formula è instabile mentre quello che implementa la seconda è sempre stabile.

Dunque il controllo degli errori passa essenzialmente per due fasi:

1. studio del condizionamento del problema matematico;
2. uso di un algoritmo stabile per la soluzione del problema.

Conclusioni

- Qualsiasi algoritmo applicato ad un problema mal condizionato genererà amplificazione degli errori di rappresentazione dei dati.
- Si avrà propagazione degli errori di arrotondamento applicando un algoritmo instabile ad un problema ben condizionato.
- Si raggiunge la massima precisione di calcolo quando un algoritmo stabile viene applicato ad un problema ben condizionato.
- A volte può accadere che un problema sia ben condizionato per certi dati iniziali e mal condizionato per altri (vedi l'operazione di somma).

L'obiettivo dei matematici che si occupano del Calcolo Scientifico è quello di risolvere problemi numerici di cui hanno studiato a priori il condizionamento con algoritmi che siano il più possibile stabili.

Esercizi in MatLab

3 Risoluzione di un Sistema Lineare Quadrato

3.1 Il Problema della Risoluzione di un Sistema Lineare

Generalità sui Sistemi Lineari Molti problemi dell'ingegneria, della fisica, della chimica, dell'informatica e dell'economia, si modellizzano mediante equazioni lineari, cioè equazioni in cui le incognite appaiono con esponente uguale ad uno e sono legate tra loro da relazioni lineari (prodotto per scalari e somme algebriche). Un sistema di n equazioni lineari in n incognite è definito come segue:

$$\sum_{j=1}^n a_{i,j}x_j = b_i \quad i \in \{1, \dots, n\}$$

ovvero:

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + \dots + a_{1,n}x_n = b_1 \\ a_{2,1}x_1 + a_{2,2}x_2 + \dots + a_{2,n}x_n = b_2 \\ \vdots \\ a_{n,1}x_1 + a_{n,2}x_2 + \dots + a_{n,n}x_n = b_n \end{cases}$$

Assumeremo che tutte le quantità in gioco siano numeri reali.

Forma Matriciale Ponendo A come la matrice dei coefficienti, $\mathbf{b}$ come il vettore dei termini noti e $\mathbf{x}$ come il vettore delle soluzioni:

$$A = \begin{pmatrix} a_{1,1} & \dots & a_{1,n} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \dots & a_{n,n} \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$$

il sistema si può riscrivere come:

$$A\mathbf{x} = \mathbf{b}$$

dove $A \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$ e $\mathbf{x} \in \mathbb{R}^n$.

Esistenza e Unicità della Soluzione È noto che il sistema ammette un'unica soluzione se e solo se la matrice A è non singolare, o equivalentemente se $\det(A) \neq 0$. In questo caso, denotata con A^{-1} la matrice inversa di A , si ha:

$$\mathbf{x} = A^{-1}\mathbf{b}$$

La matrice inversa è infatti l'unica matrice per la quale $A^{-1}A = AA^{-1} = I$, dove I è la matrice identità.

Il Metodo di Cramer La teoria dell'Algebra Lineare propone come metodo risolutivo il metodo di Cramer:

$$x_i = \frac{\det(A_i)}{\det(A)}, \quad i \in \{1, \dots, n\}$$

dove A_i denota la matrice ottenuta da A sostituendo la colonna i -esima con il vettore $\mathbf{b}$. I determinanti possono essere calcolati utilizzando la regola di Laplace:

$$\det(A) = \sum_{j=1}^n (-1)^{j+1} a_{1,j} \det(A_{1,j})$$

dove $A_{1,j}$ è la matrice di ordine $n - 1$ ottenuta da A eliminando la prima riga e la j -esima colonna.

Costo Computazionale e Necessità dei Metodi Numerici Sfortunatamente il metodo di Cramer ha un costo computazionale molto elevato, pari a circa $[(n+1)(n-1)n]!$ operazioni elementari (flops). Ciò si traduce in tempi effettivi per la computazione inaccettabili. Supponendo per esempio di poter effettuare 10^6 flops al secondo, per risolvere un sistema di 20 equazioni occorrono circa $3 \cdot 10^7$ anni (300.000 secoli). Anche i metodi classici per il calcolo dell'inversa A^{-1} hanno un costo computazionale di tipo fattoriale. Poiché nei problemi reali i sistemi possono essere costituiti da centinaia o migliaia di equazioni, occorrono metodi numerici efficienti che tengano conto della dimensione e delle proprietà della matrice.

Matrice Densa

Definizione 3.1. Una matrice $A \in \mathbb{R}^{n \times n}$ si dice **densa** se la maggior parte dei suoi elementi è non nulla.

Matrice Sparsa

Definizione 3.2. Una matrice $A \in \mathbb{R}^{n \times n}$ si dice **sparsa** se possiede un numero di elementi non nulli dell'ordine di n . Per memorizzare una matrice sparsa è possibile utilizzare solo tre vettori: uno per gli elementi non nulli, uno per gli indici di riga e uno per gli indici di colonna.

Matrici Strutturate

Definizione 3.3. Una matrice $A \in \mathbb{R}^{n \times n}$ si dice **strutturata** se i suoi elementi sono disposti secondo una regola nota.

- **Matrice Diagonale:** $a_{i,j} = 0$ per $i \neq j$.

$$\begin{pmatrix} a_{1,1} & 0 & \dots & 0 \\ 0 & a_{2,2} & \dots & \vdots \\ \vdots & \vdots & \ddots & 0 \\ 0 & \dots & 0 & a_{n,n} \end{pmatrix}$$

- **Matrice Triangolare Inferiore:** $a_{i,j} = 0$ per $i < j$.

$$\begin{pmatrix} a_{1,1} & 0 & \dots & 0 \\ a_{2,1} & a_{2,2} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \dots & a_{n,n} \end{pmatrix}$$

- **Matrice Triangolare Superiore:** $a_{i,j} = 0$ per $i > j$.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ 0 & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{n,n} \end{pmatrix}$$

- **Matrice Tridiagonale:** elementi non nulli solo sulla diagonale principale, sulla sovra-diagonale e sulla sotto-diagonale.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & 0 & \dots & 0 \\ a_{2,1} & a_{2,2} & a_{2,3} & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ 0 & \dots & 0 & a_{n,n-1} & a_{n,n} \end{pmatrix}$$

- **Hessenberg Inferiore:** $a_{i,j} = 0$ per $j > i + 1$ (elementi nulli sopra la prima sovra-diagonale).

$$\begin{pmatrix} a_{1,1} & a_{1,2} & 0 & \dots & 0 \\ a_{2,1} & a_{2,2} & a_{2,3} & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ a_{n-1,1} & a_{n-1,2} & \dots & a_{n-1,n-1} & a_{n-1,n} \\ a_{n,1} & a_{n,2} & \dots & a_{n,n-1} & a_{n,n} \end{pmatrix}$$

- **Hessenberg Superiore:** $a_{i,j} = 0$ per $i > j + 1$ (elementi nulli sotto la prima sotto-diagonale).

$$\begin{pmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n-1} & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n-1} & a_{2,n} \\ 0 & a_{3,2} & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & a_{n-1,n-1} & a_{n-1,n} \\ 0 & \dots & 0 & a_{n,n-1} & a_{n,n} \end{pmatrix}$$

Matrice Simmetrica

Definizione 3.4. Una matrice A si dice **simmetrica** se coincide con la sua trasposta, cioè $A = {}^t A$.

```

1 function isSym = isSymm(A, tol)
2 %ISSYMM Verifica se una matrice è simmetrica con una data tolleranza.
3 % isSym = ISSYMM(A) restituisce true se A è simmetrica (entro 1e-10).
4 % isSym = ISSYMM(A, tol) permette di specificare la tolleranza.
5 %
6 % Input:
7 %   A - Matrice numerica quadrata (double)
8 %   tol - (Opzionale) Soglia di tolleranza. Default: 1e-10
9 %
10 % Output:
11 %   isSym - Logical true (1) se simmetrica, false (0) altrimenti
12
13 %% 1. Validazione degli Input
14 arguments
15     A (:,:) double {mustBeNumeric}
16     tol (1,1) double {mustBeNonnegative} = 1e-10
17 end
18
19 %% 2. Controllo Dimensioni
20 % Estraggo le dimensioni della matrice
21 [r, c] = size(A);
22
23 % Verifico che la matrice sia quadrata
24 if r ~= c
25     error('isSymm:DimError', 'La matrice deve essere quadrata (Input size:
26 %dx%d).', r, c);
27 end
28
29 %% 3. Verifica Simmetria
30 % Calcoliamo la trasposta una sola volta
31 AT = A';
32
33 % Confrontiamo vettorialmente usando la tolleranza
34 isSym = all( abs(A(:) - AT(:)) <= tol );
35 end

```

Function isSymm

Matrice Definita Positiva

Definizione 3.5. Una matrice simmetrica A si dice **definita positiva** se

$$\forall \mathbf{y} \in \mathbb{R}^n (\mathbf{y} \neq \mathbf{0}), \quad {}^t\mathbf{y}A\mathbf{y} > 0.$$

Criterio di Sylvester

Proposizione 3.1. Una matrice simmetrica $A \in \mathbb{R}^{n \times n}$ è definita positiva se e solo se:

$$\det(A_k) > 0, \quad k \in \{1, \dots, n\}$$

dove $\det(A_k)$ è il determinante della sottomatrice principale di testa di ordine k .

```

1 function isPositive = isDefPos(A)
2 % ISDEFPOS Determina se una matrice è definita positiva.
3 % isPositive = ISDEFPOS(A) restituisce true se la matrice A è definita
4 % positiva utilizzando il criterio di Sylvester (tutti i minori principali
5 % di testa devono avere determinante strettamente positivo).
6 %
7 % Input:
8 %   A - Matrice quadrata numerica (reale e simmetrica).
9 %
10 % Output:
11 %   isPositive - Logical true (1) se simmetrica, false (0) altrimenti.
12
13 %% 1 - Validazione degli Input
14 arguments
15     A(:, :) {mustBeNumeric, mustBeReal}
16 end
17
18 %% 2 - Controllo Dimensioni e Simmetria
19 % Estraggo le dimensioni della matrice
20 [r,c] = size(A);
21
22 % Verifico che la matrice sia quadrata
23 if r ~= c
24     error('isDefPos:NonSquareMatrix', 'La matrice di input deve essere
25     quadrata (%dx%d).', rows, cols);
26 end
27
28 % Verifica della simmetria (condizione necessaria per def. pos. reale)
29 if ~isSymm(A)
30     error('isDefPos:NonSymmetricMatrix', 'La matrice deve essere simmetrica
31     per applicare il criterio.');
```

Function isDefPos

Matrice a Diagonale Dominante

Definizione 3.6. Una matrice A è detta a **diagonale dominante per righe** se:

$$|a_{i,i}| > \sum_{j=1, j \neq i}^n |a_{i,j}|, \quad i \in \{1, \dots, n\}.$$

È detta a **diagonale dominante per colonne** se:

$$|a_{j,j}| > \sum_{i=1, i \neq j}^n |a_{i,j}|, \quad j \in \{1, \dots, n\}.$$

```

1 function isDom = diagDomRow(A)
2 % DIAGDOMROW Verifica se una matrice è a diagonale dominante per righe.
3 % isDom = DIAGDOMROW(A) restituisce true se la matrice A è a diagonale
4 % dominante per righe.
5 %
6 % Input:
7 %     A - Matrice quadrata numerica.
8 %
9 % Output:
10 %     isDom - Logical true (1) se dominante per righe, false (0) altrimenti.
11
12 %% 1. Validazione Input
13 arguments
14     A (:,:) {mustBeNumeric}
15 end
16
17 %% 2. Controllo Dimensioni
18 % Estraggo le dimensioni
19 [m, n] = size(A);
20 if m ~= n
21     error('diagDomRow:NonSquareMatrix', 'La matrice deve essere quadrata (
Input %dx%d).', m, n);
22 end
23
24 %% 3. Verifico Proprietà
25 % Calcolo i valori assoluti
26 absA = abs(A);
27
28 % Estraggo la diagonale
29 d_diag = diag(absA);
30
31 % Calcolo la somma di ogni riga
32 sum_rows = sum(absA,2);
33
34 % Calcolo la somma degli elementi extra-diagonali
35 sum_off_diag = sum_rows - d_diag;
36
37 % Verifica della condizione
38 isDom = true;
39 for i = 1:m
40     if d_diag(i) <= sum_off_diag(i)
41         isDom = false;
42         return; % Interrompe appena trova una riga che viola la condizione
43     end
44 end
45

```

46 end

Function diagDomRow

```

1 function isDom = diagDomCol(A)
2 % DIAGDOMCOL Verifica se una matrice è a diagonale dominante per colonne.
3 % isDom = DIAGDOMCOL(A) restituisce true se la matrice A è a diagonale
4 % dominante per colonne.
5 %
6 % Input:
7 %     A - Matrice quadrata numerica.
8 %
9 % Output:
10 %     isDom - Logical true (1) se dominante per colonne, false (0) altrimenti
11 %
12 %% 1. Validazione Input
13 arguments
14     A (:,:) {mustBeNumeric}
15 end
16
17 %% 2. Controllo Dimensioni
18 % Estraggo le dimensioni
19 [m, n] = size(A);
20 if m ~= n
21     error('diagDomCol:NonSquareMatrix', 'La matrice deve essere quadrata (
Input %dx%d).', m, n);
22 end
23
24 %% 3. Verifico Proprietà
25 % Calcolo i valori assoluti
26 absA = abs(A);
27
28 % Estraggo la diagonale
29 d_diag = diag(absA);
30
31 % Calcolo la somma di ogni colonna
32 sum_cols = sum(absA).';
33
34 % Calcolo la somma degli elementi extra-diagonali
35 sum_off_diag = sum_cols - d_diag;
36
37 % Verifica della condizione
38 isDom = true;
39 for j = 1:n
40     if d_diag(j) <= sum_off_diag(j)
41         isDom = false;
42         return; % Interrompe appena trova una colonna che viola la
condizione
43     end
44 end
45
46 end

```

Function diagDomCol

Norme Vettoriali

Definizione 3.7. Una **norma vettoriale** è una funzione che associa ad un vettore $\mathbf{x} \in \mathbb{R}^n$ un numero reale $\|\mathbf{x}\|$ con le seguenti proprietà:

1. $\|\mathbf{x}\| > 0, \forall \mathbf{x} \neq \mathbf{0}$ e $\|\mathbf{x}\| = 0 \iff \mathbf{x} = \mathbf{0}$;
2. $\|c\mathbf{x}\| = |c|\|\mathbf{x}\|, \forall c \in \mathbb{R}$;
3. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|, \forall \mathbf{y} \in \mathbb{R}^n$.

Le norme più utilizzate sono:

- **Norma infinito:**

$$\|\mathbf{x}\|_{\infty} = \max_{1 \leq i \leq n} |x_i|;$$

- **Norma 1:**

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|;$$

- **Norma Euclidea (o norma 2):**

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2} = \sqrt{\mathbf{x}^t \mathbf{x}}.$$

Norme Matriciali

Definizione 3.8. Una **norma matriciale** associa ad una matrice A un numero reale $\|A\|$ soddisfacendo proprietà analoghe a quelle vettoriali, con l'aggiunta della proprietà di sub-moltiplicatività:

- $\|AB\| \leq \|A\|\|B\|, \forall B \in \mathbb{R}^{n \times n}$.

Le norme matriciali più utilizzate sono:

- **Norma infinito:**

$$\|A\|_{\infty} = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{i,j}|$$

(massima somma per righe).

- **Norma 1:**

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{i,j}|$$

(massima somma per colonne).

- **Norma Spettrale (o norma 2):**

$$\|A\|_2 = \sqrt{\rho(AA^t)},$$

dove $\rho(B)$ è il raggio spettrale di B (modulo massimo degli autovalori).

- **Norma di Frobenius:**

$$\|A\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n |a_{i,j}|^2}.$$

Norme Compatibili e Indotte

Definizione 3.9. Una norma vettoriale e una matriciale sono **compatibili** se $\|Ax\| \leq \|A\| \|x\|$. Una norma matriciale si dice **naturale** o **indotta** dalla norma vettoriale se:

$$\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \sup_{\|x\|=1} \|Ax\|.$$

Per le norme naturali vale $\|I\| = 1$. Le norme matriciali $\|\cdot\|_\infty$, $\|\cdot\|_1$ e $\|\cdot\|_2$ sono naturali (indotte dalle rispettive norme vettoriali), mentre la norma di Frobenius non lo è (poiché $\|I\|_F = \sqrt{n}$).

Condizionamento di un sistema lineare Vogliamo ora studiare il condizionamento del sistema lineare $Ax = b$. Ci chiediamo come il problema "risponda" a eventuali perturbazioni nei dati iniziali (matrice A e vettore b). Supponiamo di introdurre una perturbazione $\Delta A \in \mathbb{R}^{n \times n}$ sulla matrice A e una perturbazione $\Delta b \in \mathbb{R}^n$ sul vettore dei termini noti b . La soluzione del problema perturbato non sarà x ma $y = x + \Delta x$, con $\Delta x \in \mathbb{R}^n$. Il sistema perturbato è:

$$(A + \Delta A)(x + \Delta x) = (b + \Delta b).$$

Teorema (Principale)

Teorema 3.2. Siano $A \in \mathbb{R}^{n \times n}$ una matrice non singolare e $\Delta A \in \mathbb{R}^{n \times n}$ tale che sia soddisfatta la condizione

$$\|A^{-1}\| \|\Delta A\| \leq \frac{1}{2}$$

per una generica norma matriciale indotta $\|\cdot\|$. Allora anche la matrice $A + \Delta A$ è non singolare e se $x \in \mathbb{R}^n$ è soluzione del sistema $Ax = b$ con $b \in \mathbb{R}^n$ ($b \neq 0$) e $\Delta x \in \mathbb{R}^n$ verifica il sistema perturbato $(A + \Delta A)(x + \Delta x) = (b + \Delta b)$ per $\Delta b \in \mathbb{R}^n$, si ha che

$$\frac{\|\Delta x\|}{\|x\|} \leq 2K(A) \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right)$$

ove $K(A) = \|A\| \|A^{-1}\|$ è il numero di condizionamento della matrice A .

Per dimostrare il precedente teorema ci occorre il seguente risultato:

Teorema (Ausiliario)

Teorema 3.3. Sia A una matrice quadrata, se $\|A\| < 1$ allora la matrice $I + A$ è invertibile e

vale la seguente disuguaglianza

$$\|(I + A)^{-1}\| \leq \frac{1}{1 - \|A\|}$$

essendo $\|\cdot\|$ una norma matriciale indotta tale che $\|A\| < 1$.

Osservazioni sulla stima dell'errore Dalla stima ottenuta si evincono alcune proprietà fondamentali:

- Per ogni norma matriciale $\|\cdot\|$ naturale (che soddisfi la proprietà di sub-moltiplicatività), si ha:

$$K(A) = \|A\| \|A^{-1}\| \geq \|AA^{-1}\| = \|I\| = 1$$

- Il condizionamento è una caratteristica propria del sistema lineare e non è legato al particolare metodo numerico usato per determinarne la soluzione.
- Ponendo $K_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty$, dalla stima dell'errore relativo, se:

$$\frac{\|\Delta \mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} \leq 2K_\infty(A) \varepsilon \leq \frac{1}{2} 10^{1-p}$$

allora p cifre significative della soluzione calcolata mediante un algoritmo stabile sono da ritenere corrette.

Calcolo del Condizionamento in Matlab In Matlab esiste una function predefinita per calcolare $K(A)$:

- `cond(A, inf)`: calcola il condizionamento di A in norma infinito ($K_\infty(A)$).
- `cond(A, 1)`: calcola il condizionamento di A in norma 1 ($K_1(A)$).
- `cond(A)`: calcola il condizionamento di A in norma 2 ($K_2(A)$).

Esempi di Matrici Mal Condizionate

Matrici di Hilbert

Matrici di Vandermonde

Classificazione dei Metodi Numerici I metodi numerici si suddividono in:

- **Metodi diretti:** In assenza di errori di arrotondamento calcolano la soluzione esatta in un numero finito di passi.
- **Metodi iterativi:** Generano una successione infinita di vettori che converge alla soluzione.

La scelta dipende da stabilità, occupazione di memoria e costo computazionale.

Metodi Diretti e Sparsatezza I metodi diretti trasformano il problema in problemi equivalenti. Sono efficienti per matrici dense. Tuttavia, per matrici sparse, i metodi diretti causano il fenomeno del *fill-in* (riempimento): il numero di elementi non nulli cresce durante il procedimento, potendo saturare la memoria. In questi casi sono preferibili i metodi iterativi, che lasciano inalterata la matrice A .

3.2 Sistemi Diagonali e Triangolari

Metodi Diretti Si chiamano metodi diretti quei metodi numerici che risolvono sistemi lineari in un numero finito di passi. In altri termini, supponendo di effettuare i calcoli in precisione infinita, tali metodi forniscono la soluzione esatta del sistema mediante un numero finito di operazioni, noto a priori. In precisione di calcolo finita invece, se il metodo proposto è stabile, esso fornisce una soluzione approssimata con la precisione massima ottenibile, cioè in accordo con il condizionamento della matrice dei coefficienti.

Sistemi Diagonali Il più banale dei metodi diretti si ottiene nel caso delle matrici diagonali. Se denotiamo con $\mathbf{x} = {}^t(x_1, x_2, \dots, x_n)$ e $\mathbf{b} = {}^t(b_1, b_2, \dots, b_n)$ rispettivamente il vettore delle incognite e quello dei termini noti, e se la matrice dei coefficienti è del tipo:

$$D = \begin{pmatrix} a_{1,1} & 0 & 0 & \dots & 0 \\ 0 & a_{2,2} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \dots & \vdots \\ 0 & 0 & 0 & \dots & a_{n,n} \end{pmatrix}$$

il sistema $D\mathbf{x} = \mathbf{b}$, nell'ipotesi che D sia non singolare (cioè $a_{i,i} \neq 0, \forall i \in \{1, \dots, n\}$), si risolve banalmente mediante le n divisioni:

$$x_i = \frac{b_i}{a_{i,i}}, \quad i \in \{1, \dots, n\}$$

con un costo computazionale complessivo di n operazioni di divisione. L'algoritmo è ben posto se la matrice è non singolare. Inoltre, poiché l'algoritmo è composto da n divisioni indipendenti l'una dall'altra, esso è banalmente stabile.

```

1 function x = diagSub(D, b, tol)
2 % DIAGSUB Risolve il sistema lineare diagonale Dx = b.
3 % X = DIAGSUB(D, B, TOL) restituisce il vettore soluzione X.
4 %
5 % Input:
6 %   D - Matrice quadrata diagonale (n x n)
7 %   b - Vettore dei termini noti (n x 1)
8 %   tol (opzionale) - Tolleranza per verifica della singolarità (1 x 1)
9 %
10 % Output:
11 %   x - Vettore soluzione (n x 1)
12
13 %% 1. Validazione degli Input
14 arguments
15     D(:, :) double {mustBeNumeric}
16     b(:, 1) double {mustBeNumeric}
17     tol(1,1) double {mustBeNumeric} = 1e-10
18 end
19
20 %% 2. Controllo Dimensioni

```

```

21 % Estraggo le dimensioni
22 [n,m] = size(D);
23
24 % Verifico che D sia quadrata
25 if n ~= m
26     error('diagSub:NonSquareMatrix', 'La matrice D deve essere quadrata (
Input %dx%d).', n, m);
27 end
28
29 % Verifico che b sia compatibile con D
30 if m ~= length(b)
31     error('diagSub:DimensionMismatch', 'Le dimensioni di D e b non
corrispondono (Input (%dx%d)x(%dx1)).',n,m,length(b));
32 end
33
34 %% 3. Controllo Singolarità
35 if any(abs(diag(D)) < tol)
36     error('diagSub:SingularMatrix', 'La matrice è singolare.');
```

```

37 end
38
39 %% 4. Sostituzione in Avanti
40 % Pre-allocazione
41 x = zeros(n, 1);
42
43 % Ciclo principale
44 for i = 1 : n
45     x(i) = b(i) / D(i,i);
46 end
47
48 end

```

Function diagSub

Sistemi Triangolari Consideriamo ora il caso dei sistemi con matrice dei coefficienti triangolare. Denotiamo con L una generica matrice triangolare inferiore ("lower triangular"):

$$L = \begin{pmatrix} l_{1,1} & 0 & 0 & \dots & 0 \\ l_{2,1} & l_{2,2} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \dots & \vdots \\ l_{n-1,1} & l_{n-1,2} & \dots & l_{n-1,n-1} & 0 \\ l_{n,1} & l_{n,2} & \dots & l_{n,n-1} & l_{n,n} \end{pmatrix}$$

e con U una generica matrice triangolare superiore ("upper triangular"):

$$U = \begin{pmatrix} u_{1,1} & u_{1,2} & \dots & u_{1,n-1} & u_{1,n} \\ 0 & u_{2,2} & \dots & u_{2,n-1} & u_{2,n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & u_{n-1,n-1} & u_{n-1,n} \\ 0 & 0 & \dots & 0 & u_{n,n} \end{pmatrix}$$

Vogliamo fornire degli algoritmi per la risoluzione dei sistemi $Lx = b$ e $Ux = b$. Una matrice triangolare è non singolare se e solo se i suoi elementi diagonali sono tutti diversi da zero. Infatti:

$$\det(L) = \prod_{i=1}^n l_{ii} \neq 0 \iff l_{i,i} \neq 0, \quad \forall i \in \{1, \dots, n\};$$

$$\det(U) = \prod_{i=1}^n u_{ii} \neq 0 \iff u_{i,i} \neq 0, \quad \forall i \in \{1, \dots, n\}.$$

Gli algoritmi per la risoluzione di sistemi con matrice dei coefficienti triangolare sono essenzialmente basati sulla tecnica di sostituzione.

Algoritmo di Sostituzione in Avanti (Forward Substitution) Esaminiamo il caso della matrice triangolare inferiore $Lx = b$. Esplicitamente il sistema può scriversi come:

$$\begin{cases} l_{1,1}x_1 = b_1 \\ l_{2,1}x_1 + l_{2,2}x_2 = b_2 \\ \vdots \\ l_{n,1}x_1 + l_{n,2}x_2 + l_{n,3}x_3 + \dots + l_{n,n}x_n = b_n \end{cases}$$

Si osserva che dalla prima equazione si ricava immediatamente x_1 . Tale valore può essere sostituito nella seconda equazione per ricavare x_2 , e così via.

$$x_1 = \frac{b_1}{l_{1,1}}, \quad x_2 = \frac{b_2 - l_{2,1}x_1}{l_{2,2}}, \quad \dots, \quad x_i = \frac{b_i - \sum_{k=1}^{i-1} l_{i,k}x_k}{l_{i,i}} \quad i \in \{2, \dots, n\}.$$

L'algoritmo è ben posto se la matrice L è non singolare.

Schema Algoritmo

```
x(1) = b(1) / L(1,1)           % 1 flop (divisione)
for i = 2 : n
    x(i) = b(i)
    for k = 1 : i-1
        x(i) = x(i) - L(i,k) * x(k)   % 2 flops (1 mult, 1 sub)
    end
    x(i) = x(i) / L(i,i)           % 1 flop (divisione)
end
```

Costo Computazionale

$$\sum_{i=1}^n i \simeq \frac{n(n+1)}{2} \simeq \frac{n^2}{2} \text{ operazioni (flops)}$$

È possibile provare che l'algoritmo è stabile, cioè non produce amplificazione dell'errore di arrotondamento.

```
1 function x = forwardSub(L, b, tol)
2 % FORWARDSUB Risolve il sistema lineare triangolare inferiore Lx = b usando la
  sostituzione in avanti.
3 % x = FORWARDSUB(L, b) restituisce il vettore soluzione x.
4 %
5 % Input:
6 %   L - Matrice quadrata triangolare inferiore (n x n)
7 %   b - Vettore dei termini noti (n x 1)
8 %   tol (opzionale) - Tolleranza per verifica della singolarità (1 x 1)
9 %
10 % Output:
```

```

11 %    x - Vettore soluzione (n x 1)
12
13 %% 1. Validazione degli Input
14 arguments
15     L (:,:) double {mustBeNumeric}
16     b (:,1) double {mustBeNumeric}
17     tol (1,1) double {mustBeNumeric} = 1e-10
18 end
19
20 %% 2. Controllo Dimensioni
21 % Estraggo le dimensioni
22 [n,m] = size(L);
23
24 % Verifico che L sia quadrata
25 if n ~= m
26     error('forwardSub:NonSquareMatrix', 'La matrice L deve essere quadrata
27 (Input %dx%d).', n, m);
28 end
29
30 % Verifico che b sia compatibile con L
31 if m ~= length(b)
32     error('forwardSub:DimensionMismatch', 'Le dimensioni di L e b non
33 corrispondono (Input (%dx%d)x(%dx1)).', n,m,length(b));
34 end
35
36 %% 3. Controllo Singularità
37 if any(abs(diag(L)) < tol)
38     error('forwardSub:SingularMatrix', 'La matrice è singolare.');
```

```

39 end
40
41 %% 4. Sostituzione in Avanti
42 % Pre-allocazione
43 x = zeros(n, 1);
44
45 % Primo elemento
46 x(1) = b(1) / L(1,1);
47
48 % Ciclo principale
49 for i = 2 : n
50     x(i) = b(i);
51     % Ciclo interno: accumulo la somma parziale
52     for k = 1:i-1
53         x(i) = x(i) - L(i,k) * x(k);
54     end
55     % Divisione finale per l'elemento diagonale
56     x(i) = x(i) / L(i,i);
57 end
end

```

Function forwardSub

Calcolo dell'Inversa di una Matrice Triangolare Inferiore Il metodo di sostituzione in avanti può essere adattato per il calcolo esplicito dell'inversa di una matrice triangolare inferiore L . I vettori colonna $\mathbf{v}_i$ dell'inversa $V = (\mathbf{v}_1, \dots, \mathbf{v}_n)$ di L soddisfano i sistemi lineari:

$$L\mathbf{v}_i = \mathbf{e}_i, \quad i \in \{1, \dots, n\}$$

dove $\mathbf{e}_i$ è l' i -esima colonna della matrice identità. Poiché l'inversa di una matrice triangolare inferiore è anch'essa triangolare inferiore, le colonne di V hanno tutti gli elementi sovradiagonali nulli. Sfruttando questa proprietà, per ogni colonna j le uniche componenti incognite sono quelle da j a n . Il calcolo delle componenti non nulle $v_{i,j}$ (con $i \geq j$) avviene risolvendo i sistemi con la sostituzione in avanti.

Algoritmo di Inversione

$$v_{j,j} = \frac{1}{l_{j,j}}, \quad \text{per } j \in \{1, \dots, n\},$$

$$v_{i,j} = -\frac{1}{l_{i,i}} \sum_{k=j}^{i-1} l_{i,k} v_{k,j}, \quad \text{per } i \in \{j+1, \dots, n\}.$$

Schema Algoritmo

```

for j = 1 : n
    v(j,j) = 1 / L(j,j)
    for i = j+1 : n
        v(i,j) = 0
        for k = j : i-1
            v(i,j) = v(i,j) - L(i,k) * v(k,j)
        end
        v(i,j) = v(i,j) / L(i,i)
    end
end
end

```

Costo Computazionale

$$\sum_{j=1}^n \sum_{i=j}^n (i-j+1) \approx \frac{n^3}{6} \text{ operazioni.}$$

```

1 function V = invL(L, tol)
2 % INVL calcola l'inversa di una matrice triangolare inferiore ottimizzando
3 % la sostituzione in avanti con una certa tolleranza
4 % V = INVL(L, tol) restituisce l'inversa della matrice L.
5 %
6 % Input:
7 %   L - Matrice triangolare inferiore numerica (n x n)
8 %   tol (opzionale) - Tolleranza per la verifica della singolarità (1 x 1)
9 %
10 % Output:
11 %   V - Matrice triangolare inferiore numerica inversa di L (n x n)
12
13 %% 1. Validazione Input
14 arguments
15     L(:, :) double {mustBeNumeric}
16     tol(1,1) double {mustBeNumeric} = 1e-10
17 end
18
19 %% 2. Controllo Dimensioni
20 % Estraggo le dimensioni
21 [n,m] = size(L);

```

```

22
23 % Verifico che L sia quadrata
24 if n ~= m
25     error('invL:NonSquareMatrix', 'La matrice L deve essere quadrata (Input
    %dx%d).', n, m);
26 end
27
28 % Verifico che L sia non singolare
29 if any(abs(diag(L)) < tol)
30     error('invL:SingularMatrix', 'La matrice L è singolare.');
```

31 end

32

33 %% 3. Calcolo Inversa

34 % Pre-allocazione (V sarà triangolare inferiore, il resto rimane 0)

35 V = zeros(n);

36

37 for j = 1:n

38 % 1. Diagonale: inverso del pivot

39 V(j,j) = 1 / L(j,j);

40

41 % 2. Sostituzione in avanti per la colonna j (righe i > j)

42 for i = j+1:n

43 V(i,j) = 0;

44 % Inizializzazione dell'accumulatore per la sommatoria

45 for k = j:i-1

46 V(i,j) = V(i,j) - L(i,k) * V(k,j);

47 end

48

49 % Divisione finale per il coefficiente della diagonale

50 V(i,j) = V(i,j) / L(i,i);

51 end

52 end

53

54 end

Function invL

Algoritmo di Sostituzione all'Indietro (Backward Substitution) Si vuole risolvere il sistema $Ux = b$ dove U è triangolare superiore:

$$\begin{cases} u_{1,1}x_1 + u_{1,2}x_2 + \cdots + u_{1,n}x_n = b_1 \\ \vdots \\ u_{n-1,n-1}x_{n-1} + u_{n-1,n}x_n = b_{n-1} \\ u_{n,n}x_n = b_n \end{cases}$$

Procedendo dall'ultima equazione verso la prima:

$$x_n = \frac{b_n}{u_{n,n}}, \quad x_{n-1} = \frac{b_{n-1} - u_{n-1,n}x_n}{u_{n-1,n-1}}, \quad \dots, \quad x_i = \frac{b_i - \sum_{k=i+1}^n u_{i,k}x_k}{u_{i,i}} \quad i \in \{n-1, \dots, 1\}.$$

Anche in questo caso l'algoritmo è ben posto se U è non singolare.

Schema Algoritmo

```

x(n) = b(n) / U(n,n)           % 1 flop (divisione)
for i = n-1 : -1 : 1
```



```

x(i) = b(i)
for k = i+1 : n
    x(i) = x(i) - U(i,k) * x(k)    % 2 flops (1 mult, 1 sub)
end
x(i) = x(i) / U(i,i)              % 1 flop (divisione)
end

```

Costo Computazionale

$$\sum_{i=1}^n (n - i + 1) = \frac{n(n+1)}{2} \simeq \frac{n^2}{2} \text{ operazioni.}$$

Anche l'algoritmo di sostituzione all'indietro è stabile.

```

1 function x = backwardSub(U, b, tol)
2 % BACKWARDSUB Risolve il sistema lineare triangolare superiore Ux = b usando la
   sostituzione all'indietro.
3 % x = BACKWARDSUB(U, b, tol) restituisce il vettore soluzione x.
4 %
5 % Input:
6 %   U - Matrice quadrata triangolare superiore (n x n)
7 %   b - Vettore dei termini noti (n x 1)
8 %   tol (opzionale) - Tolleranza per verifica della singolarità (1 x 1)
9 %
10 % Output:
11 %   x - Vettore soluzione (n x 1)
12
13 %% 1. Validazione degli Input
14 arguments
15     U(:, :) double {mustBeNumeric}
16     b(:, 1) double {mustBeNumeric}
17     tol(1, 1) double {mustBeNumeric} = 1e-10
18 end
19
20 %% 2. Controllo Dimensioni
21 % Ricaviamo le dimensioni
22 [n, m] = size(U);
23
24 % Verifichiamo che U sia quadrata
25 if n ~= m
26     error('backwardSub:NonSquareMatrix', 'La matrice U deve essere quadrata
       (Input %dx%d).', n, m);
27 end
28
29 % Verifichiamo che b sia compatibile con U
30 if m ~= length(b)
31     error('backwardSub:DimensionMismatch', 'Le dimensioni di U e b non
       corrispondono (Input (%dx%d)x(%dx1)).', n, m, length(b));
32 end
33
34 %% 3. Controllo Singolarità
35 if any(abs(diag(U)) < tol)
36     error('backwardSub:SingularMatrix', 'La matrice è singolare.');
```

```

43 % Ultimo elemento
44 x(n) = b(n) / U(n,n);
45
46 % Ciclo principale
47 for i = n-1 : -1 : 1
48     x(i) = b(i);
49     % Ciclo interno: accumulo la somma parziale
50     for k = i+1:n
51         x(i) = x(i) - U(i,k) * x(k);
52     end
53     % Divisione finale per l'elemento diagonale
54     x(i) = x(i) / U(i,i);
55 end
56
57 end

```

Function backwardSub

Calcolo dell'Inversa di una Matrice Triangolare Superiore Il metodo di sostituzione all'indietro può essere adattato per il calcolo esplicito dell'inversa di una matrice triangolare superiore U . I vettori colonna $\mathbf{v}_i$ dell'inversa $V = (\mathbf{v}_1, \dots, \mathbf{v}_n)$ di U soddisfano i sistemi lineari:

$$U\mathbf{v}_i = \mathbf{e}_i, \quad i \in \{1, \dots, n\}$$

dove $\mathbf{e}_i$ è l' i -esima colonna della matrice identità. Poiché l'inversa di una matrice triangolare superiore è anch'essa triangolare superiore, le colonne di V hanno tutti gli elementi subdiagonali nulli. Sfruttando questa proprietà, indichiamo con $\mathbf{v}'_j = {}^t(v_{1,j}, \dots, v_{j,j})$ il vettore di dimensione j tale che:

$$U_j \mathbf{v}'_j = \mathbf{e}'_j, \quad j \in \{1, \dots, n\}$$

essendo U_j la sottomatrice principale di testa U di ordine j e $\mathbf{e}'_j$ il vettore di $\mathbb{R}^j$ con tutte le componenti nulle fuorché l'ultima che è 1. I sistemi sono triangolari superiori di ordine j e si risolvono con la sostituzione all'indietro.

Algoritmo di Inversione

$$v_{j,j} = \frac{1}{u_{j,j}}, \quad \text{per } j \in \{1, \dots, n\},$$

$$v_{i,j} = -\frac{1}{u_{i,i}} \sum_{k=i+1}^j u_{i,k} v_{k,j}, \quad \text{per } i \in \{j-1, \dots, 1\}.$$

Schema Algoritmo

```

for j = 1 : n
    v(j,j) = 1 / U(j,j)
    for i = j-1 : -1 : 1
        v(i,j) = 0
        for k = i+1 : j
            v(i,j) = v(i,j) - U(i,k) * v(k,j)
        end
        v(i,j) = v(i,j) / U(i,i)
    end
end
end

```

Costo Computazionale

$$\sum_{j=1}^n \sum_{i=1}^j (j-i+1) \approx \frac{n^3}{6} \text{ operazioni.}$$

```

1 function V = invU(U, tol)
2 % INVU calcola l'inversa di una matrice triangolare superiore ottimizzando
3 % la sostituzione all'indietro con una certa tolleranza
4 % V = INVU(U, tol) restituisce l'inversa della matrice U.
5 %
6 % Input:
7 %   U - Matrice triangolare superiore numerica (n x n)
8 %   tol (opzionale) - Tolleranza per la verifica della singolarità (1 x 1)
9 %
10 % Output:
11 %   V - Matrice triangolare superiore numerica inversa di U (n x n)
12
13 %% 1. Validazione Input
14 arguments
15     U (:,:) double {mustBeNumeric}
16     tol (1,1) double {mustBeNumeric} = 1e-10
17 end
18
19 %% 2. Controllo Dimensioni
20 % Estraggo le dimensioni
21 [n,m] = size(U);
22
23 % Verifico che U sia quadrata
24 if n ~= m
25     error('invU:NonSquareMatrix', 'La matrice U deve essere quadrata (Input
26 %dx%d).', n, m);
27 end
28
29 % Verifico che U sia non singolare
30 if any(abs(diag(U)) < tol)
31     error('invU:SingularMatrix', 'La matrice U è singolare.');

```

Function invU

3.3 Metodo di Eliminazione di Gauss

Metodo di eliminazione di Gauss Consideriamo il caso di una generica matrice di ordine n . Sia dunque $A \in \mathbb{R}^{n \times n}$ e come al solito $x, b \in \mathbb{R}^n$. Vogliamo risolvere il generico sistema $Ax = b$. Il metodo di Gauss è basato sulla tecnica di eliminazione delle incognite da fissate equazioni. Obiettivo del metodo è ridurre, attraverso un numero finito di passi, il sistema di partenza ad uno ad esso equivalente che abbia la matrice dei coefficienti triangolare.

Descrizione del metodo (caso $n = 4$) Per semplicità, incominciamo con il descrivere il metodo nel caso di un sistema di dimensione $n = 4$.

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 = b_1 \\ a_{2,1}x_1 + a_{2,2}x_2 + a_{2,3}x_3 + a_{2,4}x_4 = b_2 \\ a_{3,1}x_1 + a_{3,2}x_2 + a_{3,3}x_3 + a_{3,4}x_4 = b_3 \\ a_{4,1}x_1 + a_{4,2}x_2 + a_{4,3}x_3 + a_{4,4}x_4 = b_4 \end{cases}$$

Assumiamo che $a_{1,1} \neq 0$ e consideriamo la quantità $m_{2,1} = -\frac{a_{2,1}}{a_{1,1}}$. Moltiplicando ambo i membri della prima equazione per $m_{2,1}$ e sommando alla seconda equazione, si ottiene:

$$(-a_{2,1} + a_{2,1})x_1 + (a_{2,2} - \frac{a_{2,1}}{a_{1,1}}a_{1,2})x_2 + \dots + (a_{2,4} - \frac{a_{2,1}}{a_{1,1}}a_{1,4})x_4 = (b_2 - \frac{a_{2,1}}{a_{1,1}}b_1)$$

cioè

$$a_{2,2}^{(2)}x_2 + a_{2,3}^{(2)}x_3 + a_{2,4}^{(2)}x_4 = b_2^{(2)}$$

dunque

$$a_{2,j}^{(2)} = a_{2,j} + m_{2,1}a_{1,j}, \quad j = 2, 3, 4$$

Ripetendo il procedimento per la terza e quarta equazione con i moltiplicatori $m_{3,1} = -\frac{a_{3,1}}{a_{1,1}}$ e $m_{4,1} = -\frac{a_{4,1}}{a_{1,1}}$, otteniamo il sistema equivalente al passo 2:

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 = b_1 \\ a_{2,2}^{(2)}x_2 + a_{2,3}^{(2)}x_3 + a_{2,4}^{(2)}x_4 = b_2^{(2)} \\ a_{3,2}^{(2)}x_2 + a_{3,3}^{(2)}x_3 + a_{3,4}^{(2)}x_4 = b_3^{(2)} \\ a_{4,2}^{(2)}x_2 + a_{4,3}^{(2)}x_3 + a_{4,4}^{(2)}x_4 = b_4^{(2)} \end{cases}$$

dove in generale $a_{i,j}^{(2)} = a_{i,j} + m_{i,1}a_{1,j}$ e $b_i^{(2)} = b_i + m_{i,1}b_1$.

Assumendo ora che $a_{2,2}^{(2)} \neq 0$, eliminiamo l'incognita x_2 dalla terza e quarta equazione usando i moltiplicatori $m_{3,2} = -\frac{a_{3,2}^{(2)}}{a_{2,2}^{(2)}}$ e $m_{4,2} = -\frac{a_{4,2}^{(2)}}{a_{2,2}^{(2)}}$. Si ottiene il sistema al passo 3:

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 = b_1 \\ a_{2,2}^{(2)}x_2 + a_{2,3}^{(2)}x_3 + a_{2,4}^{(2)}x_4 = b_2^{(2)} \\ a_{3,3}^{(3)}x_3 + a_{3,4}^{(3)}x_4 = b_3^{(3)} \\ a_{4,3}^{(3)}x_3 + a_{4,4}^{(3)}x_4 = b_4^{(3)} \end{cases}$$

Infine, assumendo $a_{3,3}^{(3)} \neq 0$, eliminiamo x_3 dall'ultima equazione con $m_{4,3} = -\frac{a_{4,3}^{(3)}}{a_{3,3}^{(3)}}$, ottenendo il sistema triangolare finale:

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 = b_1 \\ a_{2,2}^{(2)}x_2 + a_{2,3}^{(2)}x_3 + a_{2,4}^{(2)}x_4 = b_2^{(2)} \\ a_{3,3}^{(3)}x_3 + a_{3,4}^{(3)}x_4 = b_3^{(3)} \\ a_{4,4}^{(4)}x_4 = b_4^{(4)} \end{cases}$$

Generalizzazione al caso di ordine n Assumiamo che sia $a_{1,1} \neq 0$. Possiamo eliminare l'incognita x_1 dalla 2^a, ..., n -esima equazione sommando all' i -esima equazione ($i = 2, \dots, n$) la prima equazione moltiplicata per $m_{i,1} = -\frac{a_{i,1}}{a_{1,1}}$. Le formule di aggiornamento generali al passo $k \in 1, \dots, n-1$ (per eliminare l'incognita x_k dalle equazioni successive) sono per $i, j \in \{k+1, \dots, n\}$:

$$m_{i,k} = -\frac{a_{i,k}^{(k)}}{a_{k,k}^{(k)}}, \quad a_{i,j}^{(k+1)} = a_{i,j}^{(k)} + m_{i,k}a_{k,j}^{(k)}, \quad b_i^{(k+1)} = b_i^{(k)} + m_{i,k}b_k^{(k)}.$$

Gli elementi $a_{1,1}, a_{2,2}^{(2)}, a_{3,3}^{(3)}, \dots$ che compaiono al denominatore vengono detti **elementi pivot**, mentre le quantità $m_{i,k}$ sono dette **moltiplicatori**. Dopo $n-1$ passi, se tutti i pivot sono non nulli, si ottiene un sistema triangolare superiore risolvibile con la sostituzione all'indietro.

Schema Algoritmo

```
for k = 1 : n-1
    for i = k+1 : n
        mol = -a(i,k) / a(k,k)      % 1 operazione
        for j = k+1 : n
            a(i,j) = a(i,j) + mol * a(k,j) % n-k operazioni
        end
        b(i) = b(i) + mol * b(k)    % 2 operazione
    end
end
```

Costo Computazionale

$$\sum_{k=1}^{n-1} [2(n-k) + (n-k)^2] \approx \frac{n^3}{3} \text{ operazioni}$$

```
1 function [Ag,bg] = gaussElim(A,b)
2 % GAUSSELIM Applica il metodo dell'eliminazione di Gauss alla matrice dei
  coefficienti e al vettore dei termini noti.
3 % [Ag,bg] = gaussElim(A,b) restituisce A e b risultanti dall'eliminazione.
4 %
5 % Input:
6 %   A - Matrice dei coefficienti (n x n)
7 %   b - Vettore dei termini noti (n x 1)
8 %
```

```

9 % Output:
10 %   Ag - Matrice dei coefficienti post-GE (n x n)
11 %   bg - Vettore dei termini noti post-GE (n x 1)
12
13 %% 1. Validazione degli Input
14 arguments
15     A (:,:) double {mustBeNumeric}
16     b (:,1) double {mustBeNumeric}
17 end
18
19 %% 2. Verifica Dimensioni
20 [n, m] = size(A);
21
22 % Verifico se la matrice è quadrata
23 if n ~= m
24     error('gaussElim:NonSquareMatrix', 'La matrice non è quadrata (Size %dx
25 %d).', n, m);
26 end
27
28 % Verifico se il prodotto Ab è compatibile
29 if n ~= length(b)
30     error('gaussElim:NonCompatible', 'Il prodotto non è compatibile (Size
31 (%dx%d)*(%dx1)).', n, m, length(b));
32 end
33
34 %% 3. Eliminazione di Gauss
35 for k = 1 : n-1
36     % Controllo pivot nullo (sicurezza)
37     if A(k,k) == 0
38         error('gaussElim:ZeroPivot', 'Pivot nullo alla riga %d.', k);
39     end
40
41     % Eliminazione
42     for i = k+1 : n
43         mol = -A(i,k) / A(k,k);
44         for j = k+1:n
45             A(i,j) = A(i,j) + mol * A(k,j);
46         end
47         b(i) = b(i) + mol * b(k);
48     end
49 end
50
51 % Nota: Gli elementi A(i,k) sotto la diagonale non vengono azzerati
52 % fisicamente
53 % per risparmiare tempo, ma sono logicamente considerati zero.
54
55 Ag = A;
56 bg = b;
57 end

```

Function gaussElim

Esistenza dei Pivot

Teorema 3.4. Sia $A \in \mathbb{R}^{n \times n}$. Gli elementi pivot $a_{k,k}^{(k)}$ sono tutti diversi da zero se e soltanto se tutte le matrici principali di testa di A , denotate con $A_k = (a_{i,j})_{i,j \in \{1, \dots, k\}}$, sono non singolari,

cioè:

$$\det(A_k) \neq 0 \quad \forall k \in \{1, \dots, n\}.$$

3.4 Strategia Pivoting e Fattorizzazione LU

Necessità del Pivoting Il metodo di Gauss descritto in precedenza si arresta se, a un certo passo k , l'elemento pivot $a_{k,k}^{(k)}$ risulta nullo. Tuttavia, se la matrice è non singolare, deve esistere almeno un elemento $a_{i,k}^{(k)} \neq 0$ con $i > k$ sulla stessa colonna. Per proseguire è sufficiente scambiare la riga k -esima con la riga i -esima. Anche quando il pivot non è nullo ma è molto piccolo in valore assoluto rispetto agli altri elementi, possono verificarsi errori di arrotondamento disastrosi (amplificazione degli errori o cancellazione numerica). Per garantire la stabilità numerica, è necessario permutare le righe in modo da scegliere un pivot "ottimale". Tale strategia è detta *pivoting*.

Strategie di Pivoting

- **Pivoting Parziale:** Al passo k , si sceglie come pivot l'elemento di modulo massimo nella colonna k -esima, al di sotto o sulla diagonale principale. Si cerca l'indice $r \geq k$ tale che:

$$|a_{r,k}^{(k)}| = \max_{k \leq i \leq n} |a_{i,k}^{(k)}|$$

Se $r \neq k$, si scambia la riga k con la riga r .

- **Pivoting Totale:** Si cerca l'elemento massimo su tutta la sottomatrice attiva. Si scelgono indici $r, s \geq k$ tali che:

$$|a_{r,s}^{(k)}| = \max_{k \leq i, j \leq n} |a_{i,j}^{(k)}|$$

Si scambiano le righe k ed r e le colonne k ed s (quest'ultimo scambio comporta il riordinamento delle incognite).

Il pivoting parziale è la strategia più utilizzata perché meno costosa computazionalmente ($O(n^2)$ confronti) e generalmente sufficiente a garantire stabilità.

Stabilità senza Pivoting Il metodo di eliminazione di Gauss senza pivoting è stabile a priori solo per alcune classi di matrici:

- Matrici a **diagonale dominante per righe o per colonne**;
- Matrici **simmetriche e definite positive**.

Esempio di Instabilità

Schema Algoritmo

```
for k = 1 : n-1
    % Ricerca del pivot massimo nella colonna k-esima
    trovo p tale che: |a(p,k)| = max(|a(i,k)|) per i >= k

    if k ~= p
```

```

        scambio righe k-esima e p-esima di A
        scambio righe k-esima e p-esima di b
    end

    % Eliminazione standard di Gauss
    for i = k+1 : n
        mol = -a(i,k) / a(k,k)
        for j = k+1 : n
            a(i,j) = a(i,j) + mol * a(k,j)
        end
        b(i) = b(i) + mol * b(k)
    end
end
end

```

Costo Computazionale Il costo computazionale totale dell'algoritmo di Gauss con pivoting è dato dalla somma delle operazioni aritmetiche (flops) necessarie per l'eliminazione e dei confronti necessari per la ricerca del pivot massimo.

- **Operazioni Aritmetiche:** $\approx \frac{n^3}{3}$ (invariato rispetto a Gauss senza pivoting).
- **Confronti:** Ad ogni passo k si effettuano $n - k$ confronti. Il totale è:

$$\sum_{k=1}^{n-1} (n - k) = \frac{n(n-1)}{2} \approx \frac{n^2}{2} \text{ confronti}$$

Poiché n^2 è trascurabile rispetto a n^3 per n grandi, il costo asintotico rimane dominato da $n^3/3$.

```

1 function [Ag,bg] = gaussElimPP(A,b)
2 % GAUSSELIMPP Applica il metodo dell'eliminazione di Gauss alla matrice dei
3 % coefficienti e al vettore dei termini noti implementando il pivoting
4 % parziale.
5 % [Ag,bg] = gaussElimPP(A,b) restituisce A e b risultanti dall'eliminazione
6 % con pivoting parziale.
7 %
8 % Input:
9 %   A - Matrice dei coefficienti (n x n)
10 %   b - Vettore dei termini noti (n x 1)
11 %
12 % Output:
13 %   Ag - Matrice dei coefficienti post-GE (n x n)
14 %   bg - Vettore dei termini noti post-GE (n x 1)
15 %
16 %% 1. Validazione degli Input
17 arguments
18     A(:, :) double {mustBeNumeric}
19     b(:, 1) double {mustBeNumeric}
20 end
21
22 %% 2. Verifica Dimensioni
23 [n, m] = size(A);
24
25 % Verifico se la matrice è quadrata
26 if n ~= m
27     error('gaussElimPP:NonSquareMatrix', 'La matrice non è quadrata (Size %
28     dx%d).', n, m);
29 end

```



```

26     end
27
28     % Verifico se il prodotto Ab è compatibile
29     if n ~= length(b)
30         error('gaussElimPP:NonCompatible', 'Il prodotto non è compatibile (Size
31             (%dx%d)*(%dx1)).', n, m, length(b));
32     end
33
34     %% 3. Eliminazione di Gauss
35     for k = 1 : n-1
36         % Pivoting parziale
37         % Trovo il massimo nella colonna k (dal pivot in giù)
38         [~, r_rel] = max(abs(A(k:n, k)));
39
40         % Converto in indice globale
41         r_glob = r_rel + k - 1;
42
43         % Se il pivot migliore non è quello attuale, scambio
44         if r_glob ~= k
45             A([k, r_glob], :) = A([r_glob, k], :);
46             b([k, r_glob]) = b([r_glob, k]);
47         end
48
49         % Controllo pivot nullo (sicurezza)
50         if A(k,k) == 0
51             error('gaussElimPP:ZeroPivot', 'Pivot nullo alla riga %d.', k);
52         end
53
54         % Eliminazione di Gauss
55         for i = k+1 : n
56             mol = -A(i,k) / A(k,k);
57
58             for j = k+1:n
59                 A(i,j) = A(i,j) + mol * A(k,j);
60             end
61
62             % Aggiorno il termine noto
63             b(i) = b(i) + mol * b(k);
64         end
65     end
66
67     % Nota: Gli elementi A(i,k) sotto la diagonale non vengono azzerati
68     % fisicamente
69     % per risparmiare tempo, ma sono logicamente considerati zero.
70
71     Ag = A;
72     bg = b;
73 end

```

Function gaussElimPP

La Fattorizzazione LU Il metodo di eliminazione di Gauss, dal punto di vista matriciale, può essere riletto come la costruzione di una successione di matrici:

$$[A|\mathbf{b}] = [A^{(1)}|\mathbf{b}^{(1)}], \dots, [A^{(k)}|\mathbf{b}^{(k)}], \dots, [A^{(n)}|\mathbf{b}^{(n)}]$$

in modo tale che $A^{(n)}$ sia triangolare superiore e $\mathbf{b}^{(n)}$ sia il nuovo termine noto. Le matrici della successione sono tra loro legate da una trasformazione del tipo:

$$[A^{(k+1)}|\mathbf{b}^{(k+1)}] = M^{(k)}[A^{(k)}|\mathbf{b}^{(k)}], \quad k \in \{1, \dots, n-1\}$$

dove $M^{(k)}$ è detta **matrice elementare di Gauss** ed è definita come:

$$M^{(k)} = \begin{pmatrix} 1 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & \dots & 1 & 0 & \dots & 0 \\ 0 & \dots & m_{k+1,k} & 1 & \dots & 0 \\ \vdots & & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & m_{n,k} & 0 & \dots & 1 \end{pmatrix}$$

Si ha quindi che:

$$A = [M^{(1)}]^{-1} A^{(2)} = [M^{(1)}]^{-1} [M^{(2)}]^{-1} A^{(3)} = \dots = [M^{(1)}]^{-1} \dots [M^{(n-1)}]^{-1} A^{(n)}$$

e analogamente per il termine noto:

$$\mathbf{b} = [M^{(1)}]^{-1} \dots [M^{(n-1)}]^{-1} \mathbf{b}^{(n)}$$

Ponendo:

$$L = \left(\prod_{k=1}^{n-1} [M^{(k)}]^{-1} \right), \quad U = A^{(n)}, \quad \mathbf{y} = \mathbf{b}^{(n)},$$

si ottiene la fattorizzazione:

$$A = LU \quad \text{e} \quad \mathbf{b} = L\mathbf{y}$$

dove U è la matrice triangolare superiore che si ottiene alla fine del metodo di eliminazione di Gauss e L è una matrice triangolare inferiore. La struttura di L è la seguente:

$$L = \begin{pmatrix} 1 & 0 & \dots & \dots & 0 \\ -m_{2,1} & 1 & 0 & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \\ & \ddots & 1 & & \vdots \\ \vdots & & -m_{k+1,k} & 1 & \ddots \\ & & -m_{k+2,k} & \ddots & \ddots \\ \vdots & & \vdots & & \ddots & 1 & 0 \\ -m_{n,1} & \dots & -m_{n,k} & \dots & \dots & -m_{n,n-1} & 1 \end{pmatrix}$$

Per costruirla basta, ad ogni passo del metodo di Gauss, memorizzare i moltiplicatori cambiati di segno.

Schema Algoritmo

```
for k = 1 : n-1
    for i = k+1 : n
        A(i,k) = -A(i,k) / A(k,k)      % Calcolo moltiplicatori
```

```

        for j = k+1 : n
            A(i,j) = A(i,j) + A(i,k) * A(k,j)
        end
    end
end
L = eye(n) - tril(A, -1) % Parte triangolare inferiore con 1 sulla diagonale
U = triu(A)              % Parte triangolare superiore

```

```

1 function [L, U] = factorLU(A)
2 % FACTORLU Esegue la fattorizzazione LU su una matrice A.
3 % [L, U] = factorLU(A) fattorizza la matrice A in L * U usando
4 % l'eliminazione di Gauss senza pivoting.
5 %
6 % Input:
7 %   A - Matrice dei coefficienti (n x n)
8 %
9 % Output:
10 %   L - Matrice triangolare inferiore unitaria (n x n)
11 %   U - Matrice triangolare superiore (n x n)
12
13 %% 1. Validazione degli Input
14 arguments
15     A (:,:) double {mustBeNumeric}
16 end
17
18 %% 2. Verifica Dimensioni
19 % Estraggo le dimensioni
20 [n, m] = size(A);
21
22 % Verifico se la matrice è quadrata
23 if n ~= m
24     error('factorLU:NonSquareMatrix', 'La matrice dei coefficienti deve
25     essere quadrata (Size: %dx%d).', n, m);
26 end
27
28 %% 3. Eliminazione di Gauss
29 for k = 1 : n-1
30     % Controllo pivot nullo
31     if A(k,k) == 0
32         error('factorLU:ZeroPivot', 'Pivot nullo incontrato alla riga %d.'
33         , k);
34     end
35
36     % Ciclo sulle righe sottostanti
37     for i = k+1 : n
38         % Calcolo e assegnazione del moltiplicatore
39         A(i,k) = -A(i,k) / A(k,k);
40
41         % Aggiornamento della riga i-esima (Eliminazione)
42         for j = k+1:n
43             A(i,j) = A(i,j) + A(i,k) * A(k,j);
44         end
45     end
46
47     % Costruzione di L e U
48     L = eye(n) - tril(A, -1);
49     U = triu(A);
50 end

```

Function factorLU

Risoluzione di Sistemi con LU Una volta fattorizzata $A = LU$, il sistema $A\mathbf{x} = \mathbf{b}$ diventa $LU\mathbf{x} = \mathbf{b}$. Si risolve in due passi:

1. Risoluzione del sistema triangolare inferiore $L\mathbf{y} = \mathbf{b}$ (sostituzione in avanti).
2. Risoluzione del sistema triangolare superiore $U\mathbf{x} = \mathbf{y}$ (sostituzione all'indietro).

Costo Computazionale Costo totale: $\frac{n^3}{3}$ (fattorizzazione) + n^2 (soluzione sistemi).

Applicazioni della Fattorizzazione LU

1. Calcolo del Determinante di A

Il determinante di una matrice A può essere calcolato utilizzando la formula di Binet. Avendo la fattorizzazione $A = LU$, si ha:

$$\det(A) = \det(LU) = \det(L) \det(U)$$

Poiché L è triangolare inferiore con tutti 1 sulla diagonale, $\det(L) = 1$. Poiché U è triangolare superiore, il suo determinante è il prodotto degli elementi diagonali. Dunque:

$$\det(A) = \det(U) = \prod_{i=1}^n u_{i,i}$$

Il costo computazionale complessivo è $\frac{n^3}{3} + n$ (dove $\frac{n^3}{3}$ è il costo della fattorizzazione).

2. Risoluzione di p sistemi con la stessa matrice dei coefficienti

Supponiamo di dover risolvere i sistemi:

$$A\mathbf{x}_1 = \mathbf{b}_1, \quad A\mathbf{x}_2 = \mathbf{b}_2, \quad \dots, \quad A\mathbf{x}_p = \mathbf{b}_p$$

o in altri termini il sistema matriciale:

$$AX = B$$

con $A \in \mathbb{R}^{n \times n}$ e $X, B \in \mathbb{R}^{n \times p}$. Calcolate le matrici L e U una sola volta, ogni sistema $A\mathbf{x}_i = \mathbf{b}_i$ viene risolto risolvendo i due sistemi triangolari:

$$\begin{cases} L\mathbf{y} = \mathbf{b}_i \\ U\mathbf{x}_i = \mathbf{y} \end{cases} \quad i \in \{1, \dots, p\}$$

mediante gli algoritmi di sostituzione. Il costo computazionale complessivo è:

$$\frac{n^3}{3} + pn^2$$

Se invece si risolvesse ciascun sistema indipendentemente dagli altri con il metodo di Gauss, il costo sarebbe molto più alto: $p(\frac{n^3}{3} + \frac{n^2}{2})$.

3. Calcolo dell'inversa di A

Se $p = n$ e $B = I$ (matrice identità), risolvere il sistema $AX = B$ è equivalente a calcolare l'inversa A^{-1} . Si risolvono quindi i sistemi:

$$\begin{cases} Ly = \mathbf{e}_i \\ Ux_i = y \end{cases}, \quad i \in \{1, \dots, n\}$$

dove i vettori $\mathbf{e}_i$ rappresentano le colonne della matrice I (vettori della base canonica). Il costo computazionale complessivo "grezzo" sarebbe $\frac{n^3}{3} + n^3$. Tuttavia, sfruttando opportunamente la natura dei vettori $\mathbf{e}_i$ (che contengono molti zeri), il costo si riduce a circa n^3 . Nel dettaglio:

- Risolvere il sistema matriciale $LY = I$ ha un costo di $\frac{n^3}{6}$.
- Risolvere il sistema matriciale $UX = Y$ ha un costo di $\frac{n^3}{2}$.

Sommando il costo della fattorizzazione ($\frac{n^3}{3}$), il costo totale asintotico è n^3 .

Il metodo di Gauss con la variante del pivoting Il metodo di Gauss con la variante del pivoting esegue ancora una fattorizzazione di matrice nei seguenti termini:

$$PA = \hat{L}U \quad \text{e} \quad P\mathbf{b} = \hat{L}\mathbf{y}$$

dove $P \in \mathbb{R}^{n \times n}$, detta matrice di permutazione, contiene le informazioni relative agli scambi di righe. Vale il seguente:

Teorema 3.5. Per ogni matrice $A \in \mathbb{R}^{n \times n}$ esiste una matrice di permutazione $P \in \mathbb{R}^{n \times n}$ tale che

$$PA = \hat{L}U.$$

Le matrici di permutazione Siano $1 \leq r, s \leq n$ e sia $P_{r,s}$ la matrice ottenuta da I scambiando la r -esima e la s -esima riga.

$$P_{r,s} = \begin{pmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & 0 & \cdots & 1 & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & 1 & \cdots & 0 & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{pmatrix} \begin{matrix} \\ \\ \leftarrow r \\ \\ \leftarrow s \\ \\ \end{matrix}$$

$\begin{matrix} \uparrow & \uparrow \\ r & s \end{matrix}$

Proposizione 3.6. La matrice $P_{r,s}$ è detta **matrice di permutazione** e gode delle seguenti proprietà:

1. $\det(P_{r,s}) = -1$;
2. $P_{r,s} = {}^t P_{r,s}$;

3. $P_{r,s}^2 = I$;
4. $P_{r,s}^{-1} = P_{r,s}$.

Fattorizzazione LU con pivoting I passi della fattorizzazione LU senza pivoting della matrice A possono essere sintetizzati come segue:

- Se $a_{1,1} \neq 0$: annullamento elementi prima colonna a partire dalla seconda riga della matrice iniziale, ottenuto dal prodotto $M^{(1)}A$.
- Se $a_{2,2}^{(2)} \neq 0$: annullamento elementi della seconda colonna a partire dalla terza riga della matrice attiva, ottenuto dal prodotto $M^{(2)}M^{(1)}A$.
- ...
- Se $a_{n-1,n-1}^{(n-1)} \neq 0$: annullamento elementi della $(n-1)$ -esima colonna a partire dalla n -esima riga della matrice attiva, ottenuto dal prodotto $M^{(n-1)} \dots M^{(2)}M^{(1)}A$.

Al termine si ha $M^{(n-1)} \dots M^{(2)}M^{(1)}A = U$ triangolare superiore. Possiamo descrivere la tecnica del pivoting usando le matrici di permutazione, nel seguente modo:

- Sia $|a_{r_1,1}| = \max_{i \geq 1} |a_{i,1}|$. Se $r_1 \neq 1$ si pone $P_1 = P_{1,r_1}$ e si esegue lo scambio della prima riga con la r_1 -esima con il prodotto P_1A . Se $r_1 = 1$ si pone $P_1 = I$ e non si effettuano scambi. Si esegue l'annullamento degli elementi della prima colonna a partire dalla seconda riga della matrice P_1A con il prodotto $M^{(1)}P_1A$.
- Sia $|a_{r_2,2}^{(2)}| = \max_{i \geq 2} |a_{i,2}^{(2)}|$. Se $r_2 \neq 2$ si pone $P_2 = P_{2,r_2}$ e si esegue lo scambio della seconda riga con la r_2 -esima della matrice $M^{(1)}P_1A$ con il prodotto $P_2M^{(1)}P_1A$. Se $r_2 = 2$ si pone $P_2 = I$. Si esegue l'annullamento degli elementi della seconda colonna a partire dalla terza riga della matrice $P_2M^{(1)}P_1A$ con il prodotto $M^{(2)}P_2M^{(1)}P_1A$.
- ...

Al termine della procedura si ottiene:

$$M^{(n-1)}P_{n-1} \dots M^{(2)}P_2M^{(1)}P_1A = U$$

ovvero

$$GA = U, \quad G = M^{(n-1)}P_{n-1} \dots M^{(2)}P_2M^{(1)}P_1.$$

Vogliamo ora mostrare come dalla $GA = U$ si perviene alla seguente fattorizzazione della matrice A :

$$PA = \hat{L}U$$

ove P è un'opportuna matrice di permutazione, che tiene conto di tutti gli scambi di righe, $\hat{L}$ è una matrice triangolare inferiore con elementi sulla diagonale uguali a 1 e U è la matrice triangolare superiore del metodo di eliminazione di Gauss. Ovviamente, in assenza di scambi $P = I$. Si possono infatti riordinare le matrici in modo da ottenere:

$$M^{(n-1)}P_{n-1} \dots M^{(2)}P_2M^{(1)}P_1A = (\bar{L}_{n-1} \dots \bar{L}_2\bar{L}_1)^{-1}(P_{n-1} \dots P_2P_1)A$$

e, posto $\hat{L}^{-1} = \bar{L}_{n-1} \dots \bar{L}_2\bar{L}_1$ e $P = P_{n-1} \dots P_2P_1$, possiamo scrivere:

$$\hat{L}^{-1}PA = U$$

da cui

$$PA = \hat{L}U.$$

In generale (si può provare per induzione):

$$M^{(n-1)}P_{n-1}M^{(n-2)}P_{n-2}\dots M^{(2)}P_2M^{(1)}P_1A = \bar{L}_{n-1}\dots\bar{L}_2\bar{L}_1P_{n-1}\dots P_2P_1A$$

con

$$\bar{L}_j = \begin{cases} L_j^{(n-1-j)} = (P_{n-1}P_{n-2}\dots P_{j+1})M^{(j)}(P_{j+1}\dots P_{n-2}P_{n-1}), & j < n-1 \\ M^{(j)}, & j = n-1 \end{cases}$$

Schema Algoritmo

```

for k = 1:n-1
    trovo p : |a(p,k)| = max(|a(i,k)|) per i >= k
    if k ~= p
        scambio righe k-esima e p-esima di A
        scambio righe k-esima e p-esima di I
    end
    for i = k+1:n
        a(i,k) = a(i,k) / a(k,k)
        for j = k+1:n
            a(i,j) = a(i,j) - a(i,k) * a(k,j)
        end
    end
end
end
L = eye(n) + tril(A, -1)
U = triu(A)

```

Costo computazionale

$$\frac{n^3}{3} \text{ operazioni} + \sum_{k=1}^{n-1} (n-k) \text{ confronti} \simeq \frac{n^3}{3} \text{ operazioni} + \frac{n^2}{2} \text{ confronti}$$

```

1 function [L, U, P, s] = factorLUPP(A)
2 % FACTORLUPP Esegue la fattorizzazione LU con pivoting parziale.
3 % [L, U, P, s] = factorLUPP(A) calcola la fattorizzazione PA = LU e
4 % restituisce il numero di scambi effettuati.
5 %
6 % Input:
7 %   A - Matrice dei coefficienti (n x n)
8 %
9 % Output:
10 %   L - Matrice triangolare inferiore unitaria (n x n)
11 %   U - Matrice triangolare superiore (n x n)
12 %   P - Matrice di permutazione (n x n)
13 %   s - Numero di scambi effettuati (1 x 1)
14
15 %% 1. Validazione degli Input
16 arguments
17     A(:, :) double {mustBeNumeric}
18 end
19
20 %% 2. Verifica Dimensioni
21 % Estraggo le dimensioni
22 [n, m] = size(A);

```

```

23
24 % Verifico se la matrice è quadrata
25 if n ~= m
26     error('factorLUPP:NonSquareMatrix', 'La matrice deve essere quadrata (
Size: %dx%d).', n, m);
27 end
28
29 %% 3. Eliminazione di Gauss con Pivoting Parziale
30 % Inizializzo P come matrice identità
31 P = eye(n);
32 % Inizializzo s
33 s = 0;
34
35 for k = 1 : n-1
36     % Pivoting parziale
37     % Trovo il massimo nella colonna k (dal pivot in giù)
38     [~, r_rel] = max(abs(A(k:n, k)));
39
40     % Converto in indice globale
41     r_glob = r_rel + k - 1;
42
43     % Se il pivot migliore non è quello attuale, scambio
44     if r_glob ~= k
45         A([k, r_glob], :) = A([r_glob, k], :);
46
47         % Scambio le righe della matrice di permutazione P
48         P([k, r_glob], :) = P([r_glob, k], :);
49
50         % Aggiorno variabile s
51         s = s + 1;
52     end
53
54     % Controllo pivot nullo (sicurezza)
55     if A(k,k) == 0
56         error('factorLUPP:ZeroPivot', 'Matrice singolare (pivot nullo alla
riga %d).', k);
57     end
58
59     % Ciclo sulle righe sottostanti
60     for i = k+1 : n
61         % Calcolo e assegnazione del moltiplicatore
62         A(i,k) = -A(i,k) / A(k,k);
63
64         % Aggiornamento riga i-esima
65         for j = k+1:n
66             A(i,j) = A(i,j) + A(i,k) * A(k,j);
67         end
68     end
69 end
70
71 %% 4. Costruzione Output
72 L = eye(n) - tril(A, -1);
73 U = triu(A);
74 % P è già stata costruita durante i cicli
75 end

```

Function factorLUPP

Risoluzione di sistemi lineari mediante LU con pivoting Nota la fattorizzazione del tipo $PA = \hat{L}U$, per determinare la soluzione del sistema lineare $Ax = b$ si risolvono due

sistemi triangolari:

$$\begin{cases} \hat{L}\mathbf{y} = P\mathbf{b} \\ U\mathbf{x} = \mathbf{y} \end{cases}$$

che deduciamo dalla relazione $PA\mathbf{x} = P\mathbf{b}$, ovvero $\hat{L}U\mathbf{x} = P\mathbf{b}$.

Costo computazionale L'intera procedura (fattorizzazione $PA = \hat{L}U$ e risoluzione dei due sistemi triangolari) consta di $\frac{n^3}{3} + n^2$ operazioni aritmetiche. Ci sono poi $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$ confronti per determinare il massimo pivot in ogni passo.

Applicazioni della Fattorizzazione LU

1. Calcolo del determinante

Da $PA = \hat{L}U$, essendo $P^{-1} = P^T$,

$$\det(A) = \det(P^T) \det(\hat{L}) \det(U) = \det(P) \det(U)$$

Poiché

$$\det(P) = \det(P_{n-1}) \dots \det(P_2) \det(P_1),$$

e

$$\det(P_j) = \begin{cases} 1 & \text{se } P_j = I \\ -1 & \text{altrimenti} \end{cases}$$

si ha

$$\det(A) = (-1)^s \prod_{i=1}^n a_{i,i}^{(i)}$$

dove s denota il numero complessivo di scambi effettuati.

2. Calcolo dell'inversa di A

Denotate con $\mathbf{x}_1, \dots, \mathbf{x}_n$ le colonne della matrice inversa di A , quest'ultima può essere calcolata risolvendo i $2n$ sistemi:

$$\begin{cases} \hat{L}\mathbf{y} = P\mathbf{e}_i \\ U\mathbf{x}_i = \mathbf{y} \end{cases} \quad i \in \{1, \dots, n\}$$

il primo triangolare inferiore e il secondo triangolare superiore. Il costo computazionale complessivo è $\frac{n^3}{3} + n^3$ ma può essere ridotto a n^3 se si tiene conto della natura dei vettori $P\mathbf{e}_i$.

3.5 Metodo di Cholesky

Fattorizzazione LDU Sia $A \in \mathbb{R}^{n \times n}$ una matrice per cui esiste la fattorizzazione LU di A senza pivoting. Questa decomposizione può essere sempre riscritta come:

$$A = LU = LDU_1$$

essendo L, U_1 entrambe matrici triangolari a diagonale unitaria e D matrice diagonale tale che $(D)_{i,i} = (U)_{i,i}$. Se A è simmetrica, allora $U_1 = {}^t L$, e si ha:

$$A = LD^t L$$

Fattorizzazione $L^t L$ Se $A \in \mathbb{R}^{n \times n}$ è simmetrica definita positiva, essendo gli elementi di D tutti positivi, si ha:

$$D = D^{\frac{1}{2}} D^{\frac{1}{2}}$$

dove gli elementi di $(D^{\frac{1}{2}})_{i,i} = \sqrt{D_{i,i}}$. Da cui segue il teorema:

Teorema 3.7. Se $A \in \mathbb{R}^{n \times n}$ è una matrice simmetrica definita positiva, esiste ed è unica la fattorizzazione:

$$A = L_1^t L_1 \quad \text{con} \quad L_1 = L D^{\frac{1}{2}}$$

dove L_1 è una matrice triangolare inferiore con elementi diagonali non nulli.

Nel seguito si fa riferimento alla fattorizzazione di cui nel teorema precedente come alla fattorizzazione del tipo $L^t L$. La fattorizzazione $L^t L$ può essere ottenuta attraverso la procedura di fattorizzazione LU senza pivoting con un costo pari a $n^3/6$, cioè circa la metà della fattorizzazione LU. Infatti è possibile organizzare l'algoritmo in modo da sfruttare la simmetria di tutte le matrici attive negli $n - 1$ passi. Vedremo ora come sia possibile ottenere la stessa fattorizzazione attraverso una tecnica "compatta" che porta al metodo di Cholesky e che costa circa $n^3/6$ operazioni.

Costruzione del Metodo di Cholesky Posto $A = (a_{i,j})_{i,j \in \{1, \dots, n\}}$ e $L = (l_{i,j})_{i,j \in \{1, \dots, n\}}$, si ha:

$$a_{i,j} = \sum_{k=1}^n l_{i,k}^t l_{k,j} = \sum_{k=1}^n l_{i,k} l_{j,k}$$

Poiché la matrice è simmetrica, possiamo considerare solo gli elementi di A con $j \leq i$. Otteniamo per $i \in \{1, \dots, n\}$, $j \in \{1, \dots, i-1\}$:

$$a_{i,j} = \sum_{k=1}^{j-1} l_{i,k} l_{j,k} + l_{i,j} l_{j,j}, \quad a_{i,i} = \sum_{k=1}^{i-1} l_{i,k}^2 + l_{i,i}^2.$$

Da cui si ricavano le formule per gli elementi di L per $i \in \{2, \dots, n\}$, $j \in \{1, \dots, i-1\}$:

$$l_{1,1} = \sqrt{a_{1,1}}, \quad l_{i,j} = \frac{1}{l_{j,j}} \left(a_{i,j} - \sum_{k=1}^{j-1} l_{i,k} l_{j,k} \right), \quad l_{i,i} = \sqrt{a_{i,i} - \sum_{k=1}^{i-1} l_{i,k}^2}.$$

È lecito assumere che l'operazione di radice quadrata sia equivalente (in termini di tempo) alle operazioni di prodotto e divisione.

Schema Algoritmo

```

l(1,1) = sqrt(a(1,1))           % 1 operazione
for i = 2:n
    for j = 1:i-1
        l(i,j) = 0;
        for k = 1:j-1
            l(i,j) = l(i,j) + l(i,k) * l(j,k)
        end
    end
end

```

```

        l(i,j) = (a(i,j) - l(i,j)) / l(j,j)    % j-1 operazioni + 1 op.
    end
    l(i,i) = 0;
    for k = 1:i-1
        l(i,i) = l(i,i) + l(i,k)^2
    end
    l(i,i) = sqrt(a(i,i) - l(i,i))            % i-1 operazioni + 1 op.
end

```

Costo Computazionale

$$\begin{aligned}
 1 + \sum_{i=2}^n \left[i + \sum_{j=1}^{i-1} j \right] &= \sum_{i=1}^n i + \sum_{i=1}^n \frac{i(i-1)}{2} \\
 &= \sum_{i=1}^n i + \frac{1}{2} \sum_{i=1}^n i^2 - \frac{1}{2} \sum_{i=1}^n i = \frac{1}{2} \sum_{i=1}^n i + \frac{1}{2} \sum_{i=1}^n i^2 \\
 &= \frac{n(n+1)}{4} + \frac{n(n+1)(2n+1)}{12} \simeq \frac{n^3}{6}
 \end{aligned}$$

```

1 function L = cholDec(A)
2 % CHOLDEC Esegue la fattorizzazione di Cholesky.
3 % L = cholDec(A) esegue la fattorizzazione di Cholesky (A=L*L')
4 % La matrice A deve essere Simmetrica Definita Positiva.
5 %
6 % Input:
7 %   A - Matrice dei coefficienti (n x n)
8 %
9 % Output:
10 %   L - Matrice triangolare inferiore (n x n)
11
12 %% 1. Validazione degli Input
13 arguments
14     A (:,:) double {mustBeNumeric}
15 end
16
17 %% 2. Verifica Dimensioni
18 % Estraggo le dimensioni
19 [n, m] = size(A);
20
21 % Verifico se la matrice è quadrata
22 if n ~= m
23     error('cholDec:NonSquareMatrix', 'La matrice dei coefficienti deve
24     essere quadrata (Size: %dx%d).', n, m);
25 end
26
27 %% 3. Metodo di Cholesky
28 L = zeros(n);
29
30 % Primo elemento
31 L(1,1) = sqrt(A(1,1));
32
33 for i = 2:n
34     % Calcolo elementi fuori diagonale
35     for j = 1:i-1

```

```

36
37     for k = 1:j-1
38         L(i,j) = L(i,j) + L(i,k) * L(j,k);
39     end
40
41     L(i,j) = (A(i,j) - L(i,j)) / L(j,j);
42 end
43
44 % Calcolo elemento diagonale
45 L(i,i) = 0; % Inizializza accumulatore
46 for k = 1:i-1
47     L(i,i) = L(i,i) + L(i,k)^2;
48 end
49
50 % Radice quadrata finale per la diagonale
51 argRad = A(i,i) - L(i,i);
52
53 % Controllo di sicurezza
54 if argRad <= 0
55     error('cholDec:nonDefPosMatrix','La matrice non è definita positiva
56     .')
57 end
58
59 L(i,i) = sqrt(argRad);
60 end
end

```

Function cholDec

Stabilità e Risoluzione È possibile provare che l'algoritmo di Cholesky è stabile. Ricordiamo che, peraltro, anche l'algoritmo di Gauss, senza pivoting, è stabile per le matrici simmetriche definite positive. Molte routine automatiche calcolano $R = {}^tL$ anziché L , ma è evidente che l'algoritmo è lo stesso (con un attento uso degli indici), data la simmetria della matrice di partenza A . In tali casi scriveremo:

$$A = {}^tRR$$

Osserviamo infine che se si vuole risolvere il sistema $Ax = \mathbf{b}$ mediante la fattorizzazione di Cholesky basterà, una volta computata la fattorizzazione, e quindi calcolata L (o rispettivamente R), risolvere i seguenti due sistemi:

$$Ly = \mathbf{b}, \quad {}^tLx = \mathbf{y}$$

(oppure ${}^tRy = \mathbf{b}$, $Rx = \mathbf{y}$ rispettivamente).

Le function Matlab Chiudiamo questa carrellata dei principali metodi diretti per la risoluzione di un sistema lineare descrivendo le principali function di Matlab che implementano tali metodi. Sia allora A una matrice non singolare di ordine n e $\mathbf{b}$ un vettore colonna di ordine n .

- **Risoluzione dei sistemi diagonali:** Se la matrice dei coefficienti A è diagonale, $A \setminus \mathbf{b}$ risolve il sistema mediante n operazioni di divisione.
- **Risoluzione dei sistemi triangolari:** Se la matrice dei coefficienti A è triangolare (superiore o inferiore), $A \setminus \mathbf{b}$ risolve il sistema mediante algoritmo di sostituzione (all'indietro o in avanti a seconda della struttura della matrice).

- **Risoluzione di sistemi con matrice qualsiasi:** $A \setminus b$ risolve il sistema mediante metodo di eliminazione di Gauss (in realtà con la fattorizzazione LU) con pivoting e gli algoritmi di sostituzione.
- **Risoluzione di sistemi simmetrici def. positivi:** $A \setminus b$ risolve il sistema mediante metodo di Cholesky e gli algoritmi di sostituzione.
- **Fattorizzazione LU:** Il comando $[L,U,P] = \text{lu}(A)$ calcola i fattori L, U e la matrice di permutazione P tali che $PA = LU$.
- **Fattorizzazione di Cholesky:** Se la matrice A è simmetrica e definita positiva, il comando $R = \text{chol}(A)$ calcola il fattore triangolare superiore R tale che $A = {}^t R R$.
- **Calcolo del determinante:** $\det(A)$ calcola il determinante della matrice non singolare A , utilizzando la fattorizzazione LU.
- **Condizionamento:** Il comando $\text{cond}(A, p)$, con $p = 1, 2, \text{inf}, 'fro'$, calcola il numero di condizionamento rispettivamente in norma 1, 2, infinito e Frobenius.
- **Inversa di una matrice:** $\text{inv}(A)$ calcola l'inversa della matrice utilizzando la fattorizzazione LU.

3.6 Stabilità dei Metodi di Gauss

Backward Error Analysis Con la **Backward Error Analysis** (BEA) (Analisi dell'errore all'indietro), si misura la perturbazione nei dati di ingresso che porta al risultato finale, ipotizzando che le operazioni siano eseguite con precisione infinita. Si valuta in sostanza quanto è perturbato il dato di input considerando "esatto" il risultato in output. Essa è stata introdotta da Wilkinson nel 1963 con l'obiettivo di studiare gli errori negli algoritmi di Algebra Lineare ed è particolarmente adatta quando il numero di operazioni è elevato. L'algoritmo è detto **stabile** se la perturbazione nel dato di ingresso è dell'ordine della precisione in cui si lavora.

Backward Error Analysis nella fattorizzazione LU

Teorema 3.8. *Sia A una matrice di ordine n i cui elementi sono numeri di macchina. Siano L ed U le matrici della fattorizzazione ottenute senza la tecnica del pivoting. Esiste allora una matrice E per cui:*

$$(A + E) = LU$$

con E tale che:

$$\|E\| \leq nu \|L\| \|U\|$$

essendo $u = \frac{\epsilon}{2}$.

Poiché $U = A^{(n-1)}$ la stabilità del metodo di eliminazione di Gauss può essere meglio compresa misurando la crescita degli elementi delle matrici $A^{(k)}$ e, quindi, misurando il fattore di crescita.

Fattore di Crescita

Definizione 3.10. Si definisce fattore di crescita la seguente quantità:

$$\rho = \frac{\max(\alpha_1, \dots, \alpha_n)}{\alpha_1}$$

con:

$$\alpha_1 = \max_{1 \leq i, j \leq n} |a_{i,j}|$$

$$\alpha_k = \max_{1 \leq i, j \leq n} |a_{i,j}^{(k)}|, \quad k \in \{2, \dots, n\}$$

essendo $A^{(k)} = (a_{i,j}^{(k)})_{i,j \in \{1, \dots, n\}}$ la matrice ottenuta al termine del passo k -esimo della fattorizzazione LU.

```

1 function [L, U, rho] = factorLUr(A)
2 % FACTORLUR Esegue la fattorizzazione LU su una matrice A.
3 % [L, U, rho] = factorLUr(A) fattorizza A in L*U (Gauss no pivoting)
4 % e restituisce il fattore di crescita rho.
5 %
6 % Input:
7 %   A - Matrice dei coefficienti (n x n)
8 %
9 % Output:
10 %   L - Matrice triangolare inferiore unitaria (n x n)
11 %   U - Matrice triangolare superiore (n x n)
12 %   rho - Fattore di crescita per la stabilità
13
14 %% 1. Validazione degli Input
15 arguments
16     A (:,:) double {mustBeNumeric}
17 end
18
19 %% 2. Verifica Dimensioni
20 % Estraggo le dimensioni
21 [n, m] = size(A);
22
23 % Verifica matrice quadrata
24 if n ~= m
25     error('factorLUr:NonSquareMatrix', 'La matrice deve essere quadrata (%
dx%d).', n, m);
26 end
27
28 %% 3. Inizializzazione Fattore di Crescita
29 % Pre-calcolo modulo per rho
30 absA = abs(A);
31
32 % Vettore per tracciare i massimi locali ad ogni passo
33 rhos = zeros(1,n);
34
35 % Memorizzo il massimo iniziale assoluto (denominatore di rho)
36 rhos(1) = max(absA(:));
37
38 %% 4. Eliminazione di Gauss
39 for k = 1 : n-1
40     % Controllo pivot nullo
41     if A(k,k) == 0
42         error('factorLUr:ZeroPivot', 'Pivot nullo alla riga %d.', k);
43     end

```

```

44
45     % Ciclo sulle righe sottostanti (i)
46     for i = k+1 : n
47         % Calcolo moltiplicatore (salvato con segno negativo)
48         A(i,k) = -A(i,k) / A(k,k);
49
50         % Aggiornamento riga i-esima (Eliminazione)
51         for j = k+1:n
52             A(i,j) = A(i,j) + A(i,k) * A(k,j);
53         end
54     end
55
56     % --- Calcolo Massimo Locale per Rho ---
57     % Estraggo sottomatrice attiva corrente (dimensioni n-k+1)
58     tempSub = abs(A(k:n, k:n));
59
60     % Azzeramento moltiplicatori: si trovano nella colonna 1 locale,
61     % dalla riga 2 in poi (indici locali). Vanno esclusi dal max su U.
62     tempSub(2:end, 1) = 0;
63
64     % Memorizzo il massimo valore trovato nella parte attiva/U
65     rhos(k+1) = max(tempSub(:));
66 end
67
68 %% 5. Costruzione Output
69 L = eye(n) - tril(A, -1);
70 U = triu(A);
71
72 % Calcolo finale rho: max crescita / max iniziale
73 rho = max(rhos) / rhos(1);
74 end

```

Function LUR

La definizione vale sia con pivoting che senza pivoting. Osserviamo che, sebbene il pivoting faccia in modo che i moltiplicatori siano tutti più piccoli di 1, gli elementi delle matrici $A^{(k)}$ possono ancora crescere arbitrariamente.

Backward Error Analysis nella fattorizzazione LUPP

Teorema 3.9. Siano L, U le matrici della fattorizzazione con pivoting parziale. Si ha:

$$P(A + E) = LU$$

con E tale che:

$$\|E\| \leq n^3 u \rho \|A\|_{\infty}$$

essendo $u = \frac{\epsilon}{2}$.

Ci domandiamo: quanto grande può essere ρ ?

Crescita del fattore ρ nel pivoting

Teorema 3.10. Per la fattorizzazione LU con pivoting parziale (e per l'eliminazione di Gauss

con pivoting), per ogni matrice A di ordine n si ha:

$$\rho \leq 2^{n-1}$$

```

1 function [L, U, P, s, rho] = factorLUPPr(A)
2 % FACTORLUPPR Fattorizzazione LU con Pivoting Parziale (PA = LU).
3 % [L, U, P, s, rho] = factorLUPPr(A)
4 %
5 % Input:
6 %   A - Matrice dei coefficienti (n x n)
7 %
8 % Output:
9 %   L - Matrice triangolare inferiore unitaria
10 %   U - Matrice triangolare superiore
11 %   P - Matrice di permutazione
12 %   s - Numero di scambi effettuati (per il determinante)
13 %   rho - Fattore di crescita (stabilità numerica)
14
15 %% 1. Validazione Input
16 arguments
17     A (:,:) double {mustBeNumeric}
18 end
19
20 %% 2. Verifica Dimensioni
21 % Estraggo le dimensioni
22 [n, m] = size(A);
23
24 % Verifico se la matrice è quadrata
25 if n ~= m
26     error('factorLUPPR:NonSquareMatrix', 'La matrice deve essere quadrata (
Size: %dx%d).', n, m);
27 end
28
29 %% 3. Inizializzazione
30 absA = abs(A);
31 P = eye(n); % Matrice di permutazione iniziale
32 s = 0; % Contatore scambi
33
34 % Inizializzazione vettore per rho
35 rhos = zeros(1,n);
36 if n > 0
37     rhos(1) = max(absA(:)); % Massimo iniziale
38 else
39     rhos(1) = 0;
40 end
41
42 %% 4. Eliminazione di Gauss con Pivoting Parziale
43 for k = 1 : n-1
44     % Pivoting parziale
45     % Trovo il massimo nella colonna k (dal pivot in giù)
46     [~, r_rel] = max(abs(A(k:n, k)));
47
48     % Converto in indice globale
49     r_glob = r_rel + k - 1;
50
51     % Se il pivot migliore non è quello attuale, scambio
52     if r_glob ~= k
53         % Scambio righe in A (compresi i moltiplicatori precedenti)
54         A([k, r_glob], :) = A([r_glob, k], :);
55

```



```

56     % Scambio righe in P
57     P([k, r_glob], :) = P([r_glob, k], :);
58
59     % Aggiorno variabile s
60     s = s + 1;
61 end
62
63 % Controllo pivot nullo (sicurezza)
64 if A(k,k) == 0
65     error('factorLUPPr:SingularMatrix', 'Matrice singolare: pivot
nullo alla riga %d.', k);
66 end
67
68 % Eliminazione
69 for i = k+1 : n
70     % Calcolo e assegnazione del moltiplicatore
71     A(i,k) = -A(i,k) / A(k,k);
72
73     % Aggiornamento riga i-esima
74     for j = k+1:n
75         A(i,j) = A(i,j) + A(i,k) * A(k,j);
76     end
77 end
78
79 % Calcolo rho (Massimo Locale)
80 % Estraggo sottomatrice attiva
81 tempSub = abs(A(k:n, k:n));
82
83 % Azzerare i moltiplicatori appena calcolati
84 % per misurare solo la crescita degli elementi di U.
85 tempSub(2:end, 1) = 0;
86
87 rhos(k+1) = max(tempSub(:));
88 end
89
90 %% 5. Costruzione Output
91 % L: Identità - parte triangolare inferiore (segno compensato)
92 L = eye(n) - tril(A, -1);
93
94 % U: Parte triangolare superiore
95 U = triu(A);
96
97 % Calcolo Rho finale
98 rho = max(rhos) / rhos(1);
99 end

```

Function LUPPr

Sfortunatamente esistono matrici che hanno fattore di crescita pari a 2^{n-1} . Una di esse è la matrice di Wilkinson:

$$a_{ij} = \begin{cases} 1, & i = j \\ -1, & i > j \\ 1, & j = n \\ 0 & \text{altrove} \end{cases}$$

Il pivoting totale Si sceglie una coppia (r, s) , con $r, s \geq k$ tale che:

$$|a_{rs}^{(k)}| = \max_{k \leq i, j \leq n} |a_{ij}^{(k)}|$$

e si scambia l'equazione k -esima con la r -esima e l'incognita k -esima (con il suo coefficiente) con la s -esima. Nella fattorizzazione LU con pivoting totale si costruiscono due matrici di permutazione P e Q tali che:

$$PAQ = LU$$

Il sistema lineare $Ax = b$ diventa equivalente a $(Q^{-1} = {}^tQ)$:

$$PAQ {}^tQx = Pb$$

Posto ${}^tQx = y$ si ha $PAQy = Pb$ ed usando la fattorizzazione LU si ha $LUy = Pb$. Posto $Uy = z$ si calcola il vettore soluzione di $Lz = Pb$ e poi il vettore soluzione di $Uy = z$. Infine, si calcola la soluzione x del sistema iniziale $x = Qy$. Nella fattorizzazione LU con pivoting totale il fattore di crescita ρ cresce molto lentamente, si prova infatti che:

$$\rho \leq \sqrt{n \cdot 2^1 \cdot 3^{\frac{1}{2}} \cdot 4^{\frac{1}{3}} \cdots n^{\frac{1}{n-1}}}.$$

```

1 function [L, U, P, Q] = factorLUTP(A)
2 % FACTORLUTP Esegue la fattorizzazione LU con pivoting totale.
3 % [L, U, P, Q] = factorLUTP(A) calcola la fattorizzazione PAQ = LU
4 % dove P e Q sono matrici di permutazione.
5 %
6 % Input:
7 % A - Matrice quadrata dei coefficienti (n x n)
8 %
9 % Output:
10 % L - Matrice triangolare inferiore unitaria (n x n)
11 % U - Matrice triangolare superiore (n x n)
12 % P - Matrice di permutazione righe (n x n)
13 % Q - Matrice di permutazione colonne (n x n)
14
15 %% 1. Validazione degli Input
16 arguments
17 A(:, :) double {mustBeNumeric}
18 end
19
20 %% 2. Verifica Dimensioni
21 [n, m] = size(A);
22 if n ~= m
23     error('factorLUTP:NonSquareMatrix', 'La matrice deve essere quadrata (
Size: %dx%d).', n, m);
24 end
25
26 %% 3. Inizializzazione
27 % Inizializzo le matrici di permutazione come identità
28 P = eye(n);
29 Q = eye(n);
30
31 %% 4. Eliminazione di Gauss con Pivoting Totale
32 for k = 1 : n-1
33     % Ricerca pivot
34     subA = abs(A(k:n, k:n)); % Estraggo la sottomatrice corrente
35
36     % Trovo il massimo valore e il suo indice lineare
37     [max_val, idx_lin] = max(subA(:));
38
39     % Converto l'indice lineare in indici di riga/colonna relativi alla
sottomatrice
40     [r_local, c_local] = ind2sub(size(subA), idx_lin);

```

```

41
42     % Converto in indici globali (riferiti alla matrice A intera)
43     r_glob = r_local + k - 1;
44     c_glob = c_local + k - 1;
45
46     % Controllo singolarità
47     if max_val == 0
48         error('factorLUTP:SingularMatrix', 'La matrice è singolare (pivot
49         nullo).');
50     end
51
52     % Scambio righe (A e P)
53     if r_glob ~= k
54         A([k, r_glob], :) = A([r_glob, k], :);
55         P([k, r_glob], :) = P([r_glob, k], :);
56     end
57
58     % Scambio colonne (A e Q)
59     if c_glob ~= k
60         A(:, [k, c_glob]) = A(:, [c_glob, k]);
61         Q(:, [k, c_glob]) = Q(:, [c_glob, k]);
62     end
63
64     % Eliminazione di Gauss
65     A(k+1:n, k) = -A(k+1:n, k) / A(k, k);
66     A(k+1:n, k+1:n) = A(k+1:n, k+1:n) + A(k+1:n, k) * A(k, k+1:n);
67
68     %% 5. Costruzione Output
69     U = triu(A);
70     L = eye(n) - tril(A, -1);
71
72 end

```

Function factorLUTP

Conclusioni sulla stabilità Il metodo di eliminazione di Gauss senza pivoting ha fattore di crescita pari a 1 per matrici simmetriche definite positive e ≤ 2 per matrici a diagonale dominante per righe o per colonne. Da cui segue la stabilità del metodo di Gauss senza pivoting quando viene applicato a matrici simmetriche definite positive o a diagonale dominante per righe o per colonne.

Tabella riassuntiva

- **GE** (o $A = LU$): non stabile in generale (stabile se la matrice è simmetrica definita positiva oppure a diagonale dominante).
- **GEPP** (o $PA = LU$): quasi sempre stabile.
- **GETP** (o $PAQ = LU$): stabile.

Residuo e sua valutazione In riferimento ai sistemi lineari quadrati, sia x la soluzione esatta ed x^* la soluzione calcolata. Dimostriamo che vale la seguente stima dell'errore relativo:

Proposizione 3.11.

$$\frac{\|\mathbf{x} - \mathbf{x}^*\|}{\|\mathbf{x}\|} \leq K(A) \frac{\|A\mathbf{x}^* - \mathbf{b}\|}{\|\mathbf{b}\|}.$$

Dalla stima precedente si deduce che la norma del residuo $\|A\mathbf{x}^* - \mathbf{b}\|$, pur essendo piccola (magari dell'ordine di ϵ), non garantisce un errore relativo piccolo, in particolare per matrici con elevato indice di condizionamento. Quindi la quantità $\|A\mathbf{x}^* - \mathbf{b}\|$ deve essere usata con cautela come stimatore a posteriori dell'errore.

3.7 Esercizi

1. Si consideri la matrice

$$A = \text{rand}(10), \quad A = A * {}^t A.$$

- Verificare se è simmetrica;
- Verificare se è simmetrica e definita positiva;

2. Si consideri la matrice

$$A = \text{rand}(10) + 100 * \text{diag}(\text{ones}(1, 10)).$$

Verificare se è a diagonale dominante per righe.

3. Scrivere una function Matlab che verifichi che una matrice è a diagonale dominante per colonne.
4. Si consideri la matrice

$$A(i, j) = \begin{cases} -1 & i > j \\ 0 & i < j, \\ 100 & i = j \end{cases} \quad i, j \in \{1, \dots, n\}$$

con $n = 15$.

- Verificare se è a diagonale dominante per colonne;
- Verificare se è a diagonale dominante per righe.

5. Si consideri il sistema lineare $A\mathbf{x} = \mathbf{b}$ con

$$A = \text{tril}(\text{rand}(10)), \quad \mathbf{b} = \text{sum}(A, 2).$$

Risolvere il sistema con il metodo di sostituzione in avanti.

6. Si consideri il sistema lineare $A\mathbf{x} = \mathbf{b}$ con

$$A = \text{triu}(\text{rand}(10)), \quad \mathbf{b} = \text{sum}(A, 2).$$

- Risolvere il sistema con il metodo di sostituzione all'indietro;
- calcolare l'inversa di A .

7. Risolvere il sistema lineare $A\mathbf{x} = \mathbf{b}$ con

$$A = \text{diag}(\text{diag}(\text{rand}(10))), \quad \mathbf{b} = \text{sum}(A, 2).$$

8. Calcolare l'inversa della matrice

$$A = \text{tril}(\text{rand}(20)).$$

9. Si consideri il sistema lineare $A\mathbf{x} = \mathbf{b}$ con

$$A = \text{rand}(100), \quad \mathbf{b} = \text{sum}(A, 2).$$

Risolvere il sistema con il metodo di Gauss.

10. Si consideri il sistema di equazioni lineari $A\mathbf{x} = \mathbf{b}$ di ordine $n = 15$, con

$$A(i, j) = \begin{cases} -1 & i > j \\ 0 & i < j, \\ 100 & i = j \end{cases} \quad i, j \in \{1, \dots, n\}$$

e

$$\mathbf{b} = {}^t(1, 1, \dots, 1).$$

- Calcolare l'indice di condizionamento e il numero massimo di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Calcolare il vettore soluzione con il metodo di sostituzione in avanti e riportarne le prime due componenti con le cifre significative che si possono certamente ritenere corrette.
- Confrontare la soluzione ottenuta (vettore $\mathbf{x}$) con la soluzione che si ottiene usando la function predefinita del Matlab (vettore $\mathbf{y}$). Di quanto differiscono al massimo?

11. Si consideri il sistema di equazioni lineari $A\mathbf{x} = \mathbf{b}$ di ordine $n = 10$, con

$$A = \begin{pmatrix} \frac{1}{2} & 2 & 4 & 4 & \dots & 4 & 4 \\ 1 & \frac{1}{3} & 2 & 4 & \dots & & 4 \\ 2 & 1 & \frac{1}{4} & \ddots & \ddots & & \vdots \\ 0 & 2 & \ddots & \ddots & \ddots & 4 & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 2 & 4 \\ 0 & \dots & 0 & 2 & 1 & \frac{1}{n} & 2 \\ 0 & \dots & \dots & 0 & 2 & 1 & \frac{1}{n+1} \end{pmatrix}$$

e

$$\mathbf{b} = (b_i)_{i \in \{1, \dots, n\}}, \quad b(i) = \sum_{j=1}^n A_{i,j}.$$

- Calcolare l'indice di condizionamento e il numero massimo di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Calcolare il vettore soluzione con il metodo di eliminazione di Gauss e riportarne le prime due componenti con le cifre significative che si possono certamente ritenere corrette.
- Confrontare la soluzione ottenuta (vettore $\mathbf{x}$) con la soluzione che si ottiene usando la function predefinita del Matlab (vettore $\mathbf{y}$). Di quanto differiscono al massimo?

- Confrontare la soluzione ottenuta (vettore $\mathbf{x}$) con il vettore $\mathbf{t} = {}^t(1, \dots, 1)$ che rappresenta la soluzione esatta del sistema. Di quanto differiscono al massimo?

12. Si consideri il sistema di equazioni lineari $A\mathbf{x} = \mathbf{b}$ di ordine $n = 100$, con

$$A = \begin{pmatrix} 1 & 1 & 4 & 0 & \dots & 0 & 0 \\ 6 & 3 & 1 & 4 & \ddots & & 0 \\ 0 & 6 & 5 & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \ddots & \ddots & \ddots & 4 & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & 1 & 4 \\ 0 & \dots & 0 & 0 & 6 & \ddots & 1 \\ 0 & \dots & \dots & \dots & 0 & 6 & 2n-1 \end{pmatrix}$$

e

$$\mathbf{b} = (b_i)_{i \in \{1, \dots, n\}}, \quad b(i) = \sum_{j=1}^n A_{i,j}.$$

- Calcolare l'indice di condizionamento e il numero massimo di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Calcolare il vettore soluzione con il metodo di eliminazione di Gauss e riportare le prime due componenti del vettore soluzione con le cifre significative che si possono certamente ritenere corrette.
- Confrontare la soluzione ottenuta (vettore $\mathbf{x}$) con il vettore $\mathbf{t} = {}^t(1, \dots, 1)$ che rappresenta la soluzione esatta del sistema. Di quanto differiscono al massimo?

13. Consideriamo il sistema lineare $A\mathbf{x} = \mathbf{b}$ di ordine $n = 18$, dove

$$A_{i,j} = \cos\left((j-1)\frac{2i-1}{2n}\pi\right), \quad i, j \in \{1, \dots, n\},$$

e

$$b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\},$$

la cui soluzione esatta è $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Calcolare la soluzione approssimata del sistema utilizzando il metodo di Gauss e calcolare l'errore relativo. Quante sono le cifre significative corrette?
- Calcolare la soluzione approssimata del sistema utilizzando il metodo di Gauss con pivoting parziale e calcolare l'errore relativo. Quante sono le cifre significative corrette?
- Qual è il metodo più stabile?

14. Consideriamo il sistema lineare $A\mathbf{x} = \mathbf{b}$ di ordine $n = 50$, dove

$$A = \begin{pmatrix} 3 & 2 & 2 & 2 & \dots & 2 & 6 \\ 2 & \frac{5}{2} & 2 & 2 & \ddots & & 2 \\ 2 & 2 & \frac{7}{3} & 2 & \ddots & \ddots & \vdots \\ 2 & 2 & 2 & \ddots & \ddots & 2 & 2 \\ \vdots & \vdots & \ddots & \ddots & \ddots & 2 & 2 \\ 2 & 2 & \dots & 2 & 2 & 2 + \frac{1}{n-1} & 2 \\ 6 & 2 & 2 & \dots & 2 & 2 & 2 + \frac{1}{n} \end{pmatrix}$$

e

$$b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\},$$

la cui soluzione esatta è $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Calcolare la soluzione approssimata del sistema utilizzando il metodo di Gauss e calcolare l'errore relativo. Quante sono le cifre significative corrette?
- Calcolare la soluzione approssimata del sistema utilizzando il metodo di Gauss con pivoting parziale e calcolare l'errore relativo. Quante sono le cifre significative corrette?
- Qual è il metodo più stabile?

15. Consideriamo il problema di $n = 80$

$$AX = B, \quad A \in \mathbb{R}^{n \times n}, X \in \mathbb{R}^{n \times 3}, B \in \mathbb{R}^{n \times 3},$$

dove

$$A = \begin{pmatrix} 5 & 0 & \frac{1}{2} & 0 & \dots & 0 \\ 0 & 9 & 0 & \frac{1}{3} & \ddots & \vdots \\ \vdots & 0 & 13 & \ddots & \ddots & 0 \\ 0 & \ddots & \ddots & \ddots & \ddots & \frac{1}{n-1} \\ \frac{1}{3} & \ddots & 0 & \dots & 4(n-1)+1 & 0 \\ 0 & \frac{1}{3} & 0 & \dots & 0 & 4n+1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 2 & 1 \\ 1 & 2 & 2 \\ \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots \\ 1 & 2 & n-1 \\ 1 & 2 & n \end{pmatrix}.$$

(In questa la trascrizione è corretta ma confrontare meglio con pdf esercitazioni per conferma sulla struttura.)

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della matrice soluzione X .
- Calcolare la soluzione approssimata Y utilizzando opportunamente la fattorizzazione LU della matrice A . Riportare le componenti della prima riga della matrice Y con le cifre che si possono ritenere corrette.
- Calcolare il residuo relativo in norma infinito.

16. Scrivere una function Matlab che implementi opportunamente il metodo di eliminazione di Gauss per risolvere un sistema a matrice tridiagonale.
17. Scrivere una function Matlab che implementi opportunamente il metodo di eliminazione di Gauss per risolvere un sistema a matrice di Hessemberg superiore.
18. Consideriamo il sistema lineare $A\mathbf{x} = \mathbf{b}$ di ordine $n = 80$, dove

$$A = \begin{pmatrix} 4 & 2 & 2 & 2 & \dots & 2 & 8 \\ 4 & \frac{7}{2} & 2 & 2 & \ddots & & 2 \\ 4 & 4 & \frac{10}{3} & 2 & \ddots & \ddots & \vdots \\ 4 & 4 & 4 & \ddots & \ddots & 2 & 2 \\ \vdots & \vdots & \ddots & \ddots & \ddots & 2 & 2 \\ 4 & 4 & \dots & 4 & 4 & 3 + \frac{1}{n-1} & 2 \\ 10 & 4 & 4 & \dots & 4 & 4 & 3 + \frac{1}{n} \end{pmatrix}$$

e

$$b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\},$$

la cui soluzione esatta è $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Calcolare la soluzione approssimata del sistema utilizzando opportunamente il metodo di Gauss e calcolare l'errore relativo. Quante sono le cifre significative corrette?
- Calcolare il determinante della matrice A .
- Calcolare l'inversa della matrice A .

19. Data la matrice di ordine $n = 40$

$$A = \begin{pmatrix} \frac{1}{10} & 0 & \frac{1}{2} & 0 & \dots & 0 \\ 0 & \frac{1}{10} & 0 & \frac{1}{2} & \ddots & \vdots \\ \vdots & 0 & \frac{1}{10} & \ddots & \ddots & 0 \\ 0 & \ddots & \ddots & \ddots & \ddots & \frac{1}{2} \\ \frac{1}{2} & \ddots & 0 & \dots & \frac{1}{10} & 0 \\ 0 & \frac{1}{3} & 0 & \dots & 0 & \frac{1}{10} \end{pmatrix},$$

Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolarne:

- il determinante;
- l'inversa.

20. Data la matrice di ordine $n = 40$

$$A = \begin{pmatrix} 10 & 1 & \frac{1}{2} & 3 & \dots & 3 & 3 \\ 0 & 10 & 1 & \frac{1}{3} & \ddots & & 3 \\ 2 & 0 & 10 & 1 & \ddots & \ddots & \vdots \\ \vdots & 2 & 0 & \ddots & \ddots & \frac{1}{n-2} & 3 \\ 2 & & \ddots & \ddots & \ddots & 1 & \frac{1}{n-1} \\ \frac{1}{2} & \ddots & & 2 & 0 & 10 & 1 \\ 1 & \frac{1}{2} & 2 & \dots & 2 & 0 & 10 \end{pmatrix}$$

e

$$b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\},$$

la cui soluzione esatta è $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Calcolare la soluzione approssimata del sistema utilizzando opportunamente il metodo di Gauss e calcolare l'errore relativo. Quante sono le cifre significative corrette?
- Calcolare il Determinante della matrice A .
- Calcolare l'inversa della matrice A .

21. Consideriamo il sistema lineare $A\mathbf{x} = \mathbf{b}$ di ordine $n = 100$, dove

$$A = \begin{pmatrix} 9 & 0 & 2 & 0 & \dots & 0 & 1 \\ 4 & 5 & 0 & 2 & 0 & \dots & 0 \\ 4 & 0 & 5 & \ddots & \ddots & \ddots & \vdots \\ \vdots & 0 & \ddots & \ddots & 0 & 2 & 0 \\ \vdots & \vdots & \ddots & \ddots & 5 & 0 & 2 \\ 4 & 0 & \dots & 0 & 0 & 5 & 0 \\ 4 & 0 & \dots & 0 & 0 & 0 & 5 \end{pmatrix}, \quad A = A * {}^t A$$

e

$$b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\},$$

la cui soluzione esatta è $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Proporre uno o più metodi numerici per calcolare la soluzione del sistema $A\mathbf{x} = \mathbf{b}$ con la massima precisione possibile. Calcolare l'errore relativo in norma infinito. Quante sono le cifre significative corrette della soluzione calcolata?
- Motivare la scelta del metodo effettuata e commentare i risultati ottenuti.
- Calcolare il determinante di A .
- Calcolare l'inversa di A .

- Verificare che il fattore di crescita ρ relativo alla fattorizzazione LU è uguale a 1.

22. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 100$, dove

$$A(i, j) = \begin{cases} -7 & i = j \\ -2 & |i - j| = 1 \\ 1 & |i - j| = 4 \\ 0 & \text{altrimenti} \end{cases}$$

e

$$b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\},$$

la cui soluzione esatta è $x = {}^t(1, \dots, 1)$.

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Proporre uno o più metodi numerici per calcolare la soluzione del sistema $Ax = b$ con la massima precisione possibile. Calcolare l'errore relativo in norma infinito. Quante sono le cifre significative corrette della soluzione calcolata?
- Valutare la stabilità dei metodi proposti mediante il calcolo del fattore di crescita ρ .
- Motivare la scelta del metodo effettuata e commentare i risultati ottenuti.
- Calcolare il determinante di A .
- Calcolare l'inversa di A .
- Verificare che il fattore di crescita ρ relativo alla fattorizzazione LU è minore di 2.

23. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 80$, dove

$$A = \begin{pmatrix} 15 & -3 & -3 & -3 & \dots & -3 \\ 2 & 15 & -3 & -3 & \ddots & \vdots \\ 0 & 2 & 15 & \ddots & \ddots & -3 \\ \vdots & & \ddots & \ddots & \ddots & -3 \\ \vdots & \ddots & & 2 & 15 & -3 \\ 0 & \dots & \dots & 0 & 2 & 4 \end{pmatrix}$$

e

$$b = {}^t(1, \dots, 1).$$

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Calcolare la soluzione approssimata y del sistema utilizzando opportunamente il metodo di Gauss. Riportare le prime 2 componenti del vettore y con le cifre che si possono ritenere corrette.
- Verificare la stabilità del metodo di Gauss usato, valutando il suo fattore di crescita ρ .
- Calcolare la norma infinito del residuo $Ay - b$.

- Calcolare il determinante di A .
- Calcolare l'inversa di A .

24. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 60$, con A matrice di Wilkinson

$$A = \begin{pmatrix} 1 & 0 & 0 & 0 & \dots & 0 & 1 \\ -1 & 1 & 0 & 0 & & \vdots & 1 \\ -1 & -1 & 1 & \ddots & \ddots & \vdots & \vdots \\ \vdots & -1 & \ddots & \ddots & 0 & 0 & \vdots \\ \vdots & \vdots & \ddots & \ddots & 1 & 0 & 1 \\ -1 & -1 & \dots & -1 & -1 & 1 & 1 \\ -1 & -1 & \dots & -1 & -1 & -1 & 1 \end{pmatrix}$$

e $\mathbf{b}(i) = \sum_{j=1}^n a_{ij}$, $i = 1 : n$

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
 - Stabilire in base a proprietà eventuali della matrice quale variante del metodo di Gauss o fattorizzazione è più conveniente usare per garantire la massima stabilità al minimo costo.
 - Calcolare il fattore di crescita ρ se si usa il GE.
 - Calcolare la soluzione e l'errore relativo commesso.
 - Calcolare il fattore di crescita ρ se si usa il GEPP.
 - Calcolare la soluzione e l'errore relativo commesso.
 - Fare opportune considerazioni sui risultati ottenuti
 - Calcolare la soluzione con il GETP e l'errore relativo commesso. Utilizzare opportunamente la function predefinita del Matlab `lu`.
 - Fare opportune considerazioni sui risultati ottenuti
 - Ripetere lo stesso esercizio con $n = 50$.
25. Scrivere una function Matlab che calcoli il fattore ρ di crescita per la valutazione dell'errore algoritmico in GE e GEPP.
26. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 60$, con $\mathbf{b}(i) = i$ $i = 1, 2, \dots, n$ ed $A = G * G'$, essendo

$$\begin{aligned} G(i, i+1) &= \cos(2i+1), \quad i = 1, 2, \dots, n-1 \\ G(i, i) &= n+1, \quad i = 1, 2, \dots, n \\ G(i, 1) &= 1, \quad i \geq 2 \\ &0 \quad \text{altrove} \end{aligned}$$

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Stabilire in base a proprietà eventuali della matrice quale variante del metodo di Gauss o fattorizzazione è più conveniente usare per garantire la massima stabilità al minimo costo.

- Calcolare il fattore di crescita ρ se si usa il GEPP.
- Calcolare la soluzione e l'errore relativo commesso.
- Fare opportune considerazioni sui risultati ottenuti

27. Consideriamo i sistemi lineari

$$Gx = c$$

$$Gy = d$$

con $G = (g_{i,j})_{i,j=1,\dots,n}$ matrice dell'esercizio precedente e con

$$c(i) = \sum_{j=1}^n g_{i,j}, \quad i = 1, \dots, n,$$

$$d(i) = 1, \quad i = 1 : n$$

- Calcolare l'indice di condizionamento e il numero di cifre significative corrette che ci si può aspettare nel calcolo della soluzione approssimata.
- Stabilire in base a proprietà eventuali della matrice quale variante del metodo di Gauss o fattorizzazione è più conveniente usare per garantire la massima stabilità al minimo costo.
- Calcolare il fattore di crescita ρ se si usa il GEPP.
- Calcolare la soluzione e l'errore relativo commesso.
- Fare opportune considerazioni sui risultati ottenuti

28. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 80$ con

$$A = \begin{pmatrix} 10 & 1 & 0 & \dots & 0 & 2 & 200 \\ 1 & 10 & 2 & \ddots & & \ddots & 2 \\ 0 & 1 & 10 & \ddots & \ddots & & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & & \ddots & \ddots & \ddots & n-2 & 0 \\ 2 & \ddots & & \ddots & 1 & 10 & n-1 \\ 100 & 2 & 0 & \dots & 0 & 1 & 10 \end{pmatrix}$$

e

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^n A_{i,j}, \quad i = 1, \dots, n.$$

- Riportare le istruzioni Matlab utilizzate per la costruzione di A e b .
- Calcolare la soluzione approssimata del sistema con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e le prime 2 componenti del vettore soluzione con le cifre che si possono ritenere corrette.
- Motivare la scelta del metodo.
- Qual è il costo computazionale del metodo numerico utilizzato?

- Poichè è noto che la soluzione è $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calcolare l'errore relativo. Riportarne il valore e le istruzioni Matlab utilizzate. Quante sono le cifre significative corrette?
- Commentare i risultati ottenuti.

29. Determinare la matrice inversa della matrice data nell'esercizio precedente. Indicare:

- la procedura utilizzata
- la fattorizzazione più adeguata per garantire stabilità
- il costo computazionale.

30. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 100$ con

$$A = \begin{pmatrix} 2 & 0 & \dots & 0 & 1 & 3 \\ 0 & 4 & 0 & \dots & 0 & 1 \\ 2 & 0 & 6 & \ddots & \vdots & 0 \\ \vdots & 2 & \ddots & \ddots & 0 & \vdots \\ 2 & \vdots & \ddots & \ddots & 2n-2 & 0 \\ 2 & 2 & \dots & 2 & 0 & 2n \end{pmatrix}$$

e

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Riportare le istruzioni Matlab utilizzate per la costruzione di A e b .
- Calcolare la soluzione approssimata del sistema con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e le prime 2 componenti del vettore soluzione con le cifre che si possono ritenere corrette.
- Motivare la scelta del metodo.
- Qual è il costo computazionale del metodo numerico utilizzato?
- Poichè è noto che la soluzione è $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calcolare l'errore relativo. Riportarne il valore e le istruzioni Matlab utilizzate. Quante sono le cifre significative corrette?
- Commentare i risultati ottenuti.

31. Consideriamo il sistema lineare $AX = B$ di ordine $n = 80$ con

$$A = (a_{i,j})_{i,j=1,\dots,n} = \left(\sqrt{\frac{2}{n+1}} \sin\left(\frac{\pi i j}{n+1}\right) \right)_{i,j=1,\dots,n}$$

e

$$\mathbf{b} = \begin{pmatrix} \sum_{j=1}^n A_{1,j} & 1 & 2 \\ \sum_{j=1}^n A_{2,j} & 1 & 2 \\ \vdots & \vdots & \vdots \\ \sum_{j=1}^n A_{n,j} & 1 & 2 \end{pmatrix}.$$

- Riportare le istruzioni Matlab utilizzate per la costruzione di A e b .

- Calcolare la soluzione approssimata del sistema con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e le prime 2 righe della matrice soluzione con le cifre che si possono ritenere corrette.
- Motivare la scelta del metodo.
- Qual è il costo computazionale del metodo numerico utilizzato?
- Calcolare il residuo relativo. Riportarne il valore e le istruzioni Matlab utilizzate. Quante sono le cifre significative corrette?
- Commentare i risultati ottenuti.

32. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 110$ con

$$A = \begin{pmatrix} 3 & 0 & 0 & 0 & \dots & 0 & 2 \\ 2 & 1 & 0 & 0 & \dots & 0 & 2 \\ 2 & 0 & 1 & \ddots & \ddots & \vdots & \vdots \\ \vdots & 0 & \ddots & \ddots & 0 & 0 & 2 \\ \vdots & \vdots & \ddots & \ddots & 1 & 0 & 2 \\ 2 & 0 & \dots & 0 & 0 & 1 & 2 \\ 2 & 0 & \dots & 0 & 0 & 0 & 3 \end{pmatrix}$$

e

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Riportare le istruzioni Matlab utilizzate per la costruzione di A e b .
- Calcolare la soluzione approssimata del sistema con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e le prime 2 componenti del vettore soluzione con le cifre che si possono ritenere corrette.
- Motivare la scelta del metodo.
- Qual è il costo computazionale del metodo numerico utilizzato?
- Poichè è noto che la soluzione è $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calcolare l'errore relativo. Riportarne il valore e le istruzioni Matlab utilizzate. Quante sono le cifre significative corrette?
- Commentare i risultati ottenuti.

33. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 120$ con

$$A = \begin{pmatrix} 2(n+1) & 2 & 0 & 0 & \dots & 0 \\ -1 & 2(n+1) & 4 & 0 & \dots & 0 \\ 0 & -1 & 2(n+1) & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & & \ddots & -1 & 2(n+1) & 2(n-1) \\ 0 & 0 & \dots & 0 & -1 & 2(n+1) \end{pmatrix}.$$

e

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Riportare le istruzioni Matlab utilizzate per la costruzione di A e b .
- Calcolare la soluzione approssimata del sistema con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e le prime 2 componenti del vettore soluzione con le cifre che si possono ritenere corrette.
- Motivare la scelta del metodo.
- Qual è il costo computazionale del metodo numerico utilizzato?
- Poichè è noto che la soluzione è $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calcolare l'errore relativo. Riportarne il valore e le istruzioni Matlab utilizzate. Quante sono le cifre significative corrette?
- Calcolare l'inversa B della matrice A con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e il valore delle componenti $B(3, 4)$ e $B(4, 4)$ con le cifre che si possono ritenere corrette.
- Commentare i risultati ottenuti.

34. Data la matrice di ordine $n = 80$

$$A = \begin{pmatrix} \frac{1}{4} & -2 & -2 & \dots & -2 & 5 \\ 2 & \frac{1}{8} & -2 & \ddots & & -2 \\ 2 & 2 & \frac{1}{12} & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -2 & -2 \\ 2 & & \ddots & \ddots & \frac{1}{4n-4} & -2 \\ 9 & 2 & \dots & 2 & 2 & \frac{1}{4n} \end{pmatrix}.$$

- Riportare le istruzioni Matlab utilizzate per la sua costruzione.
- Calcolare il determinante della matrice A con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e il valore del determinante con le cifre che si possono ritenere corrette.
- Motivare la scelta del metodo.
- Qual è il costo computazionale del metodo numerico utilizzato?

35. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 80$ con

$$A = (a_{i,j})_{i,j=1,\dots,n} \quad \text{con} \quad a_{i,j} = \min\{i, j\}$$

e

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Riportare le istruzioni Matlab utilizzate per la costruzione di A e b .
- Calcolare la soluzione approssimata del sistema con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e le prime 2 componenti del vettore soluzione con le cifre che si possono ritenere corrette.
- Motivare la scelta del metodo.

- Qual è il costo computazionale del metodo numerico utilizzato?
- Poichè è noto che la soluzione è $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calcolare l'errore relativo. Riportarne il valore e le istruzioni Matlab utilizzate. Quante sono le cifre significative corrette?
- Calcolare il determinante della matrice A con la massima precisione possibile usando una opportuna procedura numerica. Riportare le istruzioni Matlab utilizzate e il valore del determinante con le cifre che si possono ritenere corrette.
- Commentare i risultati ottenuti.

36. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 90$ con

$$A = \begin{pmatrix} 4 & 0 & 0 & \dots & 0 & 3 & 2 \\ 0 & 5 & 0 & & & 3 & 3 \\ 0 & 0 & 6 & \ddots & \vdots & \vdots & \vdots \\ \vdots & & \ddots & \ddots & 0 & \vdots & \vdots \\ 0 & \dots & \dots & 0 & \ddots & 3 & n-1 \\ 2 & 2 & \dots & 2 & 2 & (n-1)+3 & n \\ 2 & 3 & \dots & \dots & n-1 & n & n+3 \end{pmatrix}$$

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Calcolare la soluzione approssimata del sistema con la massima precisione possibile usando una opportuna procedura numerica. Riportare le prime 2 componenti del vettore soluzione con le cifre che si possono ritenere corrette.
- Motivare la scelta del metodo.
- Qual è il costo computazionale del metodo numerico utilizzato?
- Poichè è noto che la soluzione è $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calcolare l'errore relativo. Quante sono le cifre significative corrette?
- Commentare i risultati ottenuti.

37. Consideriamo il sistema lineare $Ax = b$ di ordine $n = 90$ con

$$A = \begin{pmatrix} 1 & 2 & 2 & \dots & 2 & 2 \\ 2 & 3 & 2 & 2 & & 2 \\ 2 & 2 & 5 & \ddots & \ddots & \vdots \\ \vdots & 2 & \ddots & \ddots & 2 & 2 \\ 2 & & \ddots & 2 & 2n-3 & 2 \\ 2 & 2 & \dots & 2 & 2 & 2n-1 \end{pmatrix}$$

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^n A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Calcolare la soluzione approssimata del sistema con la massima precisione possibile usando una opportuna procedura numerica. Riportare le prime 2 componenti del vettore soluzione con le cifre che si possono ritenere corrette.

-
- Motivare la scelta del metodo.
 - Qual è il costo computazionale del metodo numerico utilizzato?
 - Poichè è noto che la soluzione è $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calcolare l'errore relativo. Quante sono le cifre significative corrette?
 - Commentare i risultati ottenuti.

4 Metodi Numerici per la Risoluzione di un Sistema Lineare Rettangolare nel Senso dei Minimi Quadrati

Il problema dei minimi quadrati lineari Consideriamo un sistema lineare del tipo

$$Ax = \mathbf{b}$$

dove A è una matrice di ordine $m \times n$, con $m \neq n$, $\mathbf{x}$ è un vettore di lunghezza n e $\mathbf{b}$ un vettore di lunghezza m . Molte situazioni reali, per esempio nelle applicazioni statistiche, nei problemi di teoria dei segnali, in astronomia, possono essere modellizzate mediante un sistema lineare nel quale la matrice dei coefficienti è rettangolare (o quadrata ma singolare). In tali casi la soluzione può non esistere affatto oppure possono esserci infinite soluzioni.

- Se $m > n$, se cioè il numero delle equazioni è superiore a quello delle incognite, il sistema è detto **sovradeterminato** e non ammette soluzioni.
- Se invece $m < n$ il sistema è detto **sottodeterminato** e ammette infinite soluzioni.

In questi casi, invece di cercare di determinare il vettore $\mathbf{x}$ che verifichi l'identità $A\mathbf{x} = \mathbf{b}$, si cerca, se esiste, un vettore $\mathbf{x}$ tale che la distanza di $A\mathbf{x}$ da $\mathbf{b}$ sia la minima possibile. In altri termini il problema viene ricondotto ad un problema di minimo: determinare un $\bar{\mathbf{x}}$ di lunghezza n tale che, posto $r(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$ (detto residuo) risulti

$$\|r(\bar{\mathbf{x}})\| = \min_{\mathbf{x}} \|r(\mathbf{x})\| = \min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|$$

Se la norma utilizzata è quella euclidea, ovvero la norma 2,

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x}^t \mathbf{x}}$$

il problema è detto **problema dei minimi quadrati lineari (PMQ)**. Il nome è dovuto al fatto che si cerca il minimo della quantità

$$\min_{\mathbf{x}=(x_1, \dots, x_n)} \|A\mathbf{x} - \mathbf{b}\|_2 = \min_{\mathbf{x}=(x_1, \dots, x_n)} \sqrt{\sum_{i=1}^m \left[b_i - \sum_{j=1}^n a_{ij} x_j \right]^2}$$

cioè si cerca di minimizzare una somma di quadrati. Anche il problema dei minimi quadrati può avere infinite soluzioni. In questo caso si definisce **soluzione di minima norma** quella per cui accade che

$$\|\mathbf{x}_{min}\|_2 \leq \|\mathbf{x}\|_2$$

se $\mathbf{x}$ è una qualsiasi soluzione. D'ora in avanti ci occuperemo solo del caso dei sistemi sovradeterminati, cioè con $m \geq n$, che è anche il più significativo dal punto di vista delle applicazioni. Una delle più significative applicazioni del problema dei minimi quadrati sovradeterminati è quella relativa all'approssimazione di dati, detta anche **best fitting**.

Risolubilità del problema dei minimi quadrati Vediamo ora in quali ipotesi il problema dei minimi quadrati ammette soluzione.

Teorema 4.1. *Se A è una matrice $m \times n$ con $m > n$ allora esiste sempre una soluzione del PMQ. Inoltre tale soluzione è unica se e solo se il rango di A è massimo, cioè se $\text{rank}(A) = n$.*

Omettiamo la dimostrazione di tale teorema mettendo però in evidenza che tale dimostrazione è basata sulla seguente equivalenza:

$$x \text{ è soluzione del PMQ} \iff x \text{ è soluzione del sistema } {}^tAAx = {}^tAb.$$

Il sistema delle equazioni normali Il sistema

$${}^tAAx = {}^tAb$$

è detto **sistema delle equazioni normali**. Osserviamo innanzitutto che la matrice tAA è una matrice quadrata di ordine n . Inoltre è simmetrica in quanto

$${}^t({}^tAA) = {}^tA({}^tA) = {}^tAA$$

cioè coincide con la sua trasposta. Infine, se A ha rango massimo, è definita positiva poiché se y è un qualunque vettore di lunghezza n non nullo si ha

$${}^ty({}^tAA)y = ({}^ty^tA)(Ay) = {}^t(Ay)(Ay) = \|Ay\|_2^2 > 0$$

(se il rango non è massimo può accadere che $Ay = 0$, con $y \neq 0$).

Interpretazione geometrica Vogliamo ora dare un'interpretazione geometrica del problema dei minimi quadrati. Se A è di ordine $m \times n$ allora può essere pensata come un'applicazione lineare di $\mathbb{R}^n \rightarrow R(A) \subseteq \mathbb{R}^m$. Il sottospazio $R(A)$ è definito al seguente modo:

$$R(A) = \{u \in \mathbb{R}^m \mid \exists y \in \mathbb{R}^n : u = Ay\}.$$

Sia ora $b \in \mathbb{R}^m$. Poiché la norma 2 è quella euclidea, $\|b - Ax\|_2$ rappresenta la distanza tra i vettori b e Ax . È evidente allora che affinché tale distanza sia minima deve accadere che $b - Ax$ sia ortogonale a $R(A)$. Ora, ogni vettore di $R(A)$ può essere visto come combinazione lineare delle colonne di A . Infatti se $u \in R(A)$ è tale che $u = Ay$, ciò significa che

$$u = \begin{pmatrix} u_1 \\ \vdots \\ u_m \end{pmatrix} = \begin{pmatrix} a_{11}y_1 + \cdots + a_{1n}y_n \\ \vdots \\ a_{m1}y_1 + \cdots + a_{mn}y_n \end{pmatrix}$$

cioè

$$u = y_1 a_1 + y_2 a_2 + \cdots + y_n a_n$$

dove a_j denota la colonna j -esima di A . Dunque, affinché $b - Ax$ risulti ortogonale a $R(A)$, basta che sia ortogonale a tutte le colonne di A , cioè

$${}^ta_j(b - Ax) = 0, \quad j \in \{1, \dots, n\} \iff {}^tA(b - Ax) = 0 \iff {}^tAAx = {}^tAb$$

Si perviene così, anche per via geometrica, all'equivalenza tra la soluzione del problema dei minimi quadrati e quello del sistema delle equazioni normali.

La pseudoinversa

Definizione 4.1. Sia A una matrice $m \times n$, $m \geq n$ e con rango massimo ($\text{rank}(A) = n$). Si definisce **pseudoinversa** o **inversa generalizzata** di Moore-Penrose di A la matrice

$$A^+ = ({}^tAA)^{-1} {}^tA.$$

La pseudoinversa generalizza il concetto di inversa di una matrice nel caso delle matrici rettangolari. Infatti

$$A^+A = ({}^tAA)^{-1} {}^tAA = I.$$

D'altra parte se A è quadrata e non singolare si ha

$$A^+ = ({}^tAA)^{-1} {}^tA = A^{-1}({}^tA)^{-1} {}^tA = A^{-1},$$

cioè inversa e pseudoinversa coincidono. Il legame tra pseudoinversa e problema dei minimi quadrati è fornito dalla seguente

Proposizione 4.2. Sia A di ordine $m \times n$, $m \geq n$ e con rango massimo ($\text{rank}(A) = n$) e $\mathbf{b}$ un vettore di lunghezza m . L'unica soluzione del problema dei minimi quadrati relativo a tali dati è data da

$$\mathbf{x} = A^+\mathbf{b} \equiv ({}^tAA)^{-1} {}^tA\mathbf{b}.$$

La proposizione è immediata conseguenza del teorema di esistenza e unicità. Infatti abbiamo già detto che la soluzione del PMQ è quella del sistema delle equazioni normali

$${}^tAA\mathbf{x} = {}^tA\mathbf{b} \implies \mathbf{x} = ({}^tAA)^{-1} {}^tA\mathbf{b}.$$

La matrice pseudoinversa ha diverse e utili applicazioni, non solo nella soluzione del problema dei minimi quadrati. Un'importante informazione che si può dedurre dalla conoscenza di A^+ è il condizionamento del problema dei minimi quadrati. È infatti possibile dimostrare che, se si introducono delle perturbazioni su $\mathbf{b}$ ed A , allora la corrispondente perturbazione sul vettore $\mathbf{x}$, soluzione del PMQ, è proporzionale a quelle introdotte sui dati mediante la quantità

$$pcond(A) = \|A\| \|A^+\|.$$

Risoluzione del sistema delle equazioni normali Abbiamo visto come la soluzione del problema dei minimi quadrati coincida, nel caso sovradeterminato e di rango massimo, con la soluzione del sistema delle equazioni normali:

$${}^tAA\mathbf{x} = {}^tA\mathbf{b}.$$

Dunque per calcolare numericamente la soluzione del PMQ è possibile risolvere numericamente il sistema delle equazioni normali. Poiché la matrice dei coefficienti è simmetrica e definita positiva, possiamo usare il metodo di eliminazione di Gauss senza pivoting o, con un costo computazionale dimezzato, il metodo di Cholesky. Dunque il problema dei minimi quadrati può essere risolto nel seguente modo:

- si forma la matrice tAA ;
- si fattorizza tAA mediante fattorizzazione di Cholesky, ovvero si computa L tale che ${}^tAA = L^tL$;

- si forma il vettore $\mathbf{c} = {}^t A \mathbf{b}$;
- si risolve la coppia di sistemi lineari triangolari:

$$\begin{cases} L\mathbf{y} = \mathbf{c} \\ {}^t L\mathbf{x} = \mathbf{y} \end{cases}.$$

Costo computazionale e stabilità Il costo computazionale complessivo della procedura è di circa

$$m \frac{n^2}{2} + \frac{n^3}{6}$$

operazioni, in quanto ne occorrono circa $m \frac{n^2}{2}$ per formare ${}^t AA$ (che si costruisce per simmetria) e $\frac{n^3}{6}$ per la fattorizzazione di Cholesky. Le sostituzioni all'indietro e in avanti costano complessivamente n^2 e sono trascurate. Dunque la procedura è relativamente economica, in quanto l'operazione più costosa (la fattorizzazione di Cholesky) viene effettuata sulla matrice di ordine n (che in genere è molto minore di m). Sfortunatamente la procedura appena descritta è complessivamente instabile. Infatti il primo problema è la costruzione della matrice ${}^t AA$. Può accadere che vi sia una perdita di cifre significative nella rappresentazione di macchina di ${}^t AA$. Nei casi più patologici può verificarsi che la matrice "di macchina" non sia più definita positiva o addirittura che risulti singolare. È stato infatti provato che se $pcond(A) > 10^{\frac{t}{2}}$, dove t è la precisione della rappresentazione floating point, non è più assicurato che $fl({}^t AA)$ sia non singolare.

Anche se non si incorre in questi casi "patologici" tuttavia la procedura è generalmente poco stabile in quanto, mentre il PMQ ha un condizionamento che è regolato da $pcond(A)$, il sistema delle equazioni normali ha un condizionamento che dipende da $cond({}^t AA)$. Si può provare che (in norma 2):

$$cond({}^t AA) = pcond(A)^2.$$

Concludendo possiamo dire che alla risoluzione mediante equazioni normali, benché economica, si deve ricorrere solo quando si hanno informazioni sul buon condizionamento della matrice ${}^t AA$. Nei casi in cui si è incerti sul condizionamento del problema conviene utilizzare altri metodi, più costosi, ma stabili.

Metodo mediante fattorizzazione QR

La fattorizzazione QR È noto che le fattorizzazioni di matrici costituiscono un efficace strumento in algebra lineare. Le più note sono la fattorizzazione LU e la fattorizzazione $L^t L$ di Cholesky (per matrici simmetriche e definite positive). Entrambe queste fattorizzazioni sono per matrici quadrate. Il costo computazionale per la costruzione di L e U è di circa $\frac{m^3}{3}$ operazioni, mentre la fattorizzazione di Cholesky costa $\frac{m^3}{6}$ operazioni. Illustriamo ora un'altra fattorizzazione che prende il nome di fattorizzazione QR e che può essere costruita anche per matrici rettangolari. Data una matrice A di ordine $m \times n$ esistono due matrici, Q quadrata di ordine m e ortogonale (cioè tale che $Q^{-1} = {}^t Q$) e R di ordine $m \times n$, del tipo:

$$R = \begin{pmatrix} R_1 \\ 0 \end{pmatrix}$$

con R_1 triangolare superiore di ordine n , se $m > n$, oppure del tipo $(R_1 \ 0)$ con R_1 triangolare superiore di ordine m , se $m < n$. Nel caso A sia quadrata i fattori Q ed R sono

entrambi quadrati e R è triangolare superiore. Esistono vari algoritmi per la costruzione della fattorizzazione QR, il più utilizzato dei quali (quello che adopera le matrici elementari di Householder) ha un costo computazionale di $\frac{2}{3}m^3$, nel caso della matrice quadrata e piena. Nel caso rettangolare con $m > n$ invece il costo è dell'ordine di:

$$n^2 \left(m - \frac{n}{3} \right).$$

Confrontando dunque su matrici quadrate, la fattorizzazione QR "costa di più" della fattorizzazione LU. Il valore aggiunto però sta nel fatto che il fattore Q è ortogonale. Le matrici ortogonali rivestono un'importanza rilevante per le loro numerose proprietà:

- Non è necessario costruire le inverse, che sono disponibili con la sola trasposizione ($Q^{-1} \equiv {}^tQ$), il che implica ${}^tQQ = Q{}^tQ = I$.
- Se $\mathbf{y}$ è un qualsiasi vettore di lunghezza m si ha:

$$\|Q\mathbf{y}\|_2 = \sqrt{{}^t(Q\mathbf{y})(Q\mathbf{y})} = \sqrt{{}^t\mathbf{y}{}^tQQ\mathbf{y}} = \sqrt{{}^t\mathbf{y}\mathbf{y}} = \|\mathbf{y}\|_2$$

cioè la moltiplicazione per matrici ortogonali lascia invariata la norma del vettore (lo stesso vale per tQ).

- Da quanto sopra si ha $\|Q\|_2 = 1$, $\|{}^tQ\|_2 = 1$, da cui $\text{cond}(Q) = 1$, cioè le matrici ortogonali sono perfettamente condizionate.

Il metodo per il PMQ basato sulla fattorizzazione QR Ricordiamo che il problema dei minimi quadrati lineari consiste nel cercare il minimo, su tutti i vettori di lunghezza n , della quantità $\|A\mathbf{x} - \mathbf{b}\|_2$ dove A è una matrice di ordine $m \times n$, con $m \geq n$, $\text{rank}(A) = n$ e $\mathbf{b}$ è di dimensione m . Un metodo numerico alternativo è basato sull'utilizzo della fattorizzazione QR di A . Siano infatti Q e R i fattori di A , si ha:

$${}^tQA = \begin{pmatrix} R_1 \\ 0 \end{pmatrix}$$

dove R_1 è quadrata, di ordine n e triangolare superiore. Poiché tQ è ortogonale, e quindi preserva la norma 2 del vettore, si ha:

$$\|A\mathbf{x} - \mathbf{b}\|_2^2 = \|{}^tQ(A\mathbf{x} - \mathbf{b})\|_2^2 = \|{}^tQA\mathbf{x} - {}^tQ\mathbf{b}\|_2^2$$

Ponendo ora ${}^tQ\mathbf{b} = \begin{pmatrix} \mathbf{c} \\ \mathbf{d} \end{pmatrix}$ con $\mathbf{c}$ vettore di lunghezza n , si ha:

$$\|A\mathbf{x} - \mathbf{b}\|_2^2 = \left\| \begin{pmatrix} R_1\mathbf{x} \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{c} \\ \mathbf{d} \end{pmatrix} \right\|_2^2 = \left\| \begin{pmatrix} R_1\mathbf{x} - \mathbf{c} \\ -\mathbf{d} \end{pmatrix} \right\|_2^2$$

Osserviamo ora che se un vettore $\mathbf{y}$ di lunghezza m è decomposto in due sottovettori, per esempio $\mathbf{y} = {}^t(\mathbf{z}, \mathbf{t})$ con $\mathbf{z}$ di lunghezza n e $\mathbf{t}$ di lunghezza $m - n$, si ha che $\|\mathbf{y}\|_2^2 = \|\mathbf{z}\|_2^2 + \|\mathbf{t}\|_2^2$. Dunque:

$$\|A\mathbf{x} - \mathbf{b}\|_2^2 = \left\| \begin{pmatrix} R_1\mathbf{x} - \mathbf{c} \\ -\mathbf{d} \end{pmatrix} \right\|_2^2 = \|R_1\mathbf{x} - \mathbf{c}\|_2^2 + \|\mathbf{d}\|_2^2$$

Poiché il secondo termine della somma non dipende da $\mathbf{x}$ ed è sempre positivo, il minimo del primo membro sarà assunto se $\|R_1\mathbf{x} - \mathbf{c}\|_2 = 0$, se cioè $R_1\mathbf{x} = \mathbf{c}$, ovvero se $\mathbf{x}$ è soluzione del sistema triangolare con R_1 come matrice dei coefficienti e $\mathbf{c}$ come termine noto.

Procedura e Costo Dunque la procedura proposta può essere riassunta come segue:

- si effettua la fattorizzazione QR della matrice A e quindi, in particolare, si determina la matrice R_1 ;
- si forma il vettore $\mathbf{c}$ costituito dai primi n elementi del vettore ${}^tQ\mathbf{b}$;
- si risolve il sistema triangolare $R_1\mathbf{x} = \mathbf{c}$ (mediante sostituzione all'indietro).

La quantità $\|\mathbf{d}\|_2$ rappresenta la norma 2 del residuo. L'algoritmo ha un costo computazionale che è essenzialmente quello della fattorizzazione QR e pertanto è $n^2(m - \frac{n}{3})$. La procedura descritta risulta inoltre stabile.

Conclusioni Abbiamo proposto due possibili strategie numeriche per la risoluzione del problema dei minimi quadrati lineari, nel caso sia A di dimensioni $m \times n$, con $m \geq n$ e $\text{rank}(A) = n$:

1. la risoluzione del sistema delle equazioni normali ${}^tAA\mathbf{x} = {}^tA\mathbf{b}$;
2. il metodo che utilizza la fattorizzazione QR della matrice A .

Confronto Schemi

Metodo 1 (Equazioni Normali):

- formare la matrice tAA ;
- calcolare la fattorizzazione di Cholesky ${}^tAA = L^tL$;
- risolvere la coppia di sistemi triangolari $\begin{cases} L\mathbf{y} = {}^tA\mathbf{b} \\ {}^tL\mathbf{x} = \mathbf{y} \end{cases}$;
- costo computazionale: $m\frac{n^2}{2} + \frac{n^3}{6}$;
- stabilità: assicurata solo se $\text{cond}({}^tAA) = p\text{cond}(A)^2$ non è grande.

Metodo 2 (Fattorizzazione QR):

- calcolare la fattorizzazione QR di A ;
- calcolare ${}^tQ\mathbf{b} = \begin{pmatrix} \mathbf{c} \\ \mathbf{d} \end{pmatrix}$, dove $\mathbf{c}$ ha lunghezza n ;
- posto $R = \begin{pmatrix} R_1 \\ 0 \end{pmatrix}$, risolvere il sistema $R_1\mathbf{x} = \mathbf{c}$;
- la norma del residuo è data da $\|\mathbf{d}\|_2$;
- costo computazionale: $n^2(m - \frac{n}{3})$;
- stabilità: assicurata sempre.

Functions del Matlab Le funzioni Matlab coinvolte nella risoluzione del PMQ sono le seguenti:

- $A \backslash b$: implementa, nel caso A sia rettangolare, il metodo che utilizza la fattorizzazione QR, cioè il metodo 2;
- $R = \text{chol}(B)$: calcola il fattore di Cholesky della matrice simmetrica e definita positiva B ($B = {}^t R R$);
- $D = \text{pinv}(A)$: calcola la pseudoinversa (sinistra) della matrice A ;
- $[Q, R] = \text{qr}(A)$: calcola la fattorizzazione QR della matrice A (quadrata o rettangolare) mediante trasformazioni di Householder.

4.1 Esercizi

1. Considerare la matrice test A definita mediante i comandi Matlab

$$B = \text{gallery}('kahan', 80);$$

$$A = B(:, 1 : 30).$$

- Determinare le dimensioni della matrice e stabilire se è o meno a rango massimo.
- Considerare il sistema $Ax = b$ con $b(i) = \sum_{j=1}^n a_{ij}$, $i = 1, \dots, m$ nel senso dei minimi quadrati. Studiare l'esistenza e l'unicità della soluzione del sistema, il condizionamento del problema e determinare la stima dell'errore teorico a priori.
- Scegliere il metodo che conviene applicare e giustificare la scelta.
- È noto che in questo caso la soluzione è data da $\bar{x} = {}^t(1, 1, \dots, 1) \in \mathbb{R}^n$. Calcolare quindi l'errore commesso e confrontare il risultato ottenuto con la stima a priori.

2. Considerare la matrice test A definita mediante il comando Matlab

$$A = \text{gallery}('chebvand', 350, 60).$$

- Determinare le dimensioni della matrice e stabilire se è o meno a rango massimo.
- Considerare il sistema $Ax = b$ con $b(i) = \sum_{j=1}^n a_{ij}$, $i = 1, \dots, m$ nel senso dei minimi quadrati. Studiare l'esistenza e l'unicità della soluzione del sistema, il condizionamento del problema e determinare la stima dell'errore teorico a priori.
- Scegliere il metodo che conviene applicare e giustificare la scelta.
- È noto che in questo caso la soluzione è data da $\bar{x} = {}^t(1, 1, \dots, 1) \in \mathbb{R}^n$. Calcolare quindi l'errore commesso e confrontare il risultato ottenuto con la stima a priori.

3. Considerare la matrice test A definita mediante i comandi Matlab

$$B = \text{gallery}('kms', 100, .1);$$

$$A = B(:, 1 : 30).$$

- Determinare le dimensioni della matrice e stabilire se è o meno a rango massimo.

- Considerare il sistema $Ax = b$ con $b(i) = \sum_{j=1}^n a_{ij}$, $i = 1, \dots, m$ nel senso dei minimi quadrati. Studiare l'esistenza e l'unicità della soluzione del sistema, il condizionamento del problema e determinare la stima dell'errore teorico a priori.
- Scegliere il metodo che conviene applicare e giustificare la scelta.
- È noto che in questo caso la soluzione è data da $\bar{x} = {}^t(1, 1, \dots, 1) \in \mathbb{R}^n$. Calcolare quindi l'errore commesso e confrontare il risultato ottenuto con la stima a priori.

4. Considerare la matrice test A definita mediante i comandi Matlab

$$A = \text{gallery}('lauchli', 100) * \text{gallery}('lehmer', 100);$$

- Determinare le dimensioni della matrice e stabilire se è o meno a rango massimo.
- Considerare il sistema $Ax = b$ con $b(i) = \sum_{j=1}^n a_{ij}$, $i = 1, \dots, m$ nel senso dei minimi quadrati. Studiare l'esistenza e l'unicità della soluzione del sistema, il condizionamento del problema e determinare la stima dell'errore teorico a priori.
- Scegliere il metodo che conviene applicare e giustificare la scelta.
- Calcolare l'errore commesso confrontando la soluzione ottenuta con quella che si ottiene utilizzando la function \ del MatLab. Confrontare tale errore con la stima a priori.

5. Data la matrice di ordine $n = 100$

$$B = \begin{pmatrix} 1 & -2 & -2 & \dots & \dots & -1 \\ 4 & 2 & -2 & \dots & \dots & -2 \\ 4 & 4 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -2 & -2 \\ \vdots & & \ddots & \ddots & n-1 & -2 \\ 5 & 4 & \dots & 4 & 4 & n \end{pmatrix},$$

considerare la matrice $A = B(:, 1 : 7)$.

- Determinare le dimensioni della matrice e stabilire se è o meno a rango massimo.
- Considerare il sistema $Ax = b$ con $b = 2 * \text{sum}(A, 2)$, nel senso dei minimi quadrati. Studiare l'esistenza e l'unicità della soluzione del sistema, il condizionamento del problema e determinare la stima dell'errore teorico a priori.
- Scegliere il metodo che conviene applicare e giustificare la scelta.
- Commentare i risultati ottenuti.

6. Considerare la seguente matrice test

$$A = \text{gallery}('lauchli', 10).$$

- Determinare le dimensioni della matrice e stabilire se è o meno a rango massimo.
- Considerare il sistema $Ax = b$ con $b(i) = \sum_{j=1}^n a_{ij}$, $i = 1, \dots, m$ nel senso dei minimi quadrati. Studiare l'esistenza e l'unicità della soluzione del sistema, il condizionamento del problema e determinare la stima dell'errore teorico a priori.

- Scegliere il metodo che conviene applicare e giustificare la scelta. Qual è il suo costo computazionale?
- È noto che in questo caso la soluzione è data da $\bar{x} = {}^t(1, 1, \dots, 1) \in \mathbb{R}^n$. Calcolare quindi l'errore commesso e confrontare il risultato ottenuto con la stima a priori.

7. Sia $m = 90$. Consideriamo il sistema $Ax = b$ nel senso dei minimi quadrati, dove

$$A = (a_{i,j})_{\substack{i=1,\dots,m \\ j=1,\dots,\frac{m}{3}}}, \quad a_{i,j} = \begin{cases} \sin(i + 2j) + 1, & i = j \\ \cos(2i + j - 5), & i \neq j \end{cases}$$

$$\mathbf{b} = \{b_i\}_{i=1,\dots,m}, \quad b_i = \sum_{j=1}^{\frac{m}{3}} a_{i,j}, \quad i = 1, \dots, m.$$

- Riportare le istruzioni Matlab utilizzate per la costruzione di A e b .
- Determinare le dimensioni della matrice e stabilire se è o meno a rango massimo.
- Studiare l'esistenza e l'unicità della soluzione del sistema, il condizionamento del problema e determinare la stima dell'errore teorico a priori.
- Scegliere il metodo che conviene applicare e giustificare la scelta. Qual è il suo costo computazionale?
- È noto che in questo caso la soluzione è data da $\bar{x} = {}^t(1, 1, \dots, 1) \in \mathbb{R}^{\frac{m}{3}}$. Calcolare quindi l'errore commesso e confrontare il risultato ottenuto con la stima a priori.

5 Metodi Numerici per la Risoluzione di una Equazione non Lineare

5.1 Risoluzione di Equazioni non Lineari

Introduzione Sia $F \in C^0([a, b])$, cioè F è una funzione continua in un intervallo $[a, b] \subset \mathbb{R}$, tale che $F(a)F(b) < 0$. Vogliamo trovare le radici dell'equazione non lineare

$$F(x) = 0$$

La condizione $F(a)F(b) < 0$ è una condizione sufficiente per l'esistenza di almeno una radice di F in $[a, b]$.

Metodo di Bisezione Questo metodo consiste nel costruire, a partire dall'intervallo $[a, b]$, una successione di intervalli incapsulati

$$[a, b] = [a_0, b_0] \supset [a_1, b_1] \supset \cdots \supset [a_n, b_n]$$

tutti contenenti la radice $\bar{x}$ di F , tale che

$$b_n - a_n \rightarrow 0 \quad \text{per } n \rightarrow +\infty.$$

Gli intervalli $[a_k, b_k]$, $k \in \{1, \dots, n\}$, della successione vengono determinati come segue: dato $[a_{k-1}, b_{k-1}]$, determiniamo il punto medio

$$m_k = \frac{a_{k-1} + b_{k-1}}{2}$$

- se $F(m_k) = 0$ allora $\bar{x} = m_k$ e abbiamo terminato;
- se $F(m_k) \neq 0$ poniamo

$$[a_k, b_k] = \begin{cases} [a_{k-1}, m_k] & \text{se } F(a_{k-1})F(m_k) < 0 \\ [m_k, b_{k-1}] & \text{se } F(b_{k-1})F(m_k) < 0 \end{cases}$$

Dopo n passi si giunge all'intervallo $[a_n, b_n]$ contenente la radice $\bar{x}$ e di ampiezza

$$b_n - a_n = \frac{b_{n-1} - a_{n-1}}{2} = \frac{b_{n-2} - a_{n-2}}{2^2} = \cdots = \frac{b - a}{2^n}$$

Come stima di $\bar{x}$ scegliamo

$$x_{n+1} = m_{n+1} = \frac{a_n + b_n}{2}$$

così che

$$\bar{x} = x_{n+1} \pm \varepsilon_{n+1}$$

dove

$$\varepsilon_{n+1} = |\bar{x} - x_{n+1}| < \frac{b-a}{2^{n+1}}$$

è l'errore assoluto di approssimazione della radice $\bar{x}$ di F . Fissata una tolleranza TOLL, è possibile determinare il numero di iterazioni k necessarie per approssimare la radice $\bar{x}$ con precisione TOLL imponendo che

$$|\bar{x} - x_n| < \frac{b-a}{2^n} < \text{TOLL}$$

Infatti, possiamo scrivere

$$\begin{aligned} \frac{b-a}{\text{TOLL}} &< 2^n \\ \log\left(\frac{b-a}{\text{TOLL}}\right) &< \log(2^n) = n \log 2 \\ n &> \frac{\log\left(\frac{b-a}{\text{TOLL}}\right)}{\log 2} \end{aligned}$$

Dunque prendiamo

$$n = \left\lfloor \frac{\log\left(\frac{b-a}{\text{TOLL}}\right)}{\log 2} \right\rfloor + 1$$

dove $\lfloor a \rfloor$ denota la parte intera inferiore di a .

```

1 function [x, i, itmax] = bisec(a, b, f, toll)
2 % BISEC Trova la radice di f(x)=0 in [a,b] con il metodo di Bisezione.
3 %
4 %   Input:
5 %     a, b   - Estremi dell'intervallo
6 %     f       - Function handle della funzione
7 %     toll    - Tolleranza richiesta (default 1e-6)
8 %
9 %   Output:
10 %     x       - Vettore con la storia delle approssimazioni
11 %     i       - Numero di iterazioni effettuate
12 %     itmax    - Numero massimo di iterazioni teoriche previste
13
14 %% 1. Validazione degli Input
15 arguments
16     a      (1,1) double
17     b      (1,1) double
18     f      (1,1) function_handle
19     toll   (1,1) double {mustBePositive} = 1e-6
20 end
21
22 %% 2. Controllo Esistenza Zeri (Teorema degli Zeri)
23 fa = f(a);
24 fb = f(b);
25
26 % Uso sign() per evitare overflow se f(a)*f(b) diventa troppo grande
27 if sign(fa) * sign(fb) > 0
28     error('La funzione non cambia segno agli estremi dell''intervallo [a,b]');
29 end
30
31 %% 3. Inizializzazione
32 % Calcolo teorico del numero di passi necessari

```

```

33     itmax = 1 + floor(log((b-a)/toll)/log(2));
34
35     i = 1;                % Indice corrente
36     x(i) = (a + b) / 2; % Primo punto medio
37     fx = f(x(i));        % Valutazione nel punto medio
38
39     %% 4. Ciclo Iterativo
40     % Continua finché non finisco i passi E l'errore è alto
41     while (i < itmax) && (abs(fx) >= toll)
42
43         % Selezione del sotto-intervallo
44         if sign(fx) * sign(fa) < 0
45             b = x(i);    % La radice è a sinistra (tra a e x)
46         else
47             a = x(i);    % La radice è a destra (tra x e b)
48             fa = fx;     % Fondamentale: aggiorno fa per il prossimo confronto
49         end
50
51         % Nuovo passo
52         i = i + 1;
53         x(i) = (a + b) / 2; % Nuovo punto medio
54         fx = f(x(i));      % Nuova valutazione
55     end
56
57 end

```

Function bisec

Ordine di convergenza Introduciamo ora una misura della velocità di convergenza di una successione numerica.

Definizione 5.1. Sia $\{x_n\}_n$ una successione convergente al valore $\bar{x}$. Poniamo $\varepsilon_n = |\bar{x} - x_n|$. Se esiste un numero reale $p \geq 1$ e una costante reale positiva C tale che

$$\lim_{n \rightarrow +\infty} \frac{\varepsilon_{n+1}}{\varepsilon_n^p} = C$$

allora diciamo che la successione $\{x_n\}_n$ ha ordine di convergenza p .

Poiché per il metodo di bisezione si ha

$$\frac{\varepsilon_{n+1}}{\varepsilon_n} \simeq \frac{1}{2}$$

ovvero l'errore si dimezza (più o meno) ad ogni passo, il suo ordine di convergenza è 1, cioè converge molto lentamente. Va però osservato che il metodo richiede solo la continuità della funzione F e la conoscenza del segno di F negli estremi dell'intervallo $[a, b]$.

Metodi Iterativi Partendo dal punto iniziale x_0 , generiamo i valori $x_1, x_2, \dots, x_n$ nel seguente modo:

- Conduciamo dal punto $(x_0, F(x_0))$ una retta con pendenza m_0 e scegliamo come approssimazione x_1 l'intersezione di tale retta con l'asse delle x ;
- Conduciamo dal punto $(x_1, F(x_1))$ una retta con pendenza m_1 e scegliamo come approssimazione x_2 l'intersezione di tale retta con l'asse delle x ;

Dunque ad ogni passo scegliamo come approssimazione della radice $\bar{x}$ di F , la radice dell'equazione lineare

$$F(x_n) + m_n(x - x_n) = 0$$

cioè

$$x_{n+1} = x_n - \frac{F(x_n)}{m_n} \quad (1)$$

Una formula del tipo (1) viene detta **formula iterativa** e si dice che la successione $x_1, x_2, \dots, x_n, \dots$ viene costruita mediante un procedimento iterativo. La funzione

$$g(x) = x - \frac{F(x)}{m_n}$$

tale che $x_{n+1} = g(x_n)$ viene detta **funzione di iterazione semplice** perché per la costruzione della successione $\{x_n\}_n$ utilizza solo il punto x_n . Si parla di procedimenti iterativi multipli quando

$$x_{n+k} = g(x_{n+k-1}, x_{n+k-2}, \dots, x_n), \quad k > 1$$

cioè la successione $\{x_n\}_n$ è costruita a partire dai punti $x_0, x_1, \dots, x_{k-1}$.

Criteri di Arresto Quando si implementa in maniera automatica un procedimento iterativo del tipo $x_{n+1} = g(x_n)$, $n \in \mathbb{N}$, occorrono uno o più criteri per arrestare tale procedimento. Fissata una tolleranza TOLL arbitrariamente piccola, i criteri di arresto usualmente utilizzati sono:

1. $|F(x_n)| < \text{TOLL}$ (tanto più è piccolo TOLL tanto più x_n è vicino ad $\bar{x}$);
2. $|x_{n+1} - x_n| < \text{TOLL}$ oppure $\frac{|x_{n+1} - x_n|}{|x_{n+1}|} < \text{TOLL}$ (tanto più è piccolo TOLL tanto più x_n è vicino al limite della successione $\bar{x}$);
3. numero delle iterazioni $< \text{ITMAX}$, dove ITMAX è una variabile intera fissata.

Il criterio 3 entra in gioco quando i primi due falliscono. Per questo motivo la variabile ITMAX viene anche detta variabile tappo. In generale è preferibile utilizzare tutti e tre i criteri, perché per particolari funzioni si può verificare che la convergenza sia troppo lenta oppure che i criteri 1 e/o 2 non vengano mai soddisfatti numericamente.

Metodo di Newton o delle tangenti Nella formula

$$x_{n+1} = x_n - \frac{F(x_n)}{m_n}$$

le direzioni m_n possono essere scelte in vari modi. In questo metodo la direzione scelta ad ogni passo è quella della tangente alla curva $y = F(x)$ nel punto x_n , cioè

$$m_n = F'(x_n).$$

Dunque il metodo è dato da

$$x_{n+1} = x_n - \frac{F(x_n)}{F'(x_n)}, \quad n \in \mathbb{N}.$$

La funzione di iterazione è quindi

$$g(x) = x - \frac{F(x)}{F'(x)}.$$

Appare evidente che il metodo perde di significato se per qualche n risulta $F'(x_n) = 0$. Dunque in generale si suppone che $F'(x) \neq 0, \forall x \in [a, b]$. La formula iterativa ci permette di calcolare la successione $x_1, x_2, \dots$ a partire dal punto x_0 . Ma come scegliamo il punto x_0 ?

```

1 function [x, i] = newton(x0, f, df, toll, itmax)
2 % NEWTON Trova la radice di f(x)=0 usando il metodo di Newton.
3 % [x, i] = NEWTON(x0, f, df, toll, itmax) calcola l'approssimazione
4 % della radice a partire da una stima iniziale x0.
5 %
6 % Input:
7 % x0      - (double) Stima iniziale
8 % f       - (function_handle) La funzione f(x)
9 % df      - (function_handle) La derivata prima f'(x)
10 % toll    - (double, opzionale) Tolleranza per l'arresto (default 1e-6)
11 % itmax   - (integer, opzionale) Numero massimo di iterazioni (default 100)
12 %
13 % Output:
14 % x - (vector) Vettore contenente tutte le iterazioni calcolate
15 % i - (integer) Numero di iterazioni effettuate
16
17 %% 1. Validazione degli Input
18 arguments
19     x0      (1,1) double
20     f       (1,1) function_handle
21     df      (1,1) function_handle
22     toll    (1,1) double {mustBePositive} = 1e-6
23     itmax   (1,1) double {mustBePositive, mustBeInteger} = 100
24 end
25
26 %% 2. Inizializzazione
27 i = 1;           % Indice di posizione (parte da 1)
28 x(i) = x0;       % Inserimento stima iniziale nel vettore
29
30 fx = f(x(i));    % Valutazione funzione
31 dfx = df(x(i));  % Valutazione derivata
32
33 err = 1;         % Inizializzo l'errore per entrare nel loop
34
35 %% 3. Ciclo Iterativo
36 % Continua se: non ho superato itmax E l'errore è alto E il residuo è alto
37 while (i < itmax) && (err > toll) && (abs(fx) > toll)
38
39     % Controllo di sicurezza: Derivata nulla o quasi nulla
40     if abs(dfx) < eps
41         warning('Derivata quasi nulla in x = %f. Metodo arrestato.', x(i));
42         break;
43     end
44
45     % Passo di Newton
46     % Formula: x(k+1) = x(k) - f(x)/f'(x)
47     x(i + 1) = x(i) - fx / dfx; % Calcolo e aggiungo il nuovo valore al
vettore
48
49     % Calcolo Errore Relativo
50     % Nota: Aggiungo eps al denominatore per evitare crash se x(i + 1) è 0
51     err = abs(x(i + 1) - x(i)) / (abs(x(i + 1)) + eps);
52
53     % Aggiornamento indici e funzioni
54     i = i + 1;
55     fx = f(x(i));

```

```

56         dfx = df(x(i));
57     end
58
59     %% 4. Controllo finale
60     % Se siamo usciti perché i ha raggiunto itmax (e l'errore è ancora alto)
61     if (i == itmax) && (err > toll)
62         warning('Raggiunto il numero massimo di iterazioni (%d) senza
63         convergenza.', itmax);
64     end
end

```

Function newton

Teorema di Convergenza Il seguente teorema stabilisce delle condizioni per la determinazione di un intervallo $[a, b]$ per il quale il metodo di Newton converge per ogni scelta di $x_0 \in [a, b]$.

Teorema 5.1. Sia $F(x) \in C^2([a, b])$ con $[a, b]$ chiuso e limitato. Se:

- $F(a)F(b) < 0$;
- $F'(x) \neq 0, \forall x \in [a, b]$;
- $F''(x) \geq 0 \vee F''(x) \leq 0, \forall x \in [a, b]$;
- $|\frac{F(a)}{F'(a)}| < (b - a) \wedge |\frac{F(b)}{F'(b)}| < (b - a)$;

allora il Metodo di Newton converge all'unica soluzione $\bar{x} \in [a, b]$ per ogni scelta di $x_0 \in [a, b]$.

Ordine di convergenza del Metodo di Newton Per determinare l'ordine di convergenza utilizziamo il seguente teorema:

Teorema 5.2. Condizione necessaria e sufficiente affinché un procedimento iterativo semplice e convergente ad $\bar{x}$ abbia ordine di convergenza p è che, se la funzione di iterazione g è dotata di derivata p -esima continua per $x = \bar{x}$ risulti

$$g'(\bar{x}) = g''(\bar{x}) = \dots = g^{(p-1)}(\bar{x}) = 0 \quad e \quad g^{(p)}(\bar{x}) \neq 0$$

Per il Metodo di Newton si ha:

$$g'(x) = 1 - \frac{(F'(x))^2 - F(x)F''(x)}{(F'(x))^2} = \frac{F(x)F''(x)}{(F'(x))^2}$$

ma, poiché assumiamo che $F(\bar{x}) = 0$ si ha

$$g'(\bar{x}) = 0$$

e quindi

$$g(x) = \frac{(F'(x)F''(x) + F(x)F'''(x))(F'(x))^2 - 2F'(x)F(x)(F''(x))^2}{(F'(x))^4}$$

$$g''(\bar{x}) = \frac{F''(\bar{x})}{F'(\bar{x})}$$

che, in generale, è diverso da 0. Dunque, in generale, il metodo ha ordine di convergenza 2.

Domanda: Cosa succede se $\bar{x}$ è uno zero doppio, cioè $F(\bar{x}) = F'(\bar{x}) = 0$ e $F''(\bar{x}) \neq 0$? In questo caso non è possibile calcolare né g né g' in $\bar{x}$, ma è possibile definirli per continuità:

$$g(\bar{x}) = \bar{x} - \lim_{x \rightarrow \bar{x}} \frac{F(x)}{F'(x)} = \bar{x} - \lim_{x \rightarrow \bar{x}} \frac{F'(x)}{F''(x)} = \bar{x}$$

$$g'(\bar{x}) = F''(\bar{x}) \lim_{x \rightarrow \bar{x}} \frac{F(x)}{(F'(x))^2} = F''(\bar{x}) \lim_{x \rightarrow \bar{x}} \frac{F'(x)}{2F'(x)F''(x)} = \frac{1}{2}$$

Dunque se $\bar{x}$ è uno zero doppio, essendo $g'(\bar{x}) \neq 0$, il metodo di Newton ha ordine di convergenza 1. Più in generale, se $\bar{x}$ è uno zero d'ordine r , cioè

$$F(\bar{x}) = F'(\bar{x}) = \dots = F^{(r-1)}(\bar{x}) = 0 \quad \text{e} \quad F^{(r)}(\bar{x}) \neq 0$$

allora

$$g'(\bar{x}) = 1 - \frac{1}{r} \neq 0$$

dunque il metodo di Newton ha ancora ordine di convergenza 1.

Metodo di Newton per radici multiple Quando $\bar{x}$ è uno zero multiplo è possibile modificare leggermente il metodo di Newton per "recuperare" l'ordine 2. Si considera la seguente formula iterativa:

$$x_{n+1} = x_n - 2 \frac{F(x_n)}{F'(x_n)} \quad n \in \mathbb{N}$$

la cui funzione di iterazione è

$$g(x) = x - 2 \frac{F(x)}{F'(x)}$$

La precedente formula iterativa ha ordine di convergenza 2, infatti:

$$g'(x) = 1 - 2 \frac{(F'(x))^2 - F(x)F''(x)}{(F'(x))^2} = 2 \frac{F(x)F''(x)}{(F'(x))^2} - 1$$

otteniamo

$$g'(\bar{x}) = 2F''(\bar{x}) \lim_{x \rightarrow \bar{x}} \frac{F(x)}{(F'(x))^2} - 1 = 1 - 1 = 0$$

In generale se $\bar{x}$ è una radice multipla di ordine r si utilizza la seguente formula iterativa

$$x_{n+1} = x_n - r \frac{F(x_n)}{F'(x_n)}, \quad n \in \mathbb{N}$$

il cui ordine di convergenza è ancora 2. Esistono opportune modifiche anche nei casi in cui la molteplicità della radice è > 1 ma non è nota e nei casi in cui la molteplicità è infinita. Esistono delle varianti del Metodo di Newton che hanno ordine di convergenza > 2 .

```

1 function [x_bis, i_bis, x_newt, i_newt] = bisnew(a, b, f, df, toll, itmax)
2 % BISNEW Metodo ibrido: Sgrossatura con Bisezione + Raffinamento con Newton.
3 %
4 % Input:
5 % a, b - Estremi intervallo iniziale
6 % f, df - Funzione e sua Derivata (handle)

```

```

7 %   toll - Tolleranza richiesta per la fase finale (Newton)
8 %   itmax - Max iterazioni per la fase finale (Newton)
9 %
10 % Output:
11 %   x_bis - (vector) Storia delle approssimazioni della fase di Bisezione
12 %   i_bis - (integer) Numero di iterazioni della fase di Bisezione
13 %   x_newt - (vector) Storia delle approssimazioni della fase di Newton
14 %   i_newt - (integer) Numero di iterazioni della fase di Newton
15
16 %% 1. Validazione Input
17 arguments
18     a      (1,1) double
19     b      (1,1) double
20     f      (1,1) function_handle
21     df     (1,1) function_handle
22     toll   (1,1) double {mustBePositive} = 1e-6
23     itmax  (1,1) double {mustBePositive, mustBeInteger} = 100
24 end
25
26 %% 2. Fase 1: Sgrossatura (Bisezione)
27 % Usiamo una tolleranza 0.5e-1 fissa per avvicinarci in sicurezza.
28 toll_coarse = 0.5e-1;
29 [x_bis, i_bis] = bisec(a, b, f, toll_coarse);
30
31 %% 3. Fase 2: Raffinamento (Newton)
32 % Il punto di innesco per Newton è l'ultimo risultato della Bisezione.
33 start_point = x_bis(end);
34 [x_newt, i_newt] = newton(start_point, f, df, toll, itmax);
35
36 end

```

Function bisnew

5.2 Risoluzione di Equazioni Algebriche

Valutazione di un polinomio in un punto Le equazioni algebriche sono equazioni del tipo

$$P(x) = 0$$

dove P è un polinomio di grado n , cioè

$$P(x) = a_1 x^n + a_2 x^{n-1} + \cdots + a_n x + a_{n+1}, \quad a_1 \neq 0, \quad a_k \in \mathbb{R}$$

Le seguenti proprietà sono ben note:

- Un polinomio di grado n ha esattamente n radici, che possono essere reali o complesse, ciascuna contata con la sua molteplicità (Teorema Fondamentale dell'Algebra);
- Se un polinomio P ha una radice complessa $\alpha = a + ib$ allora la sua coniugata $\bar{\alpha} = a - ib$ è anche radice di P ;
- Un polinomio di grado dispari ha almeno una radice reale;
- Per $n \geq 5$ non esiste alcuna forma esplicita per le radici di P .

Per determinare un'approssimazione delle radici reali di un'equazione algebrica è possibile utilizzare i metodi visti per le equazioni non lineari, dopo aver individuato per ciascuna

radice reale un intervallo che la contenga. Quando n è molto grande il problema di individuare tali intervalli può risultare non immediato. In generale occorre effettuare delle operazioni preliminari:

- *localizzazione delle radici*: serve a determinare un cerchio del piano complesso che contenga tutte le radici (sia reali che complesse);
- *numerazione delle radici*: consiste nel determinare il numero delle radici reali.

A tale scopo sono utili i seguenti risultati.

Teorema di Cauchy

Teorema 5.3. *Tutti gli zeri di*

$$P(x) = a_1x^n + a_2x^{n-1} + \cdots + a_nx + a_{n+1}$$

sono inclusi nel cerchio del piano di centro l'origine e raggio

$$r = 1 + \max_{2 \leq k \leq n+1} \left| \frac{a_k}{a_1} \right|$$

Regola dei segni di Cartesio Sia

$$P(x) = a_1x^n + a_2x^{n-1} + \cdots + a_nx + a_{n+1}$$

Consideriamo l'insieme ordinato dei coefficienti

$$A = \{a_1, a_2, \dots, a_n, a_{n+1}\}$$

Se denotiamo con ν il numero di variazioni di segno nell'insieme A (gli eventuali zeri non si contano) si ha che il numero k delle radici reali positive di P soddisfa le seguenti proprietà:

$$k \leq \nu \quad \text{e} \quad \nu - k \text{ è un numero pari.}$$

Se applichiamo la regola di Cartesio al polinomio

$$Q(x) = P(-x)$$

otteniamo il numero delle radici reali positive di Q e quindi il numero delle radici reali negative di P . Applicando i precedenti risultati e disegnando il grafico di P sull'intervallo $[-r, r]$ si stabilisce con certezza il numero delle radici reali di P e si determinano tanti intervalli quante sono le radici di P . Si può così procedere al calcolo approssimato delle radici reali di P utilizzando i metodi visti per le equazioni non lineari.

Valutazione di un polinomio in un punto I metodi di Bisezione e Newton per il calcolo approssimato delle radici di una equazione algebrica $P(x) = 0$, richiedono la valutazione del polinomio P e della sua derivata prima P' nelle successive approssimazioni $x_0, x_1, \dots, x_n, \dots$ della radice $\bar{x}$. Consideriamo un polinomio di grado n a coefficienti reali:

$$P(x) = a_1x^n + a_2x^{n-1} + \cdots + a_nx + a_{n+1}$$

Il nostro obiettivo è costruire l'algoritmo più efficiente per valutare P in un punto fissato $\bar{x}$.

Algoritmo 1: Metodo Immediato L'approccio più diretto consiste nel calcolare ogni termine della sommatoria singolarmente:

$$P(x) = \sum_{i=0}^n a_{i+1} x^{n-i}$$

In questo metodo, ogni potenza x^k viene calcolata da zero moltiplicando x per se stesso $k - 1$ volte. Questo comporta un costo computazionale elevato per gradi n grandi.

- Operazioni moltiplicative: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
- Operazioni additive: n

Algoritmo 2: Metodo con Accumulazione delle Potenze È possibile migliorare l'efficienza evitando di ricalcolare le potenze ogni volta. Riscriviamo il polinomio come:

$$P(x) = a_{n+1} + \sum_{i=0}^{n-1} a_{n-i} x^{i+1}$$

L'idea chiave è quella di accumulare la potenza di x : invece di calcolare x^{i+1} da capo, si utilizza il valore della potenza precedente x^i e lo si moltiplica per x (cioè $x^{i+1} = x \cdot x^i$). In questo modo, il numero di moltiplicazioni necessarie diminuisce drasticamente, passando da una crescita quadratica a una lineare.

- Operazioni moltiplicative: $2n$ (una per aggiornare la potenza e una per il coefficiente ad ogni passo)
- Operazioni additive: n

Esiste un algoritmo, detto **Algoritmo di Horner**, che permette di valutare P in un punto con un costo computazionale pari a n operazioni moltiplicative e n operazioni additive.

Algoritmo di Horner Per semplificare, supponiamo che P sia di grado 3:

$$P(x) = a_4 + a_3x + a_2x^2 + a_1x^3$$

Mettendo in evidenza la x otteniamo:

$$P(x) = a_4 + x(a_3 + x(a_2 + xa_1))$$

Riepilogando, ponendo

$$b_1 := a_1$$

$$b_2 := a_2 + \bar{x}b_1$$

$$b_3 := a_3 + \bar{x}b_2$$

$$b_4 := a_4 + \bar{x}b_3$$

si ottiene $P(\bar{x}) = b_4$. In generale, per valutare il polinomio $P(x)$ in un punto $\bar{x}$, ponendo:

$$b_1 := a_1$$

$$b_i := a_i + \bar{x}b_{i-1}, \quad i \in \{2, 3, \dots, n+1\}$$

si ha

$$P(\bar{x}) = b_{n+1}.$$

Schema Algoritmo

```

START main
in n, a, x
b(1) = a(1)
for i = 2; i <= n; i++
    b(i) = a(i) + x * b(i-1)
px = a(n+1) + x * b(n)
out px, b
END

```

Costo Computazionale n operazioni moltiplicative e n operazioni additive.

```

1 function [p, b] = hornerP(a, t)
2 % HORNERP Esegue la divisione sintetica di P(x) per (x-t).
3 % [p, b] = HORNERP(a, t)
4 %
5 % Input:
6 %   a - Vettore dei coefficienti ordinati dal grado massimo al minimo
7 %   t - Punto in cui valutare il polinomio (scalare).
8 %
9 % Output:
10 %   p - Resto della divisione (ovvero P(t)).
11 %   b - Coefficienti del polinomio quoziente Q(x) di grado n-1
12
13 % Numero di coefficienti
14 m = length(a);
15
16 % Pre-allocazione di b per velocità (grado n-1)
17 b = zeros(1, m-1);
18
19 % Il primo coefficiente del quoziente è uguale al primo di P(x)
20 b(1) = a(1);
21
22 % Calcolo dei coefficienti del quoziente (Regola di Ruffini)
23 for i = 2:m-1
24     b(i) = b(i-1) .* t + a(i);
25 end
26
27 % Calcolo del resto finale
28 p = b(m-1) .* t + a(m);
29
30 end

```

Function hornerP

Calcolo della derivata Vediamo ora che, contestualmente al calcolo di $P(\xi)$ è possibile calcolare anche $P'(\xi)$. Si può dimostrare facilmente che

$$P(x) = Q(x)(x - \xi) + P(\xi)$$

dove

$$Q(x) = b_1 x^{n-1} + b_2 x^{n-2} + \dots + b_{n-1} x + b_n$$

Ricordando la formula di Taylor:

$$P(x) = P(\xi) + P'(\xi)(x - \xi) + \frac{(x - \xi)^2}{2!} P''(\xi) + \dots + \frac{(x - \xi)^n}{n!} P^{(n)}(\xi)$$

possiamo scrivere

$$Q(x) = \frac{P(x) - P(\xi)}{x - \xi} = P'(\xi) + \frac{(x - \xi)}{2!}P''(\xi) + \dots$$

Facendo il limite per $x \rightarrow \xi$ otteniamo

$$Q(\xi) = \lim_{x \rightarrow \xi} \frac{P(x) - P(\xi)}{x - \xi} = P'(\xi)$$

Dunque per calcolare $P'(\xi)$ basta calcolare $Q(\xi)$. Lo possiamo fare utilizzando ancora l'algoritmo di Horner sui coefficienti b_i . Infatti, ricordando la definizione di $Q(x)$ e ponendo:

$$\begin{cases} c_1 := b_1 \\ c_i = c_{i-1}\xi + b_i, \quad i \in \{2, \dots, n\} \end{cases}$$

si ha

$$P'(\xi) = Q(\xi) = c_n.$$

```

1 function p = hornerR(c, r, x)
2 % HORNERR Calcola le prime r derivate del polinomio P(x) nel punto x.
3 %   p = HORNERR(c, r, x) restituisce un vettore con i valori delle derivate.
4 %
5 %   Input:
6 %       c : Coefficienti del polinomio (grado decrescente).
7 %       r : Ordine massimo della derivata (es. 2 per derivata seconda).
8 %       x : Punto di valutazione.
9 %
10 %   Output:
11 %       p : Vettore dei risultati dove p(k+1) è la derivata k-esima.
12 %          p(1) = P(x), p(2) = P'(x), ..., p(r+1) = P^(r)(x).
13
14 for i = 0 : r
15     % Esegue Ruffini: ottiene il resto (val) e il quoziente aggiornato (c)
16     % Nota: 'c' viene sovrascritto con i coeff. del quoziente per il
17     prossimo giro
18     [val, c] = hornerP(c, x);
19
20     % Formula di Taylor inversa: P^(i)(x) = Resto * i!
21     p(i+1) = val * factorial(i);
22
23 end

```

Function hornerR

Schema Algoritmo L'algoritmo per calcolare $P(\xi)$ e $P'(\xi)$ è dunque:

```

p = a(1);
dp = p;
for i = 2:n
    p = p * x + a(i);
    dp = dp * x + p;
end
p = p * x + a(n+1);

```


Costo Computazionale Il suo costo computazionale complessivo è $2n$.

```

1 function [p, dp] = horner(a, t)
2 % HORNER Valuta un polinomio e la sua derivata prima usando l'algoritmo di
  Horner.
3 % [p, dp] = HORNER(a, t) calcola il valore del polinomio P(x) e della sua
  derivata P'(x) nel punto (o nei punti) t.
4 %
5 %
6 % Input:
7 %   a - Vettore dei coefficienti ordinati dal grado massimo al minimo
8 %       P(x) = a(1)*x^n + a(2)*x^(n-1) + ... + a(n)*x + a(n+1)
9 %   t - Punto/i in cui valutare il polinomio (scalare o vettore).
10 %
11 % Output:
12 %   p - Valore del polinomio calcolato in t.
13 %   dp - Valore della derivata prima calcolata in t.
14
15 % Numero totale di coefficienti (grado del polinomio n = m - 1)
16 m = length(a);
17
18 % Inizializzazione con il coefficiente di grado massimo
19 p = a(1);
20 dp = p;
21
22 % Ciclo di Horner: aggiorna p e dp iterativamente
23 % Nota: Il ciclo si ferma al penultimo coefficiente
24 for i = 2 : m-1
25     p = p .* t + a(i);      % Aggiorna valore polinomio
26     dp = dp .* t + p;      % Aggiorna derivata usando il nuovo p
27 end
28
29 % Ultimo passo per il polinomio (aggiunta del termine noto)
30 % La derivata è già completa all'uscita del ciclo
31 p = p .* t + a(m);
32
33 end

```

Function horner

Condizionamento delle radici di un polinomio Sia

$$P(x) = a_1x^n + a_2x^{n-1} + \cdots + a_nx + a_{n+1}$$

un polinomio di grado n e siano $\alpha_1, \dots, \alpha_n$ le sue radici. Per effetto della rappresentazione finita, i dati in ingresso del problema (ovvero i coefficienti) sono affetti da un errore che al massimo è l'epsilon macchina e di conseguenza le radici effettivamente calcolate saranno altre. Obiettivo dello studio del condizionamento è misurare, a posteriori, la sensibilità del problema posto. Nello stesso polinomio ci possono essere radici ben condizionate ed altre mal condizionate, cioè lo studio non è globale (tutte le radici) ma puntuale (una sola radice alla volta).

Condizionamento di una radice semplice Sia α una generica radice semplice del polinomio $P(x)$ e sia β la corrispondente radice del polinomio che si ottiene perturbando il k -esimo coefficiente. Si dimostra che:

$$\frac{|\alpha - \beta|}{|\alpha|} \leq \frac{\varepsilon}{|\alpha|} \max_{k \in \{1, \dots, n+1\}} \left| \frac{a_k \alpha^{n-k+1}}{P'(\alpha)} \right|$$

La quantità

$$\frac{1}{|\alpha|} \max_{k \in \{1, \dots, n+1\}} \left| \frac{a_k \alpha^{n-k+1}}{P'(\alpha)} \right|$$

rappresenta l'indice di condizionamento della radice. Si evince che l'errore si amplifica o perché la derivata prima del polinomio computato nella radice è molto piccola (radici quasi doppie) o perché i coefficienti del polinomio sono molto grandi.

Condizionamento di una radice multipla Sia α una generica radice di molteplicità $r > 1$. Si dimostra che:

$$\frac{|\alpha - \beta|}{|\alpha|} \leq \frac{\varepsilon^{1/r}}{|\alpha|} \max_{k \in \{1, \dots, n+1\}} \left| \frac{r! a_k \alpha^{n-k+1}}{P^{(r)}(\alpha)} \right|^{1/r}$$

Quindi il problema è in generale mal condizionato.

```

1 function cond = condRoot(c, r, x)
2 % CONDRROOT Calcola l'indice di condizionamento di una radice.
3 % cond = CONDRROOT(c, r, x) stima quanto la radice x (con molteplicità r)
4 % è sensibile alle perturbazioni sui coefficienti del polinomio.
5 %
6 % Input:
7 %   c - Vettore dei coefficienti (dal grado massimo al noto).
8 %   r - Molteplicità della radice x (r=1 per radici semplici).
9 %   x - Il valore della radice.
10 %
11 % Output:
12 %   cond - Indice di condizionamento.
13
14 n = length(c) - 1; % Grado del polinomio
15
16 % Calcola le derivate fino all'ordine r usando Horner Generalizzato
17 p = HornerR(c, r, x);
18
19 y = p(end); % Valore della derivata r-esima: P^(r)(x)
20
21 % Calcola i singoli termini del polinomio: a_k * x^potenza
22 % (Serve per trovare il termine massimo al numeratore)
23 for k = 1 : n+1
24     fatt(k) = c(k) * x^(n - k + 1);
25 end
26
27 % Formula dell'indice di condizionamento per radici multiple:
28 % K = (1/|x|) * [ (r! * max|a_k * x^k|) / |P^(r)(x)| ] ^ (1/r)
29 cond = max(abs(fatt * factorial(r) / y).^(1/r)) / abs(x);
30
31 end

```

Function condRoot

Problema del calcolo simultaneo di tutti gli zeri Esistono diversi approcci:

1. **Deflazione:** Si calcola una radice x_1 e si costruisce il quoziente $P_1(x) = P(x)/(x - x_1)$. Si itera il procedimento. *Difetti:* L'errore di arrotondamento si accumula in modo inaccettabile fornendo risultati fasulli dopo le prime radici.
2. **Matrice Companion:** Si riduce il calcolo degli zeri di P al calcolo degli autovalori della matrice companion di P , che ha P come polinomio caratteristico.

$$P_n(x) = a_1 x^n + \dots + a_n x + a_{n+1}$$

$$A = \begin{pmatrix} -\frac{a_2}{a_1} & -\frac{a_3}{a_1} & \cdots & -\frac{a_n}{a_1} & -\frac{a_{n+1}}{a_1} \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}$$

È quello che fa la function `roots` del Matlab. *Difetti:* Il metodo calcola con precisione gli autovalori della matrice, ma ciò non implica una soddisfacente approssimazione degli zeri di P , poiché i due problemi hanno un differente condizionamento.

Implementazione degli algoritmi per equazioni algebriche L'analisi svolta nei paragrafi precedenti sulla stabilità della valutazione polinomiale (Algoritmo di Horner) e sul calcolo delle derivate (Horner Generalizzato) trova la sua naturale applicazione nella ricerca delle radici. A differenza delle routine generiche per funzioni non lineari $f(x)$ qualsiasi (viste nei capitoli precedenti), le seguenti implementazioni sono ottimizzate specificamente per equazioni algebriche della forma $P_n(x) = 0$. L'utilizzo dello schema di Horner per valutare sia $P(x)$ che $P'(x)$ garantisce non solo una maggiore robustezza numerica contro gli errori di arrotondamento, ma anche un costo computazionale ridotto ($2n$ operazioni per ottenere valore e derivata). Di seguito presentiamo tre varianti algoritmiche:

bisecAlg Una specializzazione del metodo di *Bisezione* che utilizza l'algoritmo di Horner per la valutazione puntuale del residuo. Questo approccio garantisce la convergenza globale anche per polinomi di grado elevato, dove la valutazione diretta delle potenze x^n potrebbe introdurre errori significativi.

```

1 function [iter, x_vect, itmax] = bisecAlg(a, b, c, TOLL)
2 % BISECALG Trova la radice di un polinomio in [a,b].
3 % [iter, x_vect, itmax] = bisecAlg(a, b, c, TOLL)
4 %
5 % Input:
6 %   a, b - Estremi dell'intervallo di ricerca.
7 %   c     - Coefficienti del polinomio (per Horner).
8 %   TOLL  - Tolleranza richiesta per l'arresto.
9 %
10 % Output:
11 %   iter   - Numero di iterazioni effettuate.
12 %   x_vect - Vettore con la successione delle approssimazioni.
13 %   itmax  - Numero massimo di iterazioni teoriche previste.
14
15 % Calcolo dei valori agli estremi usando Horner
16 fa = Horner(c, a);
17 fb = Horner(c, b);
18
19 % Controllo esistenza dello zero (Teorema degli zeri)
20 if fa * fb >= 0
21     disp('bisecAlg:ZeroThm','La funzione non cambia segno agli estremi dell
22     ''intervallo.'');
23     iter = [];
24     x_vect = [];
25     itmax = [];
26     return;
27 end
28
29 % Calcolo numero massimo di iterazioni teoriche
30 itmax = 1 + floor(log((b - a) / TOLL) / log(2));

```

```

31 % Pre-allocazione del vettore x per efficienza
32 x_vect = zeros(itmax, 1);
33
34 % Prima iterazione
35 x_vect(1) = (a + b) / 2;
36 fx = Horner(c, x_vect(1));
37 iter = 1;
38
39 % Ciclo di bisezione
40 while iter < itmax && abs(fx) >= TOLL
41     iter = iter + 1;
42
43     % Selezione del sotto-intervallo
44     if fa * fx < 0
45         b = x_vect(iter-1);
46     else
47         a = x_vect(iter-1);
48         fa = fx;
49     end
50
51     % Nuova stima
52     x_vect(iter) = (a + b) / 2;
53     fx = Horner(c, x_vect(iter));
54 end
55
56 % Rimuove gli zeri in eccesso se il ciclo è finito prima di itmax
57 x_vect = x_vect(1:iter);
58
59 end

```

Function bisecAlg

newtonAlg Implementazione del metodo di *Newton Modificato*, progettata per gestire radici con molteplicità $m \geq 1$. Questa routine sfrutta la divisione sintetica ripetuta per valutare simultaneamente $P(x)$ e $P'(x)$ nello stesso punto, rendendo il calcolo del passo di aggiornamento $x_{k+1} = x_k - m \frac{P(x_k)}{P'(x_k)}$ particolarmente efficiente.

```

1 function [x_vect, iter] = newtonAlg(x0, c, m, TOLL, itmax)
2 % NEWTONALG Trova la radice di un polinomio con il metodo di Newton.
3 % [x_vect, iter] = newtonAlg(x0, c, m, TOLL, itmax)
4 %
5 % Input:
6 %   x0      : Stima iniziale della radice.
7 %   c       : Vettore dei coefficienti del polinomio (per Horner).
8 %   m       : Molteplicità della radice attesa (m=1 per radici semplici).
9 %   TOLL    : Tolleranza richiesta per l'arresto (su incremento e residuo).
10 %   itmax   : Numero massimo di iterazioni consentite.
11 %
12 % Output:
13 %   x_vect  : Vettore contenente la successione delle approssimazioni.
14 %   iter    : Numero di iterazioni effettuate.
15
16 % Pre-allocazione del vettore delle soluzioni per efficienza
17 x_vect = zeros(itmax, 1);
18
19 % Inizializzazione
20 x_vect(1) = x0;
21 iter = 1;
22 err = 1; % Errore relativo iniziale arbitrario > TOLL

```

```

23
24 % Calcolo valore e derivata nel punto iniziale usando Horner
25 [fx, dfx] = Horner(c, x_vect(1));
26
27 % Ciclo di Newton
28 % Criteri di arresto: max iterazioni, errore relativo, residuo
29 while (iter < itmax) && (err > TOLL) && (abs(fx) > TOLL)
30
31     % Controllo per evitare divisione per zero (derivata nulla)
32     if dfx == 0
33         disp('Arresto: La derivata si è annullata.');
```

è 0

```

34         break;
35     end
36
37     % Passo di Newton modificato per la molteplicità m
38     %  $x_{k+1} = x_k - m * (f(x_k) / f'(x_k))$ 
39     x_vect(iter + 1) = x_vect(iter) - m * (fx / dfx);
40
41     % Calcolo dell'errore relativo stimato
42     if x_vect(iter + 1) ~= 0
43         err = abs(x_vect(iter + 1) - x_vect(iter)) / abs(x_vect(iter + 1));
44     else
45         err = abs(x_vect(iter + 1) - x_vect(iter)); % Errore assoluto se x
46     end
47
48     % Aggiornamento per la prossima iterazione
49     iter = iter + 1;
50     [fx, dfx] = Horner(c, x_vect(iter));
51
52 end
53
54 % Ridimensiona il vettore al numero reale di iterazioni fatte
55 x_vect = x_vect(1:iter);
56
57 end

```

Function newtonAlg

bisnewAlg Una strategia *ibrida* che combina i punti di forza dei due metodi precedenti. L'algoritmo avvia una fase di “sgrossatura” tramite Bisezione per avvicinarsi alla radice (entrando nel bacino di attrazione), per poi commutare automaticamente al metodo di Newton (fase di “raffinamento”) e sfruttarne la convergenza quadratica.

```

1 function [x_bis, i_bis, x_newt, i_newt] = bisnewAlg(a, b, c, m, TOLL, itmax)
2 % BISNEWALG Metodo ibrido per polinomi.
3 %
4 % La funzione combina la robustezza della Bisezione (per avvicinarsi alla
5 % radice) con la velocità di Newton (per affinare il risultato), specifica
6 % per equazioni algebriche  $P(x)=0$ .
7 %
8 % Input:
9 %     a, b : Estremi dell'intervallo iniziale.
10 %     c     : Vettore dei coefficienti del polinomio.
11 %     m     : Molteplicità della radice (m=1 per radici semplici).
12 %     TOLL  : Tolleranza richiesta per la fase finale (Newton).
13 %     itmax : Max iterazioni per la fase finale (Newton).
14 %
15 % Output:

```

```

16 %      x_bis  : (vector) Storia delle approssimazioni (Bisezione).
17 %      i_bis  : (int)   Numero iterazioni (Bisezione).
18 %      x_newt : (vector) Storia delle approssimazioni (Newton).
19 %      i_newt : (int)   Numero iterazioni (Newton).
20
21 %% 1. Validazione Input
22 arguments
23     a      (1,1) double
24     b      (1,1) double
25     c      (1,:) double % Vettore riga dei coefficienti
26     m      (1,1) double {mustBePositive} = 1
27     TOLL   (1,1) double {mustBePositive} = 1e-6
28     itmax  (1,1) double {mustBeInteger, mustBePositive} = 100
29 end
30
31 %% 2. Fase 1: Sgrossatura (Bisezione)
32 % Usiamo una tolleranza 0.5e-1 fissa per avvicinarci in sicurezza.
33 toll_coarse = 0.5e-1;
34 [i_bis, x_bis, ~] = biseqAlg(a, b, c, toll_coarse);
35
36 %% 3. Fase 2: Raffinamento (Newton)
37 % Il punto di innesco per Newton è l'ultima approssimazione della Bisezione
38 start_point = x_bis(end);
39 [x_newt, i_newt] = newtonAlg(start_point, c, m, TOLL, itmax);
40
41 end

```

Function bisnewAlg

5.3 Esercizi

1. Scrivere una function Matlab che implementi il metodo di bisezione.
2. Scrivere una function Matlab che implementi il metodo di Newton.
3. Scrivere una function Matlab che implementi il metodo combinato bisezione-Newton.
4. Data l'equazione

$$F(x) = \cos(x) - 4x.$$

- Individuare un intervallo che contenga lo zero della funzione.
- Approssimare lo zero con la precisione di macchina utilizzando il metodo di Bisezione. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dal metodo.
- Approssimare lo zero con la precisione di macchina utilizzando il metodo di Newton. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dal metodo.
- Approssimare lo zero con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dai metodi.

5. Data l'equazione

$$F(x) = e^x + \frac{x}{10}.$$

- Individuare un intervallo che contenga lo zero della funzione.

- Approssimare lo zero con la precisione di macchina utilizzando il metodo di Bisezione. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dal metodo.
- Approssimare lo zero con la precisione di macchina utilizzando il metodo di Newton. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dal metodo.
- Approssimare lo zero con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dai metodi.

6. Data l'equazione

$$F(x) = x + \log(x^3).$$

- Individuare un intervallo che contenga lo zero della funzione.
- Approssimare lo zero con la precisione di macchina utilizzando il metodo di Bisezione. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dal metodo.
- Approssimare lo zero con la precisione di macchina utilizzando il metodo di Newton. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dal metodo.
- Approssimare lo zero con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dai metodi.

7. Supponiamo che una reazione chimica origini ad un certo istante t una concentrazione di un particolare ione data dalla legge:

$$c(t) = 7e^{-5t} + 3e^{-2t}.$$

Se all'istante iniziale la concentrazione iniziale è $c(0) = 10$, a quale istante $\bar{t}$ la concentrazione iniziale si sarà dimezzata, ossia

$$c(\bar{t}) = 5?$$

- Tenendo conto che il problema è equivalente a quello di determinare lo zero dell'equazione

$$F(t) = 7e^{-5t} + 3e^{-2t} - 5 = 0,$$

individuare un intervallo del semiasse positivo che contenga lo zero della funzione F .

- Approssimare lo zero con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton. Riportare il valore approssimato dello zero e il numero di iterazioni effettuate dai metodi.

- Calcolare $\sqrt{33}$ con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton.
- Calcolare $1/43$ con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton.
- Scrivere una function Matlab che implementi il metodo di bisezione per equazioni algebriche.

11. Scrivere una function Matlab che implementi il metodo di Newton per equazioni algebriche.
12. Scrivere una function Matlab che implementi il metodo combinato bisezione-Newton per equazioni algebriche.
13. Scrivere una function Matlab che implementi l'algoritmo di Horner per il calcolo del valore di un polinomio e della sua derivata in un punto.
14. Scrivere una function Matlab che calcoli gli indici di condizionamento delle radici semplici e multiple.

15. Sia

$$P(x) = x^6 - x - 1.$$

- Approssimare le radici reali di P con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton. Qual è il numero di iterazioni del metodo di bisezione? Qual è il numero di iterazioni del metodo di Newton?
- Studiare il condizionamento delle radici reali di P . Riportare il valore delle radici con le cifre che si possono ritenere corrette.

16. Sia

$$P(x) = x^9 + 2x^8 - 3x^7 + x^6 + x^4 - 2x^2 + x - 1$$

- Approssimare le radici reali di P con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton. Qual è il numero di iterazioni del metodo di bisezione? Qual è il numero di iterazioni del metodo di Newton?
- Studiare il condizionamento delle radici reali di P . Riportare il valore delle radici con le cifre che si possono ritenere corrette.

17. Sia

$$P(x) = 2x^9 - 3x^8 + 4x^5 + \frac{1}{2}x^4 - x^3 + x - \frac{1}{2}$$

- Individuare l'intervallo che contiene tutte le radici reali.
- Quante sono le radici reali? Che molteplicità hanno? Trovare per ciascuna radice reale un intervallo che la contiene.
- Approssimare le radici reali di P con la precisione di macchina utilizzando il metodo combinato di bisezione-Newton. Qual è il numero di iterazioni del metodo di bisezione? Qual è il numero di iterazioni del metodo di Newton?
- Studiare il condizionamento delle radici reali di P . Riportare il valore delle radici con le cifre che si possono ritenere corrette.

18. Sia

$$P(x) = x^7 - 3x^6 + 2.25x^5 - x^3 + 3.5x^2 - 3.75x + 1.125.$$

- Il polinomio P ha $x = \frac{3}{2}$ come radice doppia. Calcolarne il condizionamento.
- Approssimarla con la precisione di macchina utilizzando il metodo Newton. Qual è il numero di iterazioni?
- Approssimarla con la precisione di macchina utilizzando il metodo Newton opportunamente modificato. Qual è il numero di iterazioni?
- Approssimarla con la function roots del MatLab.

19. Sia

$$P(x) = x^5 + 0.631x^4 + 0.676387x^3 - 0.325473867x^2 + 0.04352613299999995x - 0.001860867$$

- Individuare l'intervallo che contiene tutte le radici reali di P .
- Quante sono le radici reali? Che molteplicità hanno? Trovare per ciascuna radice reale un intervallo che la contiene.
- Approssimare le radici reali di P con la precisione di macchina utilizzando opportunamente i metodi studiati. Qual è il numero di iterazioni del metodo utilizzato?
- Studiare il condizionamento delle radici reali di P . Riportare il valore delle radici con le cifre che si possono ritenere corrette.

20. Sia

$$P(x) = x^8 - 4.01x^7 + 4.02x^6 + x^3 - 3.01x^2 + 0.01x + 4.02.$$

- Individuare l'intervallo che contiene tutte le radici reali di P .
- Quante sono le radici reali? Che molteplicità hanno? Trovare per ciascuna radice reale un intervallo che la contiene.
- Approssimare le radici reali di P con la precisione di macchina utilizzando opportunamente i metodi studiati. Qual è il numero di iterazioni del metodo utilizzato?
- Studiare il condizionamento delle radici reali di P . Riportare il valore delle radici con le cifre che si possono ritenere corrette.

21. Sia

$$P(x) = 2x^8 - 8.02x^7 + 8.04x^6 + x^3 - 3.01x^2 + 0.001x + 4.02$$

- Individuare l'intervallo che contiene tutte le radici reali.
- Quante sono le radici reali? Che molteplicità hanno? Trovare per ciascuna radice reale un intervallo che la contiene.
- Approssimare le radici reali di P con la precisione di macchina utilizzando opportunamente i metodi studiati. Qual è il numero di iterazioni del metodo utilizzato?
- Studiare il condizionamento delle radici reali di P . Riportare il valore delle radici con le cifre che si possono ritenere corrette.

22. Sia

$$P(x) = x^{10} - 55x^9 + 1320x^8 - 18150x^7 + 157773x^6 - 902055x^5 + 3416930x^4 - 8409500x^3 + 12753576x^2 - 10628640x$$

- Individuare l'intervallo che contiene tutte le radici reali di P .
- Quante sono le radici reali? Che molteplicità hanno? Trovare per ciascuna radice reale un intervallo che la contiene.
- Approssimare le radici reali di P con la precisione di macchina utilizzando opportunamente i metodi studiati. Qual è il numero di iterazioni del metodo utilizzato?
- Studiare il condizionamento delle radici reali di P . Riportare il valore delle radici con le cifre che si possono ritenere corrette.

23. Sia

$$P(x) = x^6 - 2x^5 - 4x^4 + 6x^3 + 7x^2 - 4x - 4$$

- Individuare l'intervallo che contiene tutte le radici reali di P .
- Quante sono le radici reali? Che molteplicità hanno? Trovare per ciascuna radice reale un intervallo che la contiene.
- Approssimare le radici reali di P con la precisione di macchina utilizzando opportunamente i metodi studiati. Qual è il numero di iterazioni del metodo utilizzato?
- Studiare il condizionamento delle radici reali di P . Riportare il valore delle radici con le cifre che si possono ritenere corrette.